

基于时序学习与整数规划的世界杯观赛需求及资源协同优化

摘要

世界杯赛事安排同时连接观赛需求、竞技公平、场馆能力、跨城旅行与赛中资源保障。本文以题目附件、公开赛前信息和可核验的真实赛程为基础，构建“需求预测—联合排程—动态调配—现实比较”的闭环框架；全流程区分赛前、赛中与赛后信息截面，并对输出进行约束复算和敏感性检验。

问题一审计 55 个候选字段的时间血缘，剔除 10 个同场赛后字段并主动舍弃中立场标记，在 560 条训练样本内进行扩窗五折验证，再对 140 条标签未知的时间外样本预测。**Elastic Net** 在当前四类候选中取得最低误差，五折平均 MSE 为 220.0855、平均 **RMSE** 为 14.8121、平均 MAE 为 11.8276；生成的 72 场需求预测均值为 170321280 人次。问题二以 85680 个三维布尔变量建立 **CP-SAT** 联合模型，并用全部候选三元组的确定性包络完成 Min–Max，求得 $Z_2 = 0.484946$ ；三个独立进程跨 Python 3.11/3.12 生成逐字节相同的正式赛程。复算表明 72 场唯一指派，最小休息为 63 小时，场馆负荷为 2–8 场，全部硬约束无违反，证书间隙约 0.41%。

问题三用 20000 组联合蒙特卡洛情景同时模拟 24 场第三轮，保证每次晋级质量为 32；五个随机种子下晋级概率标准差最大为 0.00469。日预算约束下，动态目标由锁定静态方案同界重评的 6.572230 提高至 $Z_3 = 6.637484$ ，仅 8/24 场改变配置；五种子完整重优化的增益为 0.065123–0.065434，动态决策完全一致。问题四复核 2022 年世界杯 48 场小组赛，构建可观测结构与题内代理双层评价：实际与优化方案的最小休息分别为 87 和 63 小时，场均转场距离为 18.9868 和 2019.1075 公里；同构代理旅行指标 D 分别为 0.213022 和 0.201797。共同 Min–Max 边界下的条件性代理总分为 0.277058 与 0.480778，16 组权重扰动均保持优化方向；由于真实合同与运营成本仍不可观测，本文只把该总分解为题内映射下的条件性结果。

模型的优势在于时间边界清晰、预测与优化接口统一、约束可独立复算，并能在共享状态下比较静态与动态决策；局限主要来自高观看尾部样本不足、题定旅行口径和现实运营字段缺失。该框架可推广至其他分组赛事、职业联赛和大型活动，但结论强度必须服从数据与代理的证据边界。

关键字：时序交叉验证 Elastic Net CP-SAT 多目标优化 联合蒙特卡洛 动态资源配置 敏感性分析 最近邻代理 世界杯赛程

一、问题重述

1.1 研究背景

世界杯小组赛的组织并非单一的赛程排列问题。赛事关注度决定转播与票务需求，比赛安排又同时受到场馆容量、时区、球队休息、安保能力和转播资源的约束；进入比赛阶段以后，前两轮赛果还会改变第三轮的晋级形势、需求强度与风险水平。因此，组织者真正需要的是一条从赛前预测延伸到赛中调整、并能够与现实赛程比较的完整决策链。

题目给出由 48 支球队、12 个小组和 72 场比赛构成的模拟世界杯。每组 4 队进行单循环赛，每队参加 3 场；每组前 2 名直接晋级，12 个第三名中排名靠前的 8 队补足 32 强。候选资源包括 16 个场馆和 80 个参考时段，赛程必须满足唯一指派、相邻两轮至少 60 小时、场馆逐日与总负荷、黄金时段公平、安保资格以及同一时段转播容量等规则。

1.2 四个子问题

问题一要求在题定时间划分下预测转播观看人数。训练集 560 场、测试集 140 场，测试标签不可用于任何训练或调参；对历史赛后信息只能构造严格滞后的赛前统计。模型既要输出测试集百万人尺度预测，也要为 72 场模拟小组赛给出人尺度预测，从而为后续传播价值计算提供输入。

问题二需要同时确定每场比赛的场馆和开球时段。优化对象不是先选场馆、再向空闲时段填充，而是比赛、场馆、时段的联合指派；目标综合票务、传播、悬念、吸引力、成本、旅行、公平与风险八个维度。除给出综合目标值外，最终方案还须逐项通过赛程规则复算，才能称为可执行排程。

问题三固定问题二得到的第三轮场馆与时段，仅根据前两轮反馈重新配置转播优先级、安保等级、交通保障和票价。24 场第三轮比赛共享同一信息截面，并在 20,000 次联合比分模拟中统一处理组内排名与跨组第三名竞争。动态方案需与赛前已经优化并锁定的静态方案在相同动作域、相同日约束和相同归一化边界下比较，以识别信息更新带来的真实增益。

问题四选取一届已结束的四队小组制世界杯，将实际小组赛与本文 72 场优化方案放入同一评价函数。由于两项赛事分别有 48 场和 72 场，票务、传播、成本、旅行与风险等总量必须先转换为场均强度。现实数据中的商业和安保字段无法直接观测时，应把代理指标与可观测结构分层报告，不用代理综合分数替代对现实组织约束的判断。

1.3 总体研究路线

四问之间存在明确的输入输出关系：问题一提供赛前观看需求，问题二把需求转化为场馆时段排程；问题三读取该排程的第三轮部分，问题四则复用问题二的评价口径。

围绕这条链路，本文先冻结数据与单位口径，再进行预测、联合优化、动态重配置和现实比较，最后统一开展约束审计与敏感性检验。整体步骤及各阶段交付见图1。



图 1 整体技术路线图

数据合同与约束审计贯穿全部四问，而不是仅在求解结束后补做检查（见图1）。预测结果以两种单位导出，排程结果以 72 行物理方案保存，动态策略以 24 场动作保存，实际赛程以 48 行长表保存；这种逐层固化使后续模型能够直接追溯上游输入。最终评价只复用已经定义的指标和方向，避免为迎合某个方案而重新选择分母或归一化边界。

二、问题分析

2.1 信息边界与预测任务

观看需求受球队实力、比赛阶段、球迷基础、悬念和时区共同影响^[1]，但附件还含有同场进球、射门、上座和观看人数等赛后字段。若把这些字段直接送入模型，验证误差虽会下降，模型却无法在开球前使用。因此，问题一先审计字段的可获得时刻，再讨论回归精度。训练记录按日期排序；同一日期的比赛先生成特征，日末才更新球队历史，验证窗的观看标签在特征重建前整段屏蔽。

模型比较采用五折扩窗验证，每一验证窗都晚于训练窗。该设计比随机划分更接近“用过去预测未来”的部署过程^[2]。候选集包括均值基线、Elastic Net、ExtraTrees 和 CatBoost，分别覆盖线性正则化、随机树集成与有序提升^[3-5]；四类模型使用相同的训练窗和唯一选模指标 MSE，复杂模型只有在时间外证据更好时才入选。

2.2 联合排程与动态调配

问题二同时决定 72 场比赛的场馆和开球时段。当地日负荷随时区变化，球队休息、安保资格、黄金时段公平和日级资源容量又彼此耦合；若先选场馆再排时段，第一阶段可能提前耗尽第二阶段所需的可行组合。本文以三维布尔变量统一表达比赛、场馆与时段，采用 CP-SAT 求解^[6,7]。求解分为可行基线、辅助边界和加权主模型，三个阶段共享同一硬可行域。

票务、传播和成本在硬可行域内求统一边界，并按

$$\text{norm}(x; x_{\min}, x_{\max}) = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (1)$$

缩放。收益项正向进入目标，成本、旅行、公平损失和风险以负号进入；已无量纲的指标不重复缩放。辅助极值若只得到带证书的上下界，主模型使用其保守包络，不能拿主方案自身反标归一化范围。

问题三固定问题二给出的第三轮场馆与时段，在第二轮结束的共同信息截面上更新球队状态。每个蒙特卡洛样本同时生成 24 场比分，再统一完成 12 个小组排序和最佳第三名分配；逐场独立计算会遗漏跨组名额竞争。泊松强度遵循题定更新式^[8]，20000 个联合样本估计晋级概率及条件风险^[9]。随后枚举每场可行资源动作，由日级多选择整数规划在预算和等级上限内每场选一个动作；票价利用单调性取解析上界，不另设粗网格。

2.3 同口径比较与误差传播

问题四把可观测结构与题内代理分开。休息、跨时区、场馆日和实际旅行距离由逐场赛程直接计算；现实赛程缺失的票务、传播、成本和安保字段才使用最近邻代理。实际方案、优化方案和代理扰动场景共同确定一次 Min-Max 包络，随后复用同一评价函数。该做法避免分别归一化制造优势，也符合复合指标应公开方向、权重和敏感性的要求^[10]。

四问的误差不会无条件向下游扩散。问题一的需求误差会改变问题二的软目标排序，却不改变唯一指派、容量、安保和休息等硬约束；问题二的第三轮赛程一旦锁定，问题三只能重配资源，不能借新信息改排比赛；问题四的代理只服务于现实比较，不回写前三问。相应地，灵敏度分析分别检验时间外误差、排程权重、状态参数和代理映射，并按各自影响范围解释结果。

2.4 接口与验证合同

表1概括四问的输入输出和独立验收条件。数据事实、同域模型比较和代理情景属于不同证据层：前者可以直接陈述，第二类必须注明候选域与评价函数，代理综合分只能解释为给定映射下的条件性结果。

表 1 四问接口与独立验收条件

问题	输入边界	输出接口	独立验收
一	开球前字段与严格滞后历史	140 条测试预测、72 场需求	主键、行数、单位、有限正值与扩窗误差
二	赛前需求、题定候选域与资源规则	比赛-场馆-时段长表	唯一指派、UTC 冲突、休息、容量、安保、日负荷与公平
三	固定第三轮赛程及第二轮后共同状态	24 场动态资源动作	每样本 32 个晋级名额、日预算、资源上限及动态不劣于静态
四	真实逐场记录与显式代理映射	结构指标和条件性综合分	单位、方向、共同分母、共同边界与代理标记

任一硬验收失败都应回到相应数据或模型接口重算。求解器状态、图形外观或补充说明不能覆盖物理不可行或信息越界。

三、模型假设

表2只列求解所必需、而题目未直接给定的边界。每项假设都对应程序口径和失效条件；新证据若越过这些条件，必须重算相关模型，不能只修改文字解释。

表2 模型假设、固定口径与失效处理

对象	固定口径	失效时的处理
预测信息集	560/140 条记录构成时间外边界；测试标签不参与训练、调参、早停或缺失值统计。同场赛后字段只可进入严格滞后的球队历史，同日比赛在日末统一更新。内部以百万人拟合，正式赛程需求再乘 10^6 转为人。	字段获得时间晚于开球、验证窗标签进入滞后量或单位不一致时，重新构造特征并完成扩窗验证。
联合排程域	比赛、场馆和时段是一个联合决策。当地日期由 UTC 转换；第二、三轮旅行以上一轮全部安保合格候选场馆为起点取均值，遵答题定口径。归一化边界来自同一硬可行域；未证最优的辅助问题采用 incumbent 与 best bound 的保守包络。	只有题方更改旅行定义、候选场馆或硬约束时才重建域；不得把现实中未提供的禁用日或合同条件暗设为硬约束。
第三轮状态	24 场共享第二轮结束后的同一状态截面。每个样本联合生成全部比分并统一排名，跨线同分块按比例分配，使晋级名额质量恒为 32；更新后的泊松均值截断在 $[0.15, 4.50]$ 。	若状态边界长期饱和，应重估状态函数；增加模拟次数不能修复参数错配。逐场滚动更新只能作为另一个情景。
静态与动态方案	静态动作按赛前信息优化后锁定；动态动作只在更新环境中重选资源。两者使用共同动作域、价格边界、归一化边界和日预算，因此动态最优值不得低于锁定静态值。 $-1 < \varepsilon_i < 0$ 时票价取解析右端点。	若出现 $\varepsilon_i \leq -1$ ，票价改用有界一维搜索；同界下动态值更低则说明实现或接口错误。
现实比较	2022 赛程取自保存的 OpenFootball 记录并按 FIFA 页面核验，球队实力使用开赛前完整 Elo 快照。小时、公里、场馆日等结构量直接复算；商业、成本和安保缺失量仅在题内最近邻代理层使用。	代理集中或邻居替换只降低综合分的证据强度，不能改写结构事实；补齐真实运营数据后须重做代理层。
随机性与复现	主种子为 20260813，晋级模拟一次生成 20000×24 样本；CP-SAT 单线程运行，正式 CSV 再由独立程序复算硬约束。	样本名额不守恒、硬约束失败或不同环境输出不一致时，停止解释结果并回查数据与求解接口。

这些假设不把未知事实伪装成观测。题定口径用于主模型，现实缺失量保留代理标记；因此论文中的结论强度始终受数据来源和可复算范围约束。

四、符号说明

表3保留跨章节反复使用的符号，其余局部量在首次出现处定义。问题四的实际旅行距离记为 d^{path} ，问题二的复合旅行指标记为 D ，两者物理含义不同。

表 3 主要符号、单位与范围

符号	含义	单位或范围
$Y_j, \hat{Y}_j, e_j$	历史观看人数、预测值与时间外残差	百万人
$X_j(t^-), \mathcal{H}_t^-$	开球前特征向量与严格滞后历史信息集	混合
α, ρ	Elastic Net 惩罚强度与一范数占比	$> 0, [0, 1]$
x_{ivs}, y_v	比赛 i 指派到场馆 v 、时段 s ；场馆启用标记	$\{0, 1\}$
K_v, τ_s	场馆容量与 UTC 开球时刻	人、小时
L_i^{req}, L_v^{sec}	比赛最低安保需求与场馆安保能力	等级 1–4
N_i^{pre}, V_i^{pre}	赛前现场需求与转播需求	人
T, B, U, H	票务、传播、悬念与吸引力指标	$[0, 1]$
C, D, F, R	成本、复合旅行、公平损失与风险指标	$[0, 1]$
Z_2	72 场联合排程的综合目标	无量纲
G_t, P_t, xGF_t, xGA_t	球队前两轮场数、积分、预期进球与预期失球	场、分、球
RC_t, h_t, S_t	红牌数、伤病等级与竞技状态	张、 $[0, 3]$ 、 $[0, 1]$
$\lambda_{ia}, \lambda_{ib}$	状态更新后的双方泊松进球均值	球/场， $[0.15, 4.50]$
p_t, p_t^D, p_t^W	无条件晋级概率及平局、获胜条件概率	$[0, 1]$
Q_i	比赛的晋级重要性	$[0, 1]$
r_t^N, r_t^V	球队现场与转播需求反馈比值	$[0.75, 1.25]$
$A_i, \tilde{N}_i, \tilde{V}_i$	更新后的吸引力、现场需求与转播需求	$[0, 1]$ 、人
b_i, q_i, l_i, δ_i	转播、安保、交通等级与票价调整比例	离散， $[-15\%, 20\%]$
TV_i, BV_i, C_i	票务价值、转播价值与资源成本	美元、指数
$R_i^{stake}, R_i^{col}, R_i$	无激励风险、默契风险与综合风险	$[0, 1]$
z_{ia}, Z_3	第三轮动作选择变量与动态资源目标	$\{0, 1\}$ 、无量纲
$r_{tj}, d_{tj}^{path}, c_{tj}$	相邻比赛休息、实际场馆距离与跨时区标记	小时、公里、 $\{0, 1\}$
M_k^{act}, M_k^{opt}	实际与优化赛程的第 k 个结构指标	依指标
$D_i^{P^2}$	映射到问题二候选域后的复合旅行负担	$[0, 1]$
Z_4	共同边界下的条件性代理综合分	无量纲

问题二的 $[0, 1]$ 指标来自硬可行域的统一边界；问题四另以实际、优化和代理扰动场景构造共同包络。结构层的小时、公里和比例始终按原单位报告。

五、 问题一：无泄漏观看人数预测

5.1 数据边界与特征构造

目标是在开球前预测转播观看人数。附件 15 张表共 2241 行，其中历史比赛表提供 560 条训练记录和 140 条时间外测试记录，另有 72 场模拟小组赛需要预测。记录按比赛日期和编号稳定排序；同场进球、xG、射门、控球、上座与观看人数不作直接特征，只把严格早于开球日期的球队记录汇总为最近 5 场均值和半衰期 5 场的指数加权均值。同

一日期的比赛先生成特征，日末再统一更新历史。

每个外层时间折都独立完成缺失值填补、类别编码、标准化和模型拟合。最早外折的训练窗内部完成一次超参数选择，此后冻结参数并在五个外折上重拟合；验证窗的观看标签在历史特征重建前整段置空。最终生产特征共 55 列，其中 10 个同场赛后字段被禁止直接进入模型，中立场标记因解释贡献有限而舍弃。未来 72 场使用统一的赛前快照，不调用问题二尚未确定的场馆和时段。

对比赛 j ，双方某项实力量 E 写成均值与绝对差：

$$E_j^{\text{mean}} = \frac{E_{ja} + E_{jb}}{2}, \quad E_j^{\text{gap}} = |E_{ja} - E_{jb}|. \quad (2)$$

赔率先去除水位，再计算三种赛果概率的熵：

$$\pi_{jc} = \frac{o_{jc}^{-1}}{\sum_k o_{jk}^{-1}}, \quad H_j^{\text{odds}} = - \sum_{c \in \{a, D, b\}} \pi_{jc} \log \pi_{jc}. \quad (3)$$

任一特征的最晚来源时刻都必须早于开球时刻；赛事阶段、时区和预先指定交互只作条件预测量，不作因果解释。

5.2 样本分布与候选模型

表4给出核心变量尺度。训练集观看人数均值为 145.64 百万人，中位数为 142.11 百万人，四分位区间为 [128.43, 160.62]；最大值 225.41 百万人使分布呈右尾。图2显示密度峰约在 140 百万人，右尾连续延伸，并非少数孤立异常值。故本文保留高观看样本，不对目标截断；MSE 选模会放大尾部误差，后续用时间外残差单独描述其不确定性。

表 4 问题一核心变量描述性统计

变量	样本数	均值	标准差	最小值	P25	中位数	P75	最大值
观看人数	560	145.64	23.56	93.96	128.43	142.11	160.62	225.41
平均 Elo	560	1756.86	120.58	1472.50	1671.00	1758.00	1846.50	2025.50
平均排名	560	24.30	9.97	2.50	17.00	24.00	31.50	47.50
平均球迷基础	560	73.41	10.47	46.30	66.05	73.03	81.93	94.40
赔率熵	560	1.00	0.10	0.71	0.95	1.04	1.08	1.10

注：观看人数单位为百万人；其余变量采用源数据或赛前构造口径。

平均 Elo、平均排名、平均球迷基础与观看人数的线性相关系数约为 0.62、−0.61 和 0.69，赔率熵约为 0.31。平均 Elo 与平均排名高度反向，特征之间存在明显共线；同一实力区间内的观看人数仍较分散，单变量也不足以解释需求。候选模型因此包括均值基线、Elastic Net、ExtraTrees 和 CatBoost，用同一时间折比较线性正则化与非线性机制。

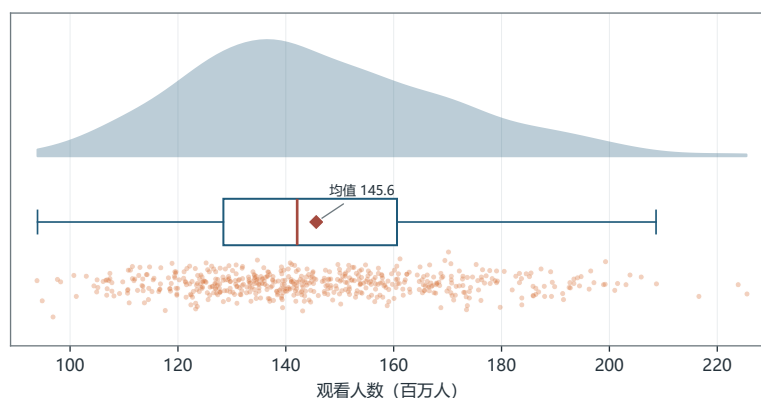


图 2 训练集观看人数分布与右尾特征

Elastic Net 的估计量为

$$\hat{\beta} = \arg \min_{\beta} \left\{ \frac{1}{n} \sum_{j=1}^n (Y_j - X_j^T \beta)^2 + \alpha \left[\rho \|\beta\|_1 + \frac{1-\rho}{2} \|\beta\|_2^2 \right] \right\}. \quad (4)$$

网格取 $\alpha \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$ 、 $\rho \in \{0.1, 0.5, 0.9\}$ ，最终选择 $\alpha = 1.0, \rho = 0.9$ 。ExtraTrees 固定 600 棵树并比较叶节点下限与特征比例；CatBoost 最多训练 3000 轮，在折内末 20% 时间段早停。

5.3 时间外验证与模型选择

表5显示，Elastic Net 的平均 MSE 为 220.086，低于 CatBoost 的 234.209 和 ExtraTrees 的 250.162；相对均值基线 581.203，MSE 降低约 62.1%。RMSE 与 MAE 给出同一排序，说明选模结果不依赖单一损失口径。五折逐折误差及其稳定性在第九、节继续检验。

表 5 候选模型时间外误差汇总

模型	平均 MSE	平均 RMSE	平均 MAE
Elastic Net	220.086	14.812	11.828
CatBoost	234.209	15.267	12.184
ExtraTrees	250.162	15.778	12.734
均值基线	581.203	24.099	19.333

复杂模型没有抵消小样本方差：CatBoost 与 ExtraTrees 的 MSE 分别高出 15.967 和 30.076。因此按预先规定的规则选择 Elastic Net，在全部 560 条训练样本上重拟合，并保留 465 条时间外 OOF 预测用于区间和残差分析。

5.4 诊断、解释与预测输出

OOF 拟合见图3。整体 R^2 约为 0.620、RMSE 约为 14.84 百万人；真实值超过约 190 百万人后，预测更常落在 45 度线下方，表明正则化模型对高关注赛事存在收缩性低估。残差均值为 -0.00093 百万人，日期序列在 2019–2024 年间围绕零线波动，没有持续同向漂移；Q-Q 尾部的轻微偏离说明经验区间比正态近似更合适。

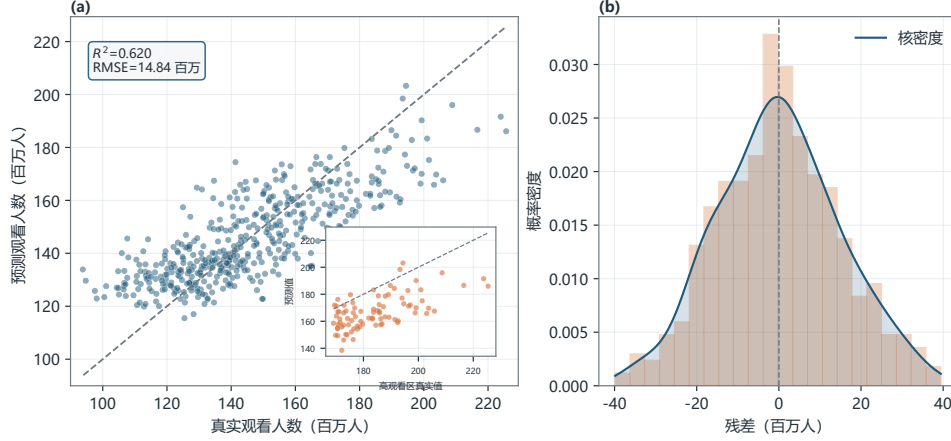


图3 最终模型 OOF 预测拟合与残差分布

OOF 残差的 5% 与 95% 分位数分别为 -22.7075 和 25.1737 百万人。对点预测 $\hat{Y}(x)$ ，本文使用

$$PI_{0.90}(x) = \left[\max\{0, \hat{Y}(x) - 22.7075\}, \hat{Y}(x) + 25.1737 \right] \quad (5)$$

描述历史时间外误差；它不是严格的条件置信区间。

图4给出后 20% 验证窗口的置换重要性和单样本影响。打乱平均球迷基础后 MSE 增加 238.032，高于世界杯标记 32.597、球迷基础–世界杯交互 26.379 和实力–赔率熵交互 24.145。主要信号集中在少数连续特征及预先指定交互，解释了线性正则化为何优于当前非线性候选；这些数值只表示预测贡献，不表示因果效应。

最终模型对 140 条测试记录的预测范围为 115.783–188.216 百万人，均值 145.331 百万人，均为有限正值。72 场模拟小组赛的预测范围为 144154622–205695590 人，均值 170321280 人；两份结果保持模板主键、行数和单位。问题二只接收这 72 场赛前点预测，不读取测试标签或赛后字段。

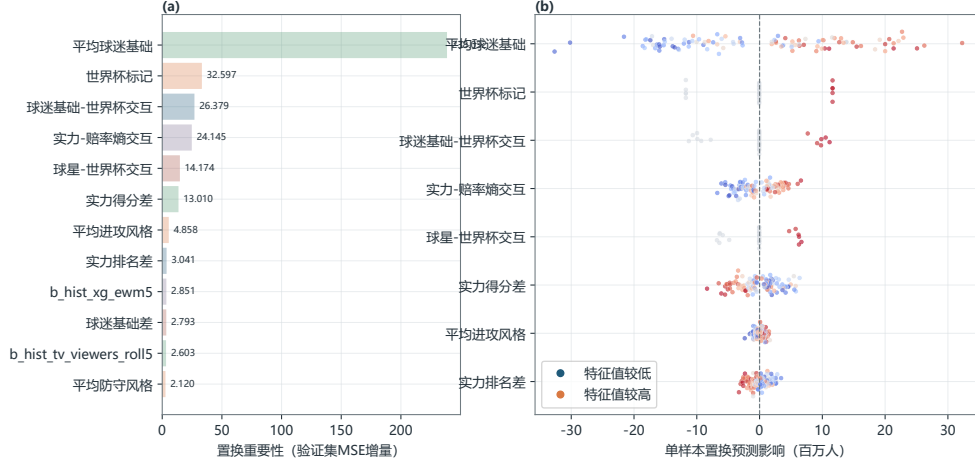


图 4 Elastic Net 置换重要性与单样本预测影响

六、问题二：场馆与开球时段协同优化

6.1 三维指派与候选域

设比赛、场馆和参考时段集合为 $\mathcal{I}, \mathcal{V}, \mathcal{S}$ 。仅当场馆安保能力满足比赛需求时保留候选

$$\Omega = \{(i, v, s) : L_v^{sec} \geq L_i^{req}\}, \quad (6)$$

并建立 $x_{ivs} \in \{0, 1\}$; y_v 表示场馆是否启用。当前数据形成 85680 个三维布尔变量。场馆和时段同时决定当地日期、票务、转播、旅行与风险，因此不把问题拆成两个相互独立的贪心步骤。

比赛 i 在场馆 v 的预计上座为 $N_{iv} = \min(N_i^{pre}, K_v)$ 。票务、转播和成本原值为

$$T^0(x) = \sum_{(i,v,s) \in \Omega} x_{ivs} P_{r(i)}^0 N_{iv}, \quad (7)$$

$$B^0(x) = \sum_{(i,v,s) \in \Omega} x_{ivs} V_i^{pre} u_{r(i)} g_s^{prime} w_{r(i)}^{sponsor}, \quad (8)$$

$$C^0(x, y) = \sum_v 10^6 c_v^{setup} y_v + \sum_{(i,v,s) \in \Omega} x_{ivs} (10^6 c_v^{op} + 10^5 L_i^{req} c_v^{sec}). \quad (9)$$

费用统一换算为美元，场馆启用费只计一次。附件中所有场馆的最小承办数均为 2，故每个可行方案必启用全部 16 个场馆，启用费在本数据中是共同常数。

6.2 旅行、风险与公平指标

旅行严格使用题面特殊口径。距离、旅行时间和时区差先在全部分合法球队-轮次-候选场馆组合上分别 Min-Max，再按 0.5, 0.3, 0.2 合成。第一轮从球队代表地出发；第二、

三轮对球队 t 和本轮场馆 v ，以前一轮全部安保合格场馆为起点：

$$\bar{q}_{trv} = \frac{1}{|\mathcal{V}_{t,r-1}^{qual}|} \sum_{u \in \mathcal{V}_{t,r-1}^{qual}} q_{uv}, \quad q \in \{distance, time, timezone\}. \quad (10)$$

每场取双方平均并对 72 场求均值得到 D ，不能替换成上一轮实际所选场馆的路径。

单场执行风险为

$$r_{iv} = 0.5c_v^{climate} + 0.3\frac{N_{iv}}{K_v} + 0.2\frac{L_i^{req}}{L_v^{sec}}, \quad (11)$$

整套赛程取均值 R 。悬念 U 为 72 场竞技悬念指数均值，吸引力 H 为吸引力指数除以 100 后的均值。若球队 t 的黄金时段和大容量场馆次数分别为 n_t^{prime} , n_t^{large} ，公平惩罚为

$$F = \frac{1}{2} \frac{\max_t n_t^{prime} - \min_t n_t^{prime}}{3} + \frac{1}{2} \frac{\max_t n_t^{large} - \min_t n_t^{large}}{3}. \quad (12)$$

黄金时段取 80 个候选时段中得分最高的 20 个，大容量场馆取 16 个场馆中容量最高的 4 个；阈值并列未跨越名额切点。

6.3 确定性的全部候选组合 Min-Max

旧实现用六个限时辅助排程的对偶最好界作 T, B, C 端点，等价表达式会改变搜索路径和端点，因而不适合作为可复现的评价尺度。本文改为直接枚举全部合法三元候选。对 $k \in \{T, B, C\}$ ，逐场在 Ω_i 中取候选最小和最大后汇总：

$$k_{min} = \sum_{i \in \mathcal{I}} \min_{(v,s) \in \Omega_i} k_i(v, s), \quad k_{max} = \sum_{i \in \mathcal{I}} \max_{(v,s) \in \Omega_i} k_i(v, s), \quad (13)$$

其中 C 的两端同时加入全部必启用场馆的一次性成本。任何满足联合约束的赛程都是该候选笛卡尔域的子集，故其指标必落在包络内；该计算不调用求解器、没有时间限制和随机性。得到

$$T^0 \in [281888385, 285910470], \quad (14)$$

$$B^0 \in [150900200.787, 367190488.581], \quad (15)$$

$$C^0 \in [47730000, 63910000]. \quad (16)$$

三项用 $(k - k_{min})/(k_{max} - k_{min})$ 归一化；若端点相等则按题意置零。 U, H, D, F, R 已按上述定义位于 $[0, 1]$ 。最终目标为

$$\max Z_2 = 0.25T + 0.25B + 0.15U + 0.10H - 0.08C - 0.07D - 0.06F - 0.04R. \quad (17)$$

6.4 硬约束与求解设置

模型逐项施加：每场恰好一次指派；同一场馆同一 UTC 时刻至多一场；球队按三轮顺序参赛且相邻轮至少休息 60 小时；场馆当地日负荷和总负荷满足附件上下限；不

建立安保不合格候选；同一时段比赛数不超过转播容量；第三轮高安保比赛逐参考日不超过动态容量；任意球队黄金时段次数极差不超过 2。核心形式包括

$$\sum_{v,s} x_{ivs} = 1, \quad \sum_i x_{ivs} \leq 1, \quad (18)$$

$$\tau_{t,r+1} - \tau_{tr} \geq 60, \quad \max_t n_t^{prime} - \min_t n_t^{prime} \leq 2. \quad (19)$$

场馆日容量按所选 UTC 时刻转换到场馆 IANA 时区后的当地日期计算，动态资源容量则按附件参考日计算，两类日期不混用。

所有浮点目标系数乘 10^6 转为整数。CP-SAT 固定单线程和随机种子 20260813，最长运行 1800 秒，相对间隙阈值 0.5%；先在宽轮次窗口内求可行提示，再在完整 80 时段域求主目标。Python 3.11 的两次独立进程分别用时约 173 与 177 秒，Python 3.12 的干净环境复现用时约 580 秒；三次均达到相同停止准则并生成逐字节相同的正式 CSV。

6.5 结果与可执行性

主方案原值为 $T^0 = 285581790$ 、 $B^0 = 323423207.006$ 、 $C^0 = 53075000$ 美元；八项标准化指标为

$$(T, B, U, H) = (0.918281, 0.797646, 0.461436, 0.639699), \quad (20)$$

$$(C, D, F, R) = (0.330346, 0.201797, 0.333333, 0.416678).$$

从而 $Z_2 = 0.484946$ ，高于同约束可行基线 0.350742。缩放目标的可行值为 17811471、最好界为 17883841，相对证书间隙约 0.406%；因此 ‘OPTIMAL’ 只按预设 0.5% 停止口径解释，不声称零间隙精确最优。

表 6 问题二关键约束复算与裕度

约束族	实际值	界值	单位	最小裕度	状态
最小轮间休息	63.000	60.000	小时	5.0%	通过
场馆承办下界	0.000	0.000	剩余场	0.0%	通过
场馆承办上界	0.000	0.000	剩余场	0.0%	通过
场馆单日本地容量	1.000	1.000	最大利用率	0.0%	通过
场馆安保等级	0.000	0.000	等级余量	0.0%	通过
时段转播容量	0.000	0.000	剩余场	0.0%	通过
第三轮高安保日容量	0.000	0.000	比例余量	0.0%	通过
黄金时段公平极差	1.000	2.000	场	50.0%	通过

注：裕度为按各约束自身上限归一化后的最小剩余比例；零表示约束在最紧样本上取等。

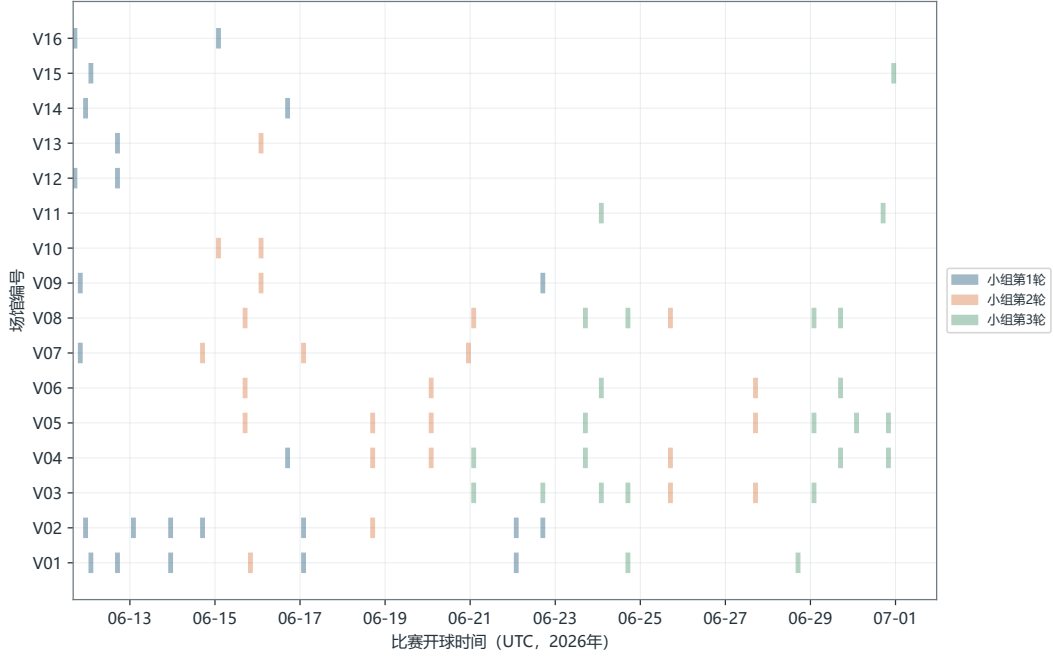


图 5 问题二最终 72 场 UTC 排程

独立从正式 CSV 复算：72 个比赛编号完整且唯一，场馆-UTC 冲突为 0，最小轮间休息为 63 小时；16 个场馆各承办 2-8 场，场馆当地日、安保、时段转播和第三轮高安保容量均无违反。黄金时段次数极差为 1，大容量场馆次数极差为 1。逐场票务、转播、旅行和风险字段与附件公式的最大数值误差低于 10^{-8} ，总目标只写在 CSV 第一条记录。

七、问题三：共同截面下的第三轮动态资源优化

7.1 信息边界与固定赛程

问题三只处理问题二的 24 场第三轮比赛。比赛编号、双方球队、场馆与开球时段全部锁定，决策仅包括转播优先级 b_i 、安保等级 q_i 、交通保障等级 l_i 和票价调整比例 δ_i 。前两轮 48 场反馈先汇总成唯一快照，再一次性模拟全部第三轮；任一第三轮模拟结果都不更新其他第三轮比赛参数。计算依次完成共同截面构造、联合模拟、静动态并行求解和同界复评，不按日或按场滚动更新。

7.2 球队状态、进球均值与联合晋级模拟

球队 t 的前两轮场数、积分和红牌数记为 G_t, P_t, RC_t ，累计预期进球与累计预期失球记为 xGF_t, xGA_t 。严格采用题定状态式与伤病截断：

$$S_t = 0.45 \frac{P_t}{3G_t} + 0.35 \left[0.5 + 0.5 \tanh \left(\frac{(xGF_t - xGA_t)/G_t}{1.25} \right) \right] + 0.20 \exp \left(-0.55 \frac{RC_t}{G_t} \right), \quad (21)$$

$$h_t = \text{clip}(\text{mean}_{m \in \mathcal{M}_t^{1:2}} h_m, 0, 3). \quad (22)$$

对第三轮比赛 $i = (a, b)$ ，令

$$\Delta_i = 0.32(S_a - S_b) - 0.055(h_a - h_b), \quad (23)$$

并在基础进球均值上更新

$$\lambda_{i,a} = \text{clip}(\lambda_{i,a}^0 e^{\Delta_i - 0.04h_a}, 0.15, 4.50), \quad (24)$$

$$\lambda_{i,b} = \text{clip}(\lambda_{i,b}^0 e^{-\Delta_i - 0.04h_b}, 0.15, 4.50). \quad (25)$$

固定 PCG64 种子后，同时生成 20000×24 个独立泊松比分。每个样本先在 12 个小组内依次按积分、净胜球、总进球排序，再把 12 个第三名统一比较并选取最好 8 队。完全同分块若跨越晋级线，就按该块可用名额质量等比例分配。因此每次模拟的晋级质量恒等于 $12 \times 2 + 8 = 32$ ，球队概率满足

$$p_t = \frac{1}{20000} \sum_{m=1}^{20000} w_{tm}^{\text{adv}, qqquad} \sum_{t=1}^{48} p_t = 32. \quad (26)$$

比赛晋级重要性为

$$Q_i = \frac{4p_a(1 - p_a) + 4p_b(1 - p_b)}{2}. \quad (27)$$

同一批样本还按本场平局、A 队胜、B 队胜三个条件计算条件晋级概率，用于后续默契风险，而没有再生成另一套不一致的随机样本。

基准种子下 48 队概率位于 $[0.0023, 1]$ ，数值和为 32.000000；更新进球均值位于 $[0.4752, 3.0146]$ 。五个固定种子下单队概率标准差最大为 0.00469，主要概率位移见图6。

7.3 反馈、需求与风险注册表

对球队 t 的每场前两轮比赛，先把实际/赛前预计现场观众比和转播观众比分别截断到 $[0.60, 1.50]$ ，再取两场均值并截断到 $[0.75, 1.25]$ ，得到 r_t^N, r_t^V 。比赛层反馈取双方几何平均：

$$f_i^N = \sqrt{r_a^N r_b^N}, \quad f_i^V = \sqrt{r_a^V r_b^V}. \quad (28)$$

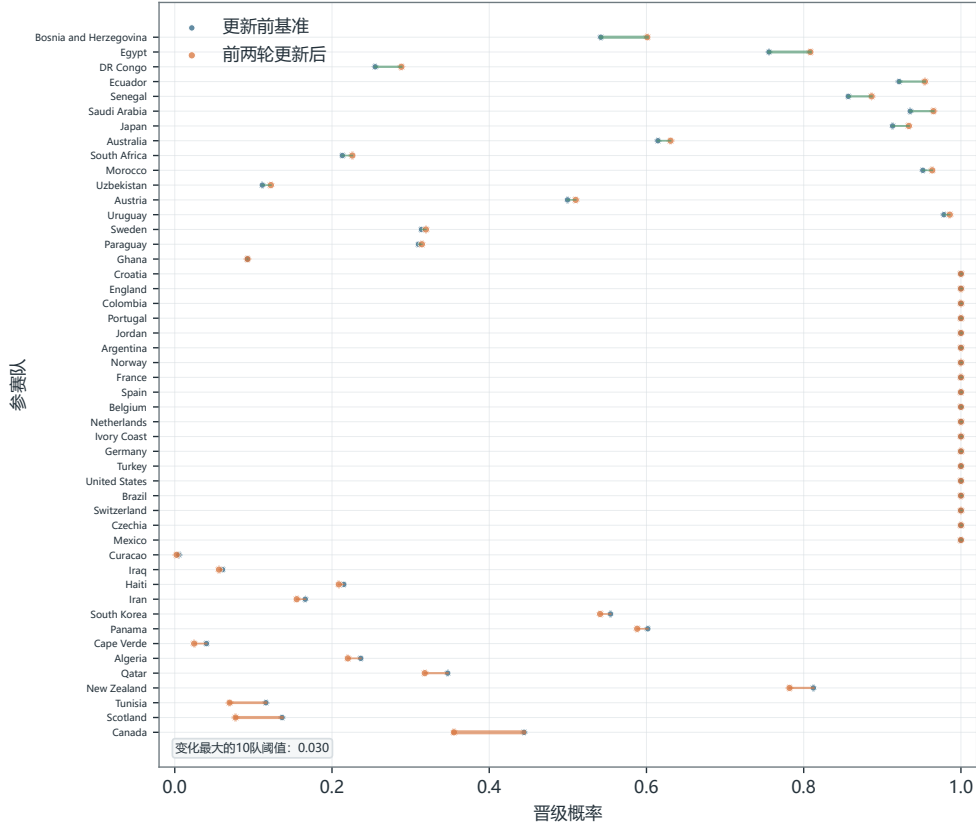


图 6 第三轮开始前的晋级概率更新

令 $S_i = (S_a + S_b)/2$ 、 $H_i = (h_a + h_b)/6$ ，以及

$$F_i = \text{clip}\left(\frac{0.5f_i^N + 0.5f_i^V - 0.75}{0.50}, 0, 1\right), \quad (29)$$

则吸引力、资源配置前现场需求与转播需求依题定公式更新为

$$A_i = \text{clip}(0.50A_i^0 + 0.25Q_i + 0.12F_i + 0.08S_i + 0.05(1 - H_i), 0, 1), \quad (30)$$

$$\tilde{N}_i = N_i^{\text{pre}} f_i^N (0.88 + 0.27Q_i)(0.94 + 0.12S_i)(1 - 0.10H_i), \quad (31)$$

$$\tilde{V}_i = V_i^{\text{pre}} f_i^V (0.87 + 0.30Q_i)(0.90 + 0.20A_i)(1 - 0.06H_i). \quad (32)$$

从资源表读取交通需求倍率 $m_{i_i}^N$ 、转播需求倍率 $m_{b_i}^V$ 、安保剩余风险倍率 $m_{q_i}^R$ 和三类成本。附件给出的第三轮价格弹性为 $\varepsilon = -0.75$ ，票务价值在合法价格域内随 δ 单调增加，故可先解析消去连续变量：

$$\delta_i^* = \min\left\{\delta_{d(i)}^+, 0.88^{1/\varepsilon} - 1\right\}, \quad (33)$$

该值同时满足 $-\delta_{d(i)}^- \leq \delta_i^* \leq \delta_{d(i)}^+$ 。最终现场人数、票务价值、转播人数和转播价值为

$$N_i(\delta_i^*) = \min\{K_i, \tilde{N}_i m_{i_i}^N (1 + \delta_i^*)^\varepsilon\}, \quad TV_i = P_i^0 (1 + \delta_i^*) N_i(\delta_i^*), \quad (34)$$

$$V_i = \tilde{V}_i m_{b_i}^V, \quad BV_i = u_i V_i. \quad (35)$$

程序逐动作验证 $N_i(\delta_i^*) \geq 0.88N_i(0)$ ，没有把需求底线留到导出后修补。

无激励风险、默契风险、动态安保需求与综合风险分别为

$$R_i^{stake} = 0.5(2p_a - 1)^2 + 0.5(2p_b - 1)^2, \quad (36)$$

$$D_i = \min(p_a^D, p_b^D), \quad G_i = 0.5[p_a^W - p_a^D]_+ + 0.5[p_b^W - p_b^D]_+, \quad (37)$$

$$R_i^{col} = \text{clip}(D_i[1 - \text{clip}(G_i, 0, 1)](1 - 0.35R_i^{stake}), 0, 1), \quad (38)$$

$$d_i = \text{clip}\left(0.45d_i^0 + 0.25O_i + 0.15A_i + 0.15\frac{R_i^{stake} + R_i^{col}}{2}, 0, 1\right), \quad (39)$$

$$R_i = 0.40R_i^{stake} + 0.40R_i^{col} + 0.20d_i m_{q_i}^R, \quad qqquad O_i = \text{clip}(\tilde{N}_i/K_i, 0, 1). \quad (40)$$

7.4 静态基线、动态动作与日级约束

每场枚举 $b_i \in \{1, 2, 3\}$ 、 $q_i \in \{L_i^{req}, \dots, L_{v(i)}^{sec}\}$ 、 $l_i \in \{1, 2, 3\}$ ，票价取上式解析最优值。资源成本为 $C_i = c_{b_i} + c_{q_i} + c_{l_i}$ 。在全部更新动作上一次计算 TV, BV, A, C, R 的共同 Min-Max 边界，目标为

$$\max Z_3 = \sum_{i \in I_3} (0.35TV'_i + 0.35BV'_i + 0.10A'_i - 0.10C'_i - 0.10R'_i). \quad (41)$$

对每个实际出现的参考日 d ，同时施加

$$\sum_{i \in I_d} \mathbf{1}(b_i = 3) \leq H_d^B, \quad \sum_{i \in I_d} \mathbf{1}(q_i \geq 3) \leq H_d^S, \quad (42)$$

$$\sum_{i \in I_d} \mathbf{1}(l_i = 3) \leq H_d^L, \quad \sum_{i \in I_d} C_i \leq B_d. \quad (43)$$

静态方案不是默认档位：它先用赛前需求、赛前吸引力及题定赛前风险，在完全相同的动作集合和日约束下独立优化；前两轮结束后四项决策锁定，只把该动作放入更新环境重评。动态方案则在更新环境中重新优化。两者最终均使用更新动作域的同一组边界计分，故 Z_3^{static} 与 $Z_3^{dynamic}$ 可直接比较；模型中不存在“变更惩罚”或另一套分资源预算。

表 7 问题三逐日动态资源使用与容量

日期	高转播	转播上限	高安保	安保上限	强化交通	交通上限	预算使用	预算上限	状态
2026-06-20	2	4	1	3	2	4	35.0	120	通过
2026-06-22	1	3	0	3	0	3	13.0	100	通过
2026-06-23	3	3	3	3	3	3	86.0	100	通过
2026-06-24	3	3	1	2	3	3	51.0	100	通过
2026-06-28	4	4	3	3	4	4	77.0	120	通过
2026-06-29	3	3	2	2	2	3	54.0	100	通过
2026-06-30	3	3	2	2	3	3	59.0	100	通过

新问题二赛程使第三轮分布在 7 个参考日。表7逐日给出三级转播、三级及以上安保、三级交通和总预算使用量，所有不等式均通过；安保档位还逐场满足最低需求且不超过固定场馆能力。

7.5 结果、同界增益与随机稳定性

表 8 问题三静态策略与动态重优化同界比较

指标	静态策略重评	动态重优化	偏好方向改善	单位
票务价值 TV	113.715	113.385	-0.330	百万美元
转播价值 BV	168.309	168.368	0.059	百万美元
吸引力 A	0.512	0.512	0.000	无量纲
资源成本 C	396.000	375.000	21.000	成本指数
风险 R	0.465	0.469	-0.004	无量纲
综合目标 Z3	6.572230	6.637484	0.065254	无量纲

注：正的偏好方向改善表示动态方案更优；成本与风险按越低越优转换。

静态锁定方案在更新环境中的同界得分为 6.572230，动态重优化得分为 6.637484，净增量为 0.065254；24 场中 8 场改变至少一个资源决策。表8显示，动态方案票务价值减少 0.330 百万美元、转播价值增加 0.059 百万美元、资源成本指数减少 21，风险均值增加约 0.004。因此增益来自题定五指标的全局权衡，而不是不存在的改动成本。

五个种子均重新执行 20000×24 联合模拟、条件概率计算、环境更新和完整动态 CP-SAT。五次动态求解均为 ‘OPTIMAL’，增益位于 $[0.065123, 0.065434]$ ，相对基准种子的 24 场动态资源决策均无变化。这说明本次配置差异不是由单个随机种子的抽样波动驱动。

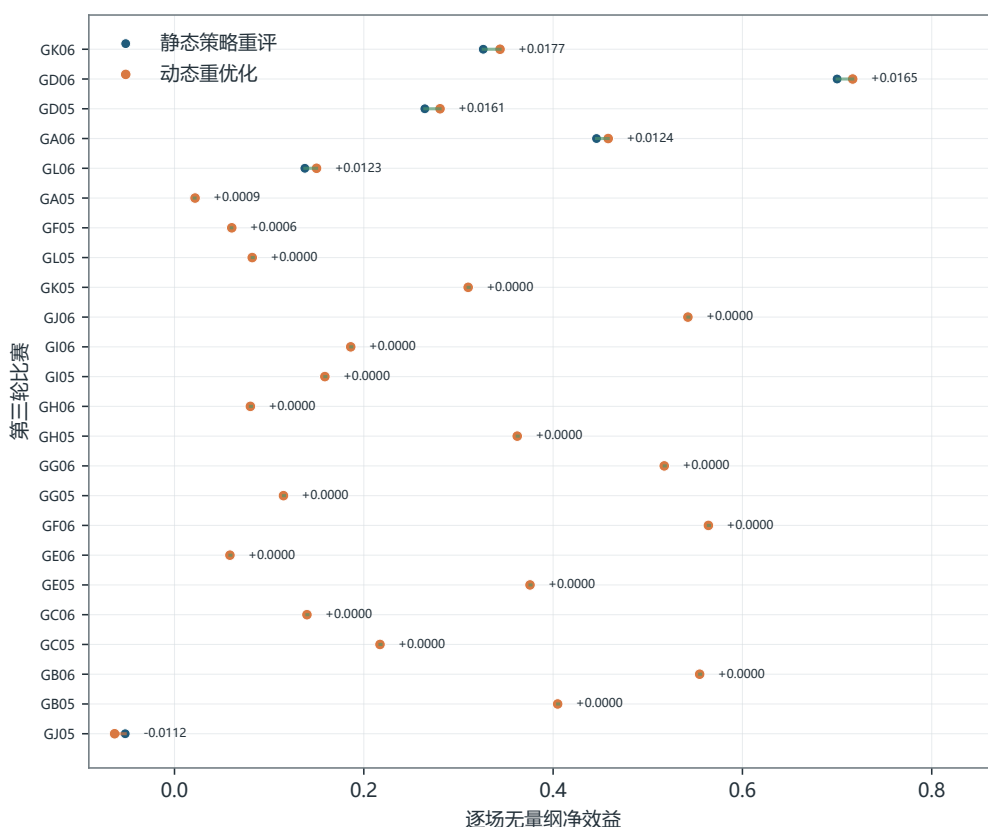


图 7 静态锁定方案与动态重优化的逐场同界净价值

八、问题四：真实世界杯赛程的双层同口径评价

8.1 比较边界与数据来源

本问选取 2022 年卡塔尔世界杯小组赛作为现实参照。机器读取源为工作区保存的 OpenFootball 结构化 JSON，FIFA 官方赛程与比分页面用于逐场核验；场馆容量取 FIFA 赛后统计，坐标按 FIFA 场馆地址核对^[11-13]。实际赛程的 48 场记录逐行保存球队、轮次、场馆、当地开球时间、UTC 时间和机器来源链接，不再把“采集源”与“官方核验源”写成同一字段。附件中的空白模板仅规定提交字段，不被当作观测数据；UTC 时间根据当地时间和 IANA 时区重新计算。

实际赛程与问题二的 72 场方案在球队规模、比赛数量和场馆集合上并不相同，直接比较总收益会把规模效应误当作安排优劣。为此，我们设置双层评价：第一层由两套赛程共同调用一个结构函数，比较休息、跨时区、实际场馆旅行、场馆日负荷、容量匹配、黄金覆盖和预计观众；第二层先将现实比赛与场馆整体映射到问题二候选域，再复算问题二八指标。

流程的关键不是寻找一个绝对“冠军方案”，而是保持比较口径对称。结构层的每一项均有明确物理单位和规模分母；代理层的现实比赛与场馆分别整体映射到问题二的比赛和安保合格候选场馆，随后使用问题二相同的距离、时间、时区边界与权重。现实

数据无法直接识别的商业、安保和题内吸引力统一标记为代理量，并与可观测结构分开解释。

8.2 结构层的共同计算函数

设球队 t 的相邻两场比赛为 $j-1, j$ ，UTC 开球时刻为 u_{tj} ，场馆坐标为 $(\phi_{tj}, \lambda_{tj})$ ，场馆 UTC 偏移为 z_{tj} 。休息、场馆旅行和跨时区指示分别定义为

$$r_{tj} = u_{tj} - u_{t,j-1}, \quad d_{tj} = \text{Haversine}((\phi_{t,j-1}, \lambda_{t,j-1}), (\phi_{tj}, \lambda_{tj})), \tag{44}$$

$$c_{tj} = \mathbb{I}(z_{tj} \neq z_{t,j-1}). \tag{45}$$

休息报告全部球队相邻轮间隔的最小值和平均值；旅行距离、换馆率和跨时区率均以球队转场次数为分母。场馆日利用率以“发生比赛的场馆日数”除以“启用场馆数乘比赛日期数”，场馆日峰值取单一场馆单日最大比赛数。容量匹配率为逐场 $\min(N_i/K_i, 1)$ 的均值，黄金覆盖率和场均预计观众均以比赛场次为分母。两套赛程只传入字段名不同，计算公式和聚合方式完全相同。

表 9 两套赛程的可观测结构与规模强度比较

指标	定义与分母	单位	2022 实际	2026 优化	偏好方向
最小休息	球队相邻比赛 UTC 间隔的最小值	小时	87.0000	63.0000	越高越优
平均休息	球队相邻比赛 UTC 间隔的平均值	小时	97.8125	139.8125	越高越优
跨时区次数	相邻比赛场馆 UTC 偏移发生变化的次数	次	0	78	越低越优
跨时区率	跨时区次数除以球队转场次数	比例	0.0000	81.2500	越低越优
场馆旅行	相邻比赛实际场馆大圆距离的平均值	公里/次	18.9868	2019.1075	越低越优
换馆率	相邻轮更换场馆的球队转场比例	比例	78.1250	94.7917	越低越优
场馆日利用率	发生比赛的场馆日占可用场馆日比例	比例	45.1923	25.0000	越高越优
场馆日峰值	单一场馆单日承办场次最大值	场/场馆日	2	1	越低越优
容量匹配	预计现场观众与场馆容量之比的场均值	比例	81.7689	61.7245	越高越优
黄金覆盖	按赛事进度日和 UTC 钟点映射后落入前四分位时段的比例	比例	25.0000	62.5000	越高越优
场均预计观众	预计现场观众总量除以比赛场次	人/场	41451.67	41704.92	越高越优

注：休息、时区、场馆路径与场馆日由逐场赛程直接复算；黄金覆盖来自唯一时段映射，容量匹配和预计观众的需求端为题内赛前代理。实际赛程有 64 次球队转场，优化赛程有 96 次。

结构层逐项比较见表9。实际赛程最小休息为 87 小时，高于优化方案的 63 小时；但优化方案平均休息达到 139.8125 小时，高于实际赛程的 97.8125 小时，说明优化方案的休息分布更不均匀。卡塔尔赛程位于同一时区，64 次球队转场中没有跨时区；跨国优化赛程的 96 次转场中有 78 次跨时区，平均场馆旅行距离也由 18.9868 公里升至 2019.1075 公里。

场馆运营维度呈现另一组取舍。实际与优化方案的场馆日峰值分别为 2 场和 1 场；场馆日利用率分别为 45.1923% 和 25.0000%。基于同一赛前需求代理，两者的平均容量匹配率分别为 81.7689% 和 61.7245%，场均预计现场观众分别为 41451.67 和 41704.92 人。按后述唯一时段映射，优化方案的黄金时段覆盖率为 62.5000%，高于实际赛程的 25.0000%；该项属于时间映射指标，不能替代跨时区和长距离旅行负担。

为统一展示不同物理单位下的方向和相对幅度，图8先按两方案绝对值最大者归一每个结构指标，再把“优化更符合偏好”记为正值。原始小时、公里、场次与比例仍以表9为准。

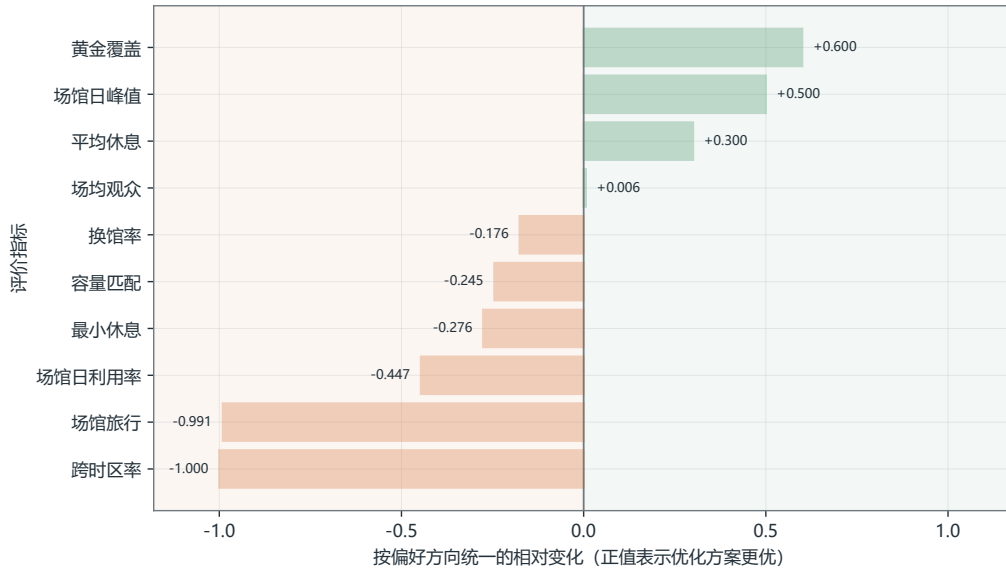


图 8 两套赛程的结构指标相对差异

8.3 代理层的同构映射与强度化

对受比赛数量影响的总量先做场均强度化。若方案 s 有 n_s 场比赛，则电视价值、品牌价值 and 成本分别写为

$$\bar{T}_s = \frac{T_s^0}{n_s}, \quad \bar{B}_s = \frac{B_s^0}{n_s}, \quad \bar{C}_s = \frac{C_s^0}{n_s}. \quad (46)$$

旅行与风险使用逐场均值， U, H 本身已是均值，公平性 F 是每队三场资源次数极差的标准化结果。对 T, B, C ，先将问题二候选上下界除以 72 转为场均边界，再与基准实际、优化及第二近邻三个已报告场景的原始值取共同包络。两个方案随后共享这一组边界执行一次 Min-Max，既保留共同参照，又保证进入 Z_4 的八项值严格属于 $[0, 1]$ 。

实际赛程缺少与题目附件完全同构的商业和安保字段。为避免目标型特征和静默缺失填补，本文引入 2022 年 11 月 12 日完整的世界杯赛前 Elo 快照^[14]，覆盖全部 32 支球队。比赛距离只使用双方 Elo 均值与差、由 Elo 构造的胜平负概率、不确定性和组内轮

次，在题内 72 场中寻找最近邻，再整体继承基础需求、吸引力与安保向量；预测观看人数不再参与邻居距离。实际场馆只依据有官方来源的容量，在满足该场安保等级的题内场馆中映射成本参数，不再声称使用逐场馆气候或公共交通数据。风险中的气候项固定为透明情景值 0.12。 U 不继承邻居，而由同一组三向赛前概率直接计算：

$$U_i = 1 - \max\{p_{ia}, p_{iD}, p_{ib}\}. \quad (47)$$

黄金时段也复用问题二规则：先固定附件 80 个候选时段中得分最高的 20 个。为消除不同日期具有相同时钟距离时由行顺序决定结果的问题，先把现实赛程的 13 个比赛日按赛事进度线性映射到模拟赛程的 20 个候选日，再在对应日期内选择圆周 UTC 钟点距离最近的时段；若距离仍相同，则按时段编号稳定选择。由此每场现实比赛得到唯一、可复算的候选时段。

为保证两套赛程的旅行指标具有相同物理定义，现实赛程的每场比赛和场馆先整体映射到问题二索引，再调用问题二同一个旅行负担表；不把卡塔尔实际路径公里数除以全球最大距离后直接与问题二复合量混列：

$$D_i = \frac{1}{2} \sum_{t \in \{a_i, b_i\}} \left(0.5\tilde{d}_{tiv} + 0.3\tilde{h}_{tiv} + 0.2\tilde{z}_{tiv} \right). \quad (48)$$

其中三项边界、第二三轮上一轮安保合格场馆集合与问题二完全一致。真实场馆路径仍在结构层以公里报告，不再进入这一复合量。

综合目标仅作为给定映射体系下的条件性分数：

$$Z_4 = 0.25T' + 0.25B' + 0.15U + 0.10H - (0.08C' + 0.07D' + 0.06F + 0.04R). \quad (49)$$

8.4 代理指标结果与稳健性边界

同构映射后，实际赛程的代理指标为 $T' = 0.413433$ 、 $B' = 0.633928$ 、 $U = 0.441275$ 、 $H = 0.649119$ 、 $C' = 0.534340$ 、 $D' = 0.213022$ 、 $F = 0.666667$ 、 $R = 0.455654$ ；优化方案对应为 0.918281、0.797646、0.461436、0.639699、0.382445、0.201797、0.333333 和 0.416678。八项值均位于 $[0, 1]$ ，不再存在边界外推后由单一指标主导综合分的问题；各指标证据等级及单位列于附录表17。

条件性代理层中，优化方案在 T, B, U, C, D, F, R 上更符合目标方向，实际赛程仅在 H 上略高。成本、旅行、公平和风险只按既定偏好方向解释，不再依据两方案的最大值二次缩放。由于各指标证据等级不同，这一比较只能说明题内映射下的取舍，不能替代结构层的小时和公里结果。

基准最近邻代理下，实际赛程的 Z_4 为 0.277058，优化赛程为 0.480778。逐一将八个权重的绝对值乘以 0.9 和 1.1 后重新归一，共形成 16 个扰动情景：实际分位于 $[0.267908, 0.285763]$ ，优化分位于 $[0.469560, 0.491449]$ ，方向保持率为 100%。将实际比赛与场馆代理整体换成第二近邻后，实际方案的 Z_4 为 0.226072，优化方向仍保持。

修复后的代理方向在已检验权重和邻居替换下稳定，但证据边界仍不能省略：48 场现实比赛映射到 26 个题内比赛，单一比赛最多复用 6 次；场馆成本代理只覆盖 4 个题内场馆，其中一个被复用 33 次。因此可以报告“当前题内映射下优化方案条件性总分较高”，却不能据此断言 FIFA 现实排程总体较差。抽签规则、转播合同、场馆可用性、当地治理、球队迁移策略和商业谈判等现实约束并未被附件完整观察；若用于现实决策，仍须补充真实合同、场馆运营和安保成本数据后重新校准代理层。

九、灵敏度分析与模型检验

四问对应四类不确定性：预测误差、排程偏好、状态反馈和现实代理。检验分别作用于这些接口，不把硬约束、可观测事实和代理得分混成一个“鲁棒性分数”。

9.1 预测模型的时间外稳定性

五折扩窗验证的总体 MSE、RMSE 和 MAE 分别为 220.0855、14.8121 和 11.8276，OOF 残差均值为 -0.001 百万人。表10显示各折 RMSE 为 13.2908–15.5929 百万人，后期折没有随时间距离增加而恶化；Elastic Net 在五折中均优于两个非线性候选。相对均值基线，平均 RMSE 从 24.099 降至 14.812，降幅约 38.5%。

表 10 Elastic Net 五个时间外折的误差

外折	MSE	RMSE	MAE
1	243.1382	15.5929	12.7572
2	226.6187	15.0539	11.9290
3	239.3400	15.4706	12.2031
4	214.6845	14.6521	11.5100
5	176.6463	13.2908	10.7386

残差的 5% 与 95% 分位数为 -22.7075 和 25.1737 百万人，整体大致对称，但高观看场次仍有收缩性低估。预测值可用于排程中的相对需求排序；当新增窗口的 RMSE 明显超过当前最差折，或残差均值持续偏离零时，应检查分布漂移和编码，再决定是否重训。

9.2 赛程权重与动态参数

问题二只改变已求得赛程的评价权重，不重新求解整数规划，因此检验的是固定方案的评分敏感性。商业价值和风险惩罚权重在基准值的 0.9–1.1 倍内变化时， Z_2 由 0.445463 变到 0.522504，基准为 0.484946；图9左图的等高线连续，方向与目标符号一致。权重变化不影响唯一指派、休息、安保和日负荷等硬约束。最终方案经独立复算无违反，求解证书间隙约 0.41%。

问题三将状态系数和伤病系数分别乘以 0.9, 1.0, 1.1，每个参数点都在相同随机数流、动作域和日预算下重新求解。图9右图中 Z_3 位于 [6.623009, 6.713491]，九点极差约为中心值的 1.4%；局部非单调来自离散动作切换，并非系数符号错误。五个随机种子下，晋级概率标准差最大为 0.004691，完整重优化增益为 0.065123–0.065434，24 场动态决策均与基准一致。

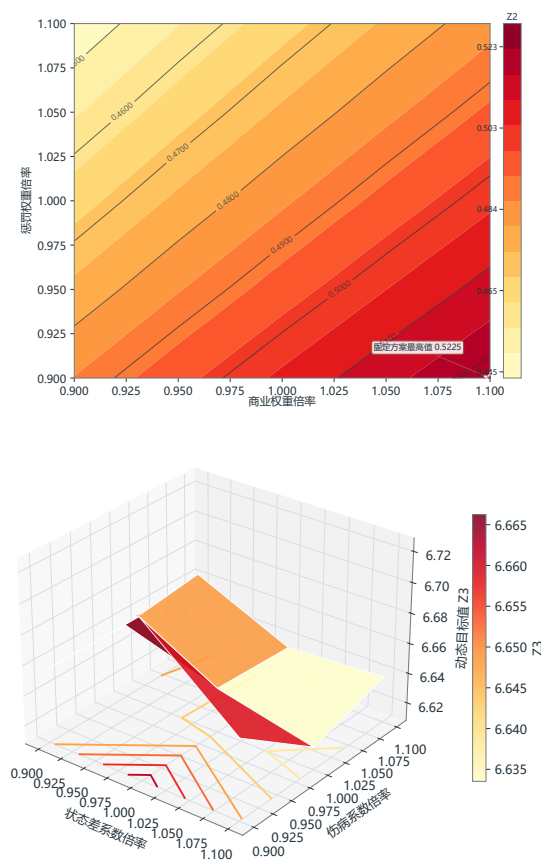


图 9 关键参数灵敏度：上图为问题二权重扰动，下图为问题三状态与伤病系数扰动

9.3 现实比较的权重与代理检验

表11区分权重扰动、代理替换和结构复算。在基准最近邻下，八个权重分别上下调 10% 后再归一化，共 16 个情景，优化方案得分始终高于实际赛程。把全部代理由最近

邻换为第二近邻后，实际赛程得分变为 0.226072，优化方案仍为 0.480778，方向没有翻转。

表 11 问题四不同证据层的稳健性结果

检验	实际赛程	优化赛程	允许的结论
基准最近邻	0.277058	0.480778	题内映射下的条件性排序
16 个权重扰动	0.267908–0.285763	0.469560–0.491449	局部方向稳定
第二近邻替换	0.226072	0.480778	已检验邻居内方向保持
结构层复算	小时、公里、场馆日	同单位物理量	可直接报告逐场事实

这两类代理检验都不能代表真实合同和运营成本。现实 48 场仍映射到 26 个题内比赛和 4 个题内场馆，最高复用次数分别为 6 和 33；因此 Z_4 只能支持题内条件性方向，结构层的小时、公里和场馆日证据更强。

十、结论、评价与适用边界

10.1 主要结论

1. 在严格赛前信息边界和整窗标签屏蔽下，Elastic Net 的五折平均 MSE、RMSE 和 MAE 分别为 220.0855、14.8121 和 11.8276，均优于另外三类候选。72 场需求预测均值为 170321280 人；高关注赛事的不确定性由 OOF 残差分位数单独给出。
2. 三维 CP-SAT 联合决定 72 场比赛的场馆与时段，得到 $Z_2 = 0.484946$ 。独立复算显示无硬约束违反，最小相邻轮休息为 63 小时，场馆负荷为 2–8 场；0.41% 的证书间隙满足预设停止准则，但不构成零间隙最优证明。
3. 在第二轮结束的共同截面上，联合模拟保持每个样本的晋级名额质量为 32。动态方案仅调整 8/24 场，目标由锁定静态方案的 6.572230 提高到 6.637484；五种子完整重优化的增益为 0.065123–0.065434，离散决策一致。
4. 实际赛程与优化方案存在结构取舍：实际赛程最小休息更长、跨时区和实际旅行更少，优化方案平均休息和黄金覆盖更高。共同边界下的代理综合分为 0.277058 与 0.480778，权重和邻居扰动均保持方向；该结果只适用于题内映射，不能解释为 FIFA 现实排程的总体优劣。

10.2 可核验性

四问按信息产生顺序连接：问题一只读开球前字段，问题二接收赛前需求，问题三在第二轮结束后更新状态，问题四才引入现实记录和代理。需求预测以“人”为单位进

入排程；求解完成后，独立程序从正式 CSV 复算唯一指派、UTC 冲突、休息、容量、安保、日负荷与公平指标。求解器状态说明搜索程度，物理可行性由这些复算结果决定。

静态与动态方案共用动作域、更新环境、价格边界和归一化上下界，因此增益可以直接比较。现实评价则保留数据来源和代理标记：结构量按小时、公里和比例报告，缺失商业字段不伪装成观测。灵敏度分析进一步说明哪些扰动只改变数值，哪些可能改变离散动作，以及哪些结论仍受代理集中限制。

10.3 局限与使用范围

需求样本只有 560 条，高观看尾部稀缺，Elastic Net 对极端赛事存在收缩性低估；特征重要性表示预测贡献，不是因果效应。排程模型受题定候选场馆、旅行口径和成本参数约束，未包含真实转播合同、临时管制、训练基地和维护窗口。动态模型采用泊松比分与离散资源档位，并固定第三轮共同截面；实际执行时，更新频率还必须服从通知提前量和资源调度周期。

问题四的商业价值、成本和安保字段来自代理，且映射存在复用集中。真实运营数据补齐前， Z_4 只能描述条件性取舍，不能触发“替换现实排程”的决策。迁移到其他赛事时，可复用的是“需求预测–联合指派–滚动重评–独立审计”的结构；轮次规则、候选场馆、休息下限和资源预算必须按新场景重建。若真实约束无法进入模型或代理稳定性不足，应停止输出总体优劣，改为报告局部方案和待补数据。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于论文技术审阅、题面–公式–代码一致性核查、代码调试、数值复算、LaTeX 排版与语言整理，详细使用情况见支撑材料。

参考文献

- [1] BORLAND J, MACDONALD R. Demand for sport[J/OL]. Oxford Review of Economic Policy, 2003, 19(4): 478-502. DOI:10.1093/oxrep/19.4.478.
- [2] BERGMEIR C, HYNDMAN R J, KOO B. A note on the validity of cross-validation for evaluating autoregressive time series prediction[J/OL]. Computational Statistics and Data Analysis, 2018, 120: 70-83. DOI:10.1016/J.CSDA.2017.11.003.
- [3] ZOU H, HASTIE T. Regularization and variable selection via the elastic net[J/OL]. Journal of the Royal Statistical Society Series B: Statistical Methodology, 2005, 67(2): 301-320. DOI:10.1111/j.1467-9868.2005.00503.x.
- [4] GEURTS P, ERNST D, WEHENKEL L. Extremely randomized trees[J/OL]. Machine Learning, 2006, 63(1): 3-42. DOI:10.1007/S10994-006-6226-1.

- [5] PROKHORENKOVA L O, GUSEV G, VOROBEOV A, et al. CatBoost: unbiased boosting with categorical features [C/OL]//Advances in neural information processing systems 31. 2018: 6639-6649. <https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html>.
- [6] RASMUSSEN R V, TRICK M A. Round robin scheduling - a survey[J/OL]. European Journal of Operational Research, 2008, 188(3): 617-636. DOI:10.1016/J.EJOR.2007.05.046.
- [7] PERRON L, DIDIER F, GAY S. The CP-SAT-LP solver[C/OL]//LIPIcs: v.280 29th international conference on principles and practice of constraint programming. 2023: 3:1-3:2. DOI:10.4230/LIPICS.CP.2023.3.
- [8] DIXON M J, COLES S G. Modelling association football scores and inefficiencies in the football betting market[J/OL]. Journal of the Royal Statistical Society Series C: Applied Statistics, 1997, 46(2): 265-280. DOI:10.1111/1467-9876.00065.
- [9] KROESE D P, BRERETON T, TAIMRE T, et al. Why the monte carlo method is so important today[J/OL]. WIREs Computational Statistics, 2014, 6(6): 386-392. DOI:10.1002/wics.1314.
- [10] OECD. Handbook on constructing composite indicators: methodology and user guide[M/OL]. OECD, 2008. DOI:10.1787/9789264043466-en.
- [11] OpenFootball. World cup 2022 jSON dataset[EB/OL]. <https://github.com/openfootball/worldcup.json/tree/master/2022>.
- [12] FIFA. Final match schedule for the fifa world cup qatar 2022[EB/OL]. <https://inside.fifa.com/tournaments/mens/worldcup/qatar2022/media-releases/final-match-schedule-for-the-fifa-world-cup-qatar-2022-tm-now-available-on>.
- [13] FIFA. FIFA world cup qatar 2022 at a glance[EB/OL]. <https://inside.fifa.com/tournament-organisation/world-cup-2022-in-numbers/fifa-world-cup-qatar-2022-at-a-glance>.
- [14] International-Football.net. World football elo ratings as on november 12th, 2022[EB/OL]. <https://www.international-football.net/elo-ratings-table?confed=&day=12&month=11&old-team=&year=2022>.

A 支撑材料清单与完整源程序

1.1 支撑材料文件列表

支撑材料压缩包采用如下目录结构；赛题提供的原始 Excel 不重复打包，复现时按 README.md 放入 user_data/ 即可。

路径	内容与用途
README.md	从空环境开始的安装、运行、输出和论文编译说明
requirements-lock.txt	Python 3.12 的直接依赖精确版本
reproduce.py	四问、灵敏度、约束复核和图表的一键入口
code/	问题一至问题四、参数、工具、数据检查、灵敏度和约束复核的完整 Python 源码
figures/*.py、_utils/ paper/	图表数据整理、统一绘图和 LaTeX 表格生成程序 论文 LaTeX 源文件、模板、字体、参考文献和已生成图表
output/	正式 CSV 及问题四可选比较输出的同内容副本
figures/problem_*.json	四问完整机器结果、求解器状态、边界、审计和敏感性记录
user_data/worldcup_2022_ openfootball.json	问题四真实赛程的机器读取快照
user_data/worldcup_2022_ pre_tournament_elo.csv	问题四完整赛前 Elo 快照
actual_schedule_2022.csv	真实赛程逐场整理表、来源 URL 与访问日期
AI 工具使用详情.pdf	工具、用途、提示方式、采纳和核验记录

1.2 完整结果附表

以下附表由正式 JSON/CSV 直接生成，用于复核正文汇总值。

表 13 问题一特征字典与时间血缘

特征	来源表	来源列	时间规则	拟合样本
competition	historical_matches/ teams/base_predictions	competition	pregame_available	outer_fold_train_only; final=560 train ids
stage	historical_matches/ teams/base_predictions	stage	pregame_available	outer_fold_train_only; final=560 train ids
timezone_pair	historical_matches/ teams/base_predictions	timezone_pair	pregame_available	outer_fold_train_only; final=560 train ids

续下页

续表

特征	来源表	来源列	时间规则	拟合样本
strength_rank_mean	historical_matches/ teams/base_predictions	strength_rank_mean	pregame_available	outer_fold_train_only; final=560 train ids
strength_rank_gap	historical_matches/ teams/base_predictions	strength_rank_gap	pregame_available	outer_fold_train_only; final=560 train ids
elo_mean	historical_matches/ teams/base_predictions	elo_mean	pregame_available	outer_fold_train_only; final=560 train ids
elo_gap	historical_matches/ teams/base_predictions	elo_gap	pregame_available	outer_fold_train_only; final=560 train ids
p_a	historical_matches/ teams/base_predictions	p_a	pregame_available	outer_fold_train_only; final=560 train ids
p_draw	historical_matches/ teams/base_predictions	p_draw	pregame_available	outer_fold_train_only; final=560 train ids
p_b	historical_matches/ teams/base_predictions	p_b	pregame_available	outer_fold_train_only; final=560 train ids
odds_entropy	historical_matches/ teams/base_predictions	odds_entropy	pregame_available	outer_fold_train_only; final=560 train ids
market_value_musd_ mean	historical_matches/ teams/base_predictions	market_value_musd_ mean	pregame_available	outer_fold_train_only; final=560 train ids
market_value_musd_ gap	historical_matches/ teams/base_predictions	market_value_musd_ gap	pregame_available	outer_fold_train_only; final=560 train ids
avg_age_mean	historical_matches/ teams/base_predictions	avg_age_mean	pregame_available	outer_fold_train_only; final=560 train ids
avg_age_gap	historical_matches/ teams/base_predictions	avg_age_gap	pregame_available	outer_fold_train_only; final=560 train ids
star_index_mean	historical_matches/ teams/base_predictions	star_index_mean	pregame_available	outer_fold_train_only; final=560 train ids
star_index_gap	historical_matches/ teams/base_predictions	star_index_gap	pregame_available	outer_fold_train_only; final=560 train ids
fan_base_index_mean	historical_matches/ teams/base_predictions	fan_base_index_mean	pregame_available	outer_fold_train_only; final=560 train ids
fan_base_index_gap	historical_matches/ teams/base_predictions	fan_base_index_gap	pregame_available	outer_fold_train_only; final=560 train ids
style_attack_mean	historical_matches/ teams/base_predictions	style_attack_mean	pregame_available	outer_fold_train_only; final=560 train ids
style_attack_gap	historical_matches/ teams/base_predictions	style_attack_gap	pregame_available	outer_fold_train_only; final=560 train ids
style_defense_mean	historical_matches/ teams/base_predictions	style_defense_mean	pregame_available	outer_fold_train_only; final=560 train ids
style_defense_gap	historical_matches/ teams/base_predictions	style_defense_gap	pregame_available	outer_fold_train_only; final=560 train ids
host_flag_mean	historical_matches/ teams/base_predictions	host_flag_mean	pregame_available	outer_fold_train_only; final=560 train ids
host_flag_gap	historical_matches/ teams/base_predictions	host_flag_gap	pregame_available	outer_fold_train_only; final=560 train ids
strength_score_mean	historical_matches/ teams/base_predictions	strength_score_mean	pregame_available	outer_fold_train_only; final=560 train ids
strength_score_gap	historical_matches/ teams/base_predictions	strength_score_gap	pregame_available	outer_fold_train_only; final=560 train ids
a_hist_goals_roll5	historical_matches	a_hist_goals_roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_goals_ewm5	historical_matches	a_hist_goals_ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids

续下页

续表

特征	来源表	来源列	时间规则	拟合样本
a_hist_xg_roll5	historical_matches	a_hist_xg_roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_xg_ewm5	historical_matches	a_hist_xg_ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_shots_roll5	historical_matches	a_hist_shots_roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_shots_ewm5	historical_matches	a_hist_shots_ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_possession_ roll5	historical_matches	a_hist_possession_ roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_possession_ ewm5	historical_matches	a_hist_possession_ ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_attendance_ roll5	historical_matches	a_hist_attendance_ roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_attendance_ ewm5	historical_matches	a_hist_attendance_ ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_tv_viewers_ roll5	historical_matches	a_hist_tv_viewers_ roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
a_hist_tv_viewers_ ewm5	historical_matches	a_hist_tv_viewers_ ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_goals_roll5	historical_matches	b_hist_goals_roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_goals_ewm5	historical_matches	b_hist_goals_ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_xg_roll5	historical_matches	b_hist_xg_roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_xg_ewm5	historical_matches	b_hist_xg_ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_shots_roll5	historical_matches	b_hist_shots_roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_shots_ewm5	historical_matches	b_hist_shots_ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_possession_ roll5	historical_matches	b_hist_possession_ roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_possession_ ewm5	historical_matches	b_hist_possession_ ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_attendance_ roll5	historical_matches	b_hist_attendance_ roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_attendance_ ewm5	historical_matches	b_hist_attendance_ ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_tv_viewers_ roll5	historical_matches	b_hist_tv_viewers_ roll5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
b_hist_tv_viewers_ ewm5	historical_matches	b_hist_tv_viewers_ ewm5	strictly_before_kickoff_ shift1	outer_fold_train_only; final=560 train ids
worldcup_stage	historical_matches/ teams/base_predictions	worldcup_stage	pregame_available	outer_fold_train_only; final=560 train ids
fan_stage_interaction	historical_matches/ teams/base_predictions	fan_stage_interaction	pregame_available	outer_fold_train_only; final=560 train ids
star_stage_interaction	historical_matches/ teams/base_predictions	star_stage_interaction	pregame_available	outer_fold_train_only; final=560 train ids
strength_entropy_ interaction	historical_matches/ teams/base_predictions	strength_entropy_ interaction	pregame_available	outer_fold_train_only; final=560 train ids

表 14 问题一时序交叉验证误差

模型	窗口	MSE	RMSE	MAE	训练截至	验证起始
MeanBaseline	折 1	598.919	24.473	19.751	2019-01-26	2019-02-03
ElasticNet	折 1	243.138	15.593	12.757	2019-01-26	2019-02-03
ExtraTrees	折 1	298.141	17.267	14.214	2019-01-26	2019-02-03
CatBoost	折 1	274.080	16.555	13.172	2019-01-26	2019-02-03
MeanBaseline	折 2	621.856	24.937	19.817	2020-03-29	2020-04-04
ElasticNet	折 2	226.619	15.054	11.929	2020-03-29	2020-04-04
ExtraTrees	折 2	261.458	16.170	12.694	2020-03-29	2020-04-04
CatBoost	折 2	244.634	15.641	12.265	2020-03-29	2020-04-04
MeanBaseline	折 3	528.105	22.981	18.864	2021-04-19	2021-04-22
ElasticNet	折 3	239.340	15.471	12.203	2021-04-19	2021-04-22
ExtraTrees	折 3	256.627	16.020	13.012	2021-04-19	2021-04-22
CatBoost	折 3	243.231	15.596	12.693	2021-04-19	2021-04-22
MeanBaseline	折 4	575.265	23.985	18.588	2022-07-03	2022-07-05
ElasticNet	折 4	214.684	14.652	11.510	2022-07-03	2022-07-05
ExtraTrees	折 4	242.182	15.562	12.326	2022-07-03	2022-07-05
CatBoost	折 4	230.889	15.195	11.874	2022-07-03	2022-07-05
MeanBaseline	折 5	581.872	24.122	19.646	2023-07-28	2023-07-29
ElasticNet	折 5	176.646	13.291	10.739	2023-07-28	2023-07-29
ExtraTrees	折 5	192.403	13.871	11.426	2023-07-28	2023-07-29
CatBoost	折 5	178.211	13.350	10.915	2023-07-28	2023-07-29
ElasticNet	总体均值	220.086	14.812	11.828	—	—
CatBoost	总体均值	234.209	15.267	12.184	—	—
ExtraTrees	总体均值	250.162	15.778	12.734	—	—
MeanBaseline	总体均值	581.203	24.099	19.333	—	—

注：误差单位为百万人口径；最终选择模型为 ElasticNet。

表 15 问题二场馆负载与赛程汇总

场馆	城市	场次	黄金时段场次	平均旅行指数	预计观众合计（万人）
V01	Atlanta	8	4	0.254	32.6
V02	Boston	8	5	0.321	30.0
V03	Dallas	7	7	0.081	27.1
V04	Guadalajara	8	6	0.152	35.1
V05	Houston	8	6	0.086	35.2
V06	Kansas City	5	5	0.086	17.3
V07	Los Angeles	4	1	0.270	16.7
V08	Mexico City	7	7	0.110	34.3
V09	Miami	3	1	0.244	13.9
V10	Monterrey	2	0	0.096	6.7
V11	New York/New Jersey	2	1	0.117	6.9
V12	Philadelphia	2	1	0.272	9.3
V13	San Francisco Bay Area	2	1	0.414	9.0
V14	Seattle	2	0	0.556	8.4
V15	Toronto	2	0	0.266	8.5
V16	Vancouver	2	0	0.504	9.4

注：主方案 Z2=0.484946，最小轮间休息 63 小时。

表 16 问题三晋级概率、条件概率与风险

比赛	A 队	A 队晋级	B 队	B 队晋级	A 胜条件晋级	B 胜条件晋级	无悬念风险	默契风险
GA05	Mexico	1.000	Czechia	1.000	1.000	1.000	1.000	0.650
GA06	South Africa	0.226	South Korea	0.541	1.000	0.991	0.154	0.000
GB05	Canada	0.355	Switzerland	1.000	1.000	1.000	0.542	0.803
GB06	Bosnia and Herzegovina	0.601	Qatar	0.318	1.000	1.000	0.087	0.013
GC05	Brazil	1.000	Scotland	0.077	1.000	1.000	0.857	0.015
GC06	Morocco	0.963	Haiti	0.209	1.000	1.000	0.599	0.704
GD05	United States	1.000	Turkey	1.000	1.000	1.000	1.000	0.650
GD06	Paraguay	0.314	Australia	0.631	0.851	1.000	0.103	0.000
GE05	Germany	1.000	Ecuador	0.954	1.000	1.000	0.912	0.681
GE06	Curacao	0.002	Ivory Coast	1.000	0.021	1.000	0.995	0.000
GF05	Netherlands	1.000	Tunisia	0.070	1.000	1.000	0.870	0.002
GF06	Japan	0.934	Sweden	0.319	1.000	1.000	0.442	0.001
GG05	Belgium	1.000	New Zealand	0.782	1.000	1.000	0.659	0.769
GG06	Egypt	0.808	Iran	0.155	1.000	0.416	0.428	0.000
GH05	Spain	1.000	Uruguay	0.986	1.000	1.000	0.972	0.660
GH06	Cape Verde	0.025	Saudi Arabia	0.965	0.100	1.000	0.885	0.000
GI05	France	1.000	Norway	1.000	1.000	1.000	1.000	0.650
GI06	Senegal	0.886	Iraq	0.056	1.000	0.496	0.692	0.000
GJ05	Argentina	1.000	Jordan	1.000	1.000	1.000	1.000	0.650
GJ06	Algeria	0.220	Austria	0.510	0.986	0.995	0.157	0.000
GK05	Portugal	1.000	Colombia	1.000	1.000	1.000	1.000	0.650
GK06	DR Congo	0.288	Uzbekistan	0.122	0.956	0.292	0.375	0.000
GL05	England	1.000	Panama	0.588	1.000	1.000	0.516	0.820
GL06	Croatia	1.000	Ghana	0.093	1.000	0.614	0.832	0.000

注：条件晋级概率来自同一 20000 次联合蒙特卡洛样本，晋级名额质量守恒为每次 32 队。

表 17 问题四评价指标定义与可比口径

指标	定义	单位	偏好方向	数据源	口径说明
rest_min_hours	球队相邻比赛 UTC 间隔的最小值	小时	越高越优	统一结构评价函数	现实赛程直接复算
rest_mean_hours	球队相邻比赛 UTC 间隔的平均值	小时	越高越优	统一结构评价函数	现实赛程直接复算
timezone_crossings	相邻比赛场馆 UTC 偏移发生变化的次数	次	越低越优	统一结构评价函数	现实赛程直接复算
timezone_crossing_rate	跨时区次数除以球队转场次数	比例	越低越优	统一结构评价函数	现实赛程直接复算

续下页

续表					
指标	定义	单位	偏好方向	数据源	口径说明
travel_	相邻比赛实际场馆大圆距离的平均	公里/次	越低越优	统一结构评价函数	现实赛程直接复算
mean_km	值				
venue_	相邻轮更换场馆的球队转场比例	比例	越低越优	统一结构评价函数	现实赛程直接复算
change_rate					
venue_	发生比赛的场馆日占可用场馆日比	比例	越高越优	统一结构评价函数	现实赛程直接复算
utilization_	例				
rate					
venue_	单一场馆单日承办场次最大值	场/场馆日	越低越优	统一结构评价函数	现实赛程直接复算
daily_peak					
capacity_	预计现场观众与场馆容量之比的场	比例	越高越优	统一结构评价函数	需求端为题内代理
occupancy_	均值				
rate					
prime_	按赛事进度日和 UTC 钟点映射后落	比例	越高越优	统一结构评价函数	赛事进度与 UTC 时刻映射
coverage_	入前四分位时段的比例				
rate					
expected_	预计现场观众总量除以比赛场次	人/场	越高越优	统一结构评价函数	需求端为题内代理
attendance_					
per_match					
T	$T = \text{norm}(\sum \text{ticket})$	无量纲	越高越优	统一代理评价函数	共同包络 Min-Max
B	$B = \text{norm}(\sum \text{broadcast})$	无量纲	越高越优	统一代理评价函数	共同包络 Min-Max
U	$U = \overline{\text{uncertainty}}$	无量纲	越高越优	统一代理评价函数	题内代理评价
H	$H = \overline{\text{attractiveness}}/100$	无量纲	越高越优	统一代理评价函数	题内代理评价
C	$C = \overline{\text{norm}(\text{setup} + \text{operation} + \text{security})}$	无量纲	越低越优	统一代理评价函数	共同包络 Min-Max
D	$D = \overline{\text{travel}}$	无量纲	越低越优	统一代理评价函数	题内代理评价
F	$F = (\Delta \text{prime} + \Delta \text{large})/6$	无量纲	越低越优	统一代理评价函数	题内代理评价
R	$R = \overline{\text{risk}}$	无量纲	越低越优	统一代理评价函数	题内代理评价
Z4	$Z4 = 0.25T + 0.25B + 0.15U + 0.10H - 0.08C - 0.07D - 0.06F - 0.04R$	无量纲	越高越优	统一代理评价函数	条件性加权总分

注：结构层报告物理量；代理层用完整赛前 Elo 映射比赛、仅按容量和安保可行域映射场馆，黄金时段按赛事进度日与 UTC 钟点唯一映射；T/B/C 在全部报告场景共用的包络边界上归一化。

1.3 完整可运行源程序

下列清单给出模型与正式复核链所使用的完整源程序，不省略函数或执行入口。图表生成程序的完整文件同时保存在支撑材料中。

Listing 1: 一键复现入口

```

1  """One-command reproduction of model outputs, audits, figures and tables."""
2  from __future__ import annotations
3
4  import argparse
5  import subprocess
6  import sys
7  from pathlib import Path
8
9
10 ROOT = Path(__file__).resolve().parent
11
12
13 def run(*parts: str) -> None:
14     command = [sys.executable, *parts]
15     print(f"[reproduce] {' '.join(command)}", flush=True)
16     subprocess.run(command, cwd=ROOT, check=True)

```

```

17
18
19 def main() -> None:
20     parser = argparse.ArgumentParser()
21     parser.add_argument(
22         "--reuse",
23         action="store_true",
24         help="validate persisted model outputs and regenerate figures without re-solving",
25     )
26     args = parser.parse_args()
27
28     run("code/data_check.py")
29     main_args = ["code/main.py"]
30     if args.reuse:
31         main_args.append("--reuse")
32     run(*main_args)
33     run("code/constraint_audit.py")
34     run("figures/generate_all.py")
35     print("[reproduce] PASS", flush=True)
36
37
38 if __name__ == "__main__":
39     main()

```

Listing 2: 四问总入口

```

1 """竞赛编程实现总入口：按 P1-P2-P3-P4+灵敏度分析串联并汇总结果。
2
3 默认执行完整计算；使用 ``--reuse`` 时仅验证并汇总工作区已经完成的四问结果，
4 适合交付前快速检查。任一子问题失败都会返回非零退出码，不会把残缺结果冒充成功。
5 """
6 import os, sys
7 _HERE = os.path.dirname(os.path.abspath(__file__))
8 if _HERE not in sys.path:
9     sys.path.insert(0, _HERE)
10
11 import argparse
12 import json
13 import shutil
14 from pathlib import Path
15
16 from utils import FIGURES_DIR, OUTPUT_DIR, write_json
17
18
19 def run_all(force: bool) -> dict:
20     modules=[]
21     if force:
22         from problem1 import solve as solve1
23         from problem2 import solve as solve2
24         from problem3 import solve as solve3
25         from problem4 import solve as solve4
26         for n,func in enumerate((solve1,solve2,solve3,solve4),1):
27             print(f"[main] running problem {n}",flush=True)
28             func()
29         # 灵敏度结果被正文与图表直接引用，必须属于正式复现链，不能依赖手工补跑。
30         from sensitivity_analysis import main as solve_sensitivity
31         print("[main] running sensitivity analysis", flush=True)
32         solve_sensitivity()
33     for n in range(1,5):
34         path=FIGURES_DIR/f"problem_{n}_results.json"
35         if not path.exists() or path.stat().st_size==0:
36             raise FileNotFoundError(f"缺少非空结果: {path}")
37         modules.append(json.loads(path.read_text(encoding="utf-8")))
38     all_results={f"problem_{i+1}":value for i,value in enumerate(modules)}
39     sensitivity=FIGURES_DIR/"sensitivity_results.json"
40     if sensitivity.exists():
41         all_results["sensitivity"]=json.loads(sensitivity.read_text(encoding="utf-8"))
42     write_json(FIGURES_DIR/"all_results.json",all_results)
43     # 题目模板要求的交付文件位于工作区根目录；output/ 保留同一份过程副本。
44     for name in [
45         "result_1_test_prediction.csv", "result_1_match_prediction.csv",
46         "result_2_group_schedule.csv", "result_3_dynamic_strategy.csv",
47         "result_4_schedule_comparison.csv", "actual_schedule_2022.csv",
48     ]:

```

```

49     source=Path(OUTPUT_DIR)/name
50     if not source.exists() or source.stat().st_size==0:
51         raise FileNotFoundError(f"缺少交付文件: {source}")
52     shutil.copyfile(source, Path(FIGURES_DIR).parent/name)
53     print(f"[main] PASS problems={len(modules)} all_results={FIGURES_DIR/'all_results.json'}", flush=True)
54     return all_results
55
56
57 if __name__=="__main__":
58     parser=argparse.ArgumentParser()
59     parser.add_argument("--reuse", action="store_true", help="复用已完成结果, 只做汇总")
60     args=parser.parse_args()
61     run_all(force=not args.reuse)

```

Listing 3: 官方附件摄入检查

```

1  """附件数据摄入验证: 必须先于四问求解运行。"""
2  import os, sys
3  _HERE = os.path.dirname(os.path.abspath(__file__))
4  if _HERE not in sys.path:
5      sys.path.insert(0, _HERE)
6
7  import json
8  from pathlib import Path
9
10 import pandas as pd
11
12 from utils import DATA_FILE, ROOT, assert_key_equal, load_all_sheets
13
14
15 def main() -> None:
16     sheets = load_all_sheets()
17     print(f"[data_check] 文件={DATA_FILE.name} sheets={len(sheets)} total_rows={sum(map(len, sheets.values()))}")
18     for name, df in sheets.items():
19         missing = int(df.isna().sum().sum())
20         print(f" {name}: rows={len(df)} cols={len(df.columns)} missing_cells={missing}")
21         print("    columns=" + ", ".join(map(str, df.columns)))
22
23     hist = sheets["historical_matches"].copy()
24     split_counts = hist["dataset_split"].value_counts(dropna=False).to_dict()
25     assert split_counts == {"train": 560, "test": 140}, f"固定 split 异常: {split_counts}"
26     train = hist.loc[hist["dataset_split"].eq("train")]
27     test = hist.loc[hist["dataset_split"].eq("test")]
28     assert train["match_id"].is_unique and test["match_id"].is_unique
29     assert set(train["match_id"]).isdisjoint(set(test["match_id"]))
30     assert train["tv_viewers"].notna().all(), "train 标签存在缺失"
31     assert test["tv_viewers"].isna().all(), "test 标签不应可见"
32
33     assert_key_equal(
34         (sheets["groups_matches"], "match_id"),
35         (sheets["base_predictions"], "match_id"),
36         (sheets["security_requirements"], "match_id"),
37     )
38     assert len(sheets["live_group_results"]) == 48
39     assert set(sheets["live_group_results"]["round_in_group"]) == {1, 2}
40     print("[data_check] PASS 固定560/140 split、72场跨表键、48场前两轮和全表摄入均一致")
41
42
43 if __name__ == "__main__":
44     main()

```

Listing 4: 统一参数定义

```

1  """自动生成: 唯一数据源为 ../PROBLEM_FACTS.json; 请勿手改常数。"""
2  import json
3  from pathlib import Path
4  _FACTS = json.loads((Path(__file__).resolve().parent.parent / "PROBLEM_FACTS.json").read_text(encoding="utf-8"))
5  MASTER_SEED = 20260813
6  N_TIME_SPLITS = 5
7  TARGET_SCALE = 1000000
8  N_TEAMS = 48

```

```

9  N_GROUPS = 12
10 N_GROUP_MATCHES = 72
11 N_THIRD_ROUND_MATCHES = 24
12 MC_DRAWS = 20000
13 MINIMUM_REST_HOURS = 60
14 GLOBAL_PRIME_FRACTION = 0.25
15 MAX_PRIME_COUNT_RANGE = 2
16 LARGE_VENUE_FRACTION = 0.25
17 HIGH_SECURITY_THRESHOLD = 3
18 SECURITY_BASE_COST_USD = 100000
19 P2_WEIGHTS = {'ticket_revenue': 0.25, 'broadcast_value': 0.25, 'uncertainty': 0.15, 'attractiveness': 0.1, 'organization_cost': {'magnitude': 0.08, '
    direction': 'subtract'}, 'travel_burden': {'magnitude': 0.07, 'direction': 'subtract'}, 'fairness_penalty': {'magnitude': 0.06, 'direction': '
    subtract'}, 'execution_risk': {'magnitude': 0.04, 'direction': 'subtract'}}
20 TRAVEL_WEIGHTS = {'distance': 0.5, 'travel_time': 0.3, 'timezone_difference': 0.2}
21 P2_FAIRNESS_WEIGHTS = {'prime_time_range': 0.5, 'large_venue_range': 0.5, 'divisor': 3}
22 P2_RISK_WEIGHTS = {'climate': 0.5, 'occupancy': 0.3, 'security_ratio': 0.2}
23 STATE_WEIGHTS = {'points': 0.45, 'xg_form': 0.35, 'discipline': 0.2}
24 STATE_XG_SCALE = 1.25
25 RED_CARD_DECAY = 0.55
26 DELTA_STATE_COEF = 0.32
27 DELTA_INJURY_COEF = 0.055
28 DIRECT_INJURY_COEF = 0.04
29 GOAL_MEAN_CLIP = [0.15, 4.5]
30 RAW_FEEDBACK_CLIP = [0.6, 1.5]
31 TEAM_FEEDBACK_CLIP = [0.75, 1.25]
32 ATTENDANCE_FLOOR_RATIO = 0.88
33 P3_WEIGHTS = {'ticket_value': 0.35, 'broadcast_value': 0.35, 'attractiveness': 0.1, 'resource_cost': {'magnitude': 0.1, 'direction': 'subtract'}, 'risk': {
    'magnitude': 0.1, 'direction': 'subtract'}}
34 CP_SAT_SCALE = 1_000_000 # dimensionless integer objective scale
35 P2_TIME_LIMIT_SECONDS = 1800 # second, solver hard limit
36 P3_TIME_LIMIT_SECONDS = 300 # second, solver hard limit
37 assert MC_DRAWS == _FACTS['problem_3']['simulation_count']
38 assert MINIMUM_REST_HOURS == _FACTS['schedule_constants']['minimum_rest_hours']
39 assert abs(sum(abs(float(v if not isinstance(v, dict) else v['magnitude']))) for v in P2_WEIGHTS.values()) - 1.0) < 1e-12
40 print(['params'] 题目参数与跨问口径一致性 OK')

```

Listing 5: 共享数据与运行工具

```

1  """四个子问题共用的可复现、数据摄入和序列化工具。"""
2  import os, sys
3  _HERE = os.path.dirname(os.path.abspath(__file__))
4  if _HERE not in sys.path:
5      sys.path.insert(0, _HERE)
6
7  import json
8  import platform
9  import random
10 from importlib.metadata import PackageNotFoundError, version
11 from pathlib import Path
12
13 import numpy as np
14 import pandas as pd
15
16 ROOT = Path(_HERE).parent
17 DATA_FILE = ROOT / "user_data" / "C题_数据附件.xlsx"
18 FIGURES_DIR = ROOT / "figures"
19 OUTPUT_DIR = ROOT / "output"
20
21
22 def set_all_seeds(seed: int) -> None:
23     """固定 Python 与 NumPy 随机源；各模型另显式接收同一 seed。"""
24     random.seed(seed)
25     np.random.seed(seed)
26
27
28 def load_all_sheets() -> dict[str, pd.DataFrame]:
29     """完整读取 Excel；禁止依赖 pandas 默认首表行为。"""
30     sheets = pd.read_excel(DATA_FILE, sheet_name=None)
31     profile = json.loads((ROOT / "DATA_PROFILE.json").read_text(encoding="utf-8"))
32     expected = profile["files"][DATA_FILE.name]
33     actual_rows = int(sum(len(df) for df in sheets.values()))
34     assert len(sheets) == int(expected["n_sheets"]), (
35         f"【摄入不全】实读 {len(sheets)} 张表，建档为 {expected['n_sheets']} 张"
36     )

```

```

37     assert actual_rows == int(expected["total_rows"]), (
38         f"[摄入不全] 实读 {actual_rows} 行, 建档为 {expected['total_rows']} 行"
39     )
40     expected_names = set(expected["sheets"])
41     assert set(sheets) == expected_names, (
42         f"[摄入不全] sheet 集合差异: {sorted(set(sheets) ^ expected_names)}"
43     )
44     for name, df in sheets.items():
45         prof_rows = int(expected["sheets"][name]["rows"])
46         assert len(df) == prof_rows, f"[摄入不全] {name}: {len(df)} != {prof_rows}"
47     return sheets
48
49
50 def assert_key_equal(*dfs_and_cols: tuple[pd.DataFrame, str]) -> None:
51     """断言多张表的业务主键集合完全相同且各自唯一。"""
52     sets = []
53     for df, col in dfs_and_cols:
54         assert df[col].is_unique, f"主键 {col} 不唯一"
55         sets.append(set(df[col].astype(str)))
56     assert all(s == sets[0] for s in sets[1:]), "跨表主键集合不一致"
57
58
59 def write_json(path: Path, payload: dict) -> None:
60     """以 UTF-8 和有限浮点数写出机器可读结果。"""
61     path.parent.mkdir(parents=True, exist_ok=True)
62     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")
63
64
65 def runtime_metadata(seed: int) -> dict:
66     """记录复现实验所需的核心版本与运行环境。"""
67     def installed(package: str) -> str:
68         try:
69             return version(package)
70         except PackageNotFoundError:
71             return "not-installed"
72
73     return {
74         "seed": int(seed),
75         "python": platform.python_version(),
76         "numpy": np.__version__,
77         "pandas": pd.__version__,
78         "scipy": installed("scipy"),
79         "scikit_learn": installed("scikit-learn"),
80         "catboost": installed("catboost"),
81         "ortools": installed("ortools"),
82         "openpyxl": installed("openpyxl"),
83         "run_id": f"cumcm-c-{seed}",
84         "data_file": DATA_FILE.name,
85     }
86
87
88 def minmax(value, lower: float, upper: float):
89     """题定零极差规则的 Min-Max 归一化。"""
90     if abs(float(upper) - float(lower)) <= 1e-12:
91         return np.zeros_like(value, dtype=float) if hasattr(value, "__len__") else 0.0
92     return (value - lower) / (upper - lower)

```

Listing 6: 问题一完整程序

```

1  """问题一: 固定题定 split 的无泄漏观看人数时序预测。"""
2  import os, sys
3  _HERE = os.path.dirname(os.path.abspath(__file__))
4  if _HERE not in sys.path:
5      sys.path.insert(0, _HERE)
6
7  import math
8  import time
9  from collections import defaultdict, deque
10 from pathlib import Path
11
12 import numpy as np
13 import pandas as pd
14 from catboost import CatBoostRegressor
15 from sklearn.compose import ColumnTransformer

```

```

16 from sklearn.ensemble import ExtraTreesRegressor
17 from sklearn.impute import SimpleImputer
18 from sklearn.linear_model import ElasticNet
19 from sklearn.metrics import mean_absolute_error, mean_squared_error
20 from sklearn.model_selection import TimeSeriesSplit
21 from sklearn.pipeline import Pipeline
22 from sklearn.preprocessing import OneHotEncoder, StandardScaler
23
24 from params import MASTER_SEED, N_TIME_SPLITS, TARGET_SCALE
25 from utils import OUTPUT_DIR, FIGURES_DIR, load_all_sheets, runtime_metadata, set_all_seeds, write_json
26
27 POSTMATCH_FIELDS = ["goals", "xg", "shots", "possession", "attendance", "tv_viewers"]
28 POSTMATCH_DIRECT = {
29     "goals_a", "goals_b", "xg_a", "xg_b", "shots_a", "shots_b",
30     "possession_a", "possession_b", "attendance", "tv_viewers",
31 }
32 OMITTED_PREGAME = {"neutral"}
33 FORBIDDEN_DIRECT = POSTMATCH_DIRECT | OMITTED_PREGAME
34
35
36 def _team_metadata(teams: pd.DataFrame) -> dict[str, dict]:
37     return teams.set_index("team_name").to_dict(orient="index")
38
39
40 def _pre_probabilities(frame: pd.DataFrame) -> pd.DataFrame:
41     inv = np.column_stack([
42         1.0 / frame["odds_a"].to_numpy(float),
43         1.0 / frame["odds_draw"].to_numpy(float),
44         1.0 / frame["odds_b"].to_numpy(float),
45     ])
46     prob = inv / inv.sum(axis=1, keepdims=True)
47     out = pd.DataFrame(prob, columns=["p_a", "p_draw", "p_b"], index=frame.index)
48     out["odds_entropy"] = -(prob * np.log(np.clip(prob, 1e-15, 1.0))).sum(axis=1)
49     return out
50
51
52 def _empty_hist() -> dict[str, deque]:
53     return {f: deque(maxlen=30) for f in POSTMATCH_FIELDS}
54
55
56 def _history_features(hist: dict[str, deque], prefix: str) -> dict[str, float]:
57     ans: dict[str, float] = {}
58     for field in POSTMATCH_FIELDS:
59         values = np.asarray(hist[field], dtype=float)
60         values = values[np.isfinite(values)]
61         if len(values) == 0:
62             ans[f"{prefix}_{field}_roll5"] = np.nan
63             ans[f"{prefix}_{field}_ewm5"] = np.nan
64         else:
65             recent = values[-5:]
66             weights = np.power(0.5, np.arange(len(recent) - 1, -1, -1) / 5.0)
67             ans[f"{prefix}_{field}_roll5"] = float(recent.mean())
68             ans[f"{prefix}_{field}_ewm5"] = float(np.average(recent, weights=weights))
69     return ans
70
71
72 def build_features(hist: pd.DataFrame, teams: pd.DataFrame):
73     """逐场先提取过去、后更新状态，实现等价 shift(1) 的血缘安全特征。"""
74     frame = hist.copy()
75     frame["date"] = pd.to_datetime(frame["date"], utc=True)
76     frame = frame.sort_values(["date", "match_id"], kind="mergesort").reset_index(drop=True)
77     prob = _pre_probabilities(frame)
78     meta = _team_metadata(teams)
79     histories: defaultdict[str, dict[str, deque]] = defaultdict(_empty_hist)
80     rows: list[dict] = []
81     source_cutoff: list[str] = []
82     last_date: dict[str, pd.Timestamp] = {}
83     numeric_meta = [
84         "market_value_musd", "avg_age", "star_index", "fan_base_index",
85         "style_attack", "style_defense", "host_flag", "strength_score",
86     ]
87     # 同日可能出现重复记录：整日先提特征、后统一更新，保证严格 date_m < date_j。
88     for current_date, day in frame.groupby("date", sort=True):
89         pending = []
90         for idx, row in day.iterrows():
91             a, b = str(row["team_a"]), str(row["team_b"])

```

```

92     feat: dict[str, object] = {
93         "competition": str(row["competition"]),
94         "stage": str(row["stage"]),
95         "timezone_pair": "|".join(sorted([
96             str(meta.get(a, {}).get("home_timezone_region", "Unknown")),
97             str(meta.get(b, {}).get("home_timezone_region", "Unknown")),
98         ])),
99         "strength_rank_mean": (float(row["strength_rank_a"]) + float(row["strength_rank_b"])) / 2.0,
100        "strength_rank_gap": abs(float(row["strength_rank_a"]) - float(row["strength_rank_b"])),
101        "elo_mean": (float(row["elo_a"]) + float(row["elo_b"])) / 2.0,
102        "elo_gap": abs(float(row["elo_a"]) - float(row["elo_b"])),
103        "p_a": float(prob.loc[idx, "p_a"]),
104        "p_draw": float(prob.loc[idx, "p_draw"]),
105        "p_b": float(prob.loc[idx, "p_b"]),
106        "odds_entropy": float(prob.loc[idx, "odds_entropy"]),
107    }
108    for col in numeric_meta:
109        av = float(meta.get(a, {}).get(col, np.nan))
110        bv = float(meta.get(b, {}).get(col, np.nan))
111        feat[f"{col}_mean"] = np.nanmean([av, bv]) if np.isfinite([av, bv]).any() else np.nan
112        feat[f"{col}_gap"] = abs(av - bv) if np.isfinite(av) and np.isfinite(bv) else np.nan
113    feat.update(_history_features(histories[a], "a_hist"))
114    feat.update(_history_features(histories[b], "b_hist"))
115    feat["worldcup_stage"] = float(str(row["competition"]) == "World Cup")
116    feat["fan_stage_interaction"] = feat["fan_base_index_mean"] * feat["worldcup_stage"]
117    feat["star_stage_interaction"] = feat["star_index_mean"] * feat["worldcup_stage"]
118    feat["strength_entropy_interaction"] = feat["strength_score_mean"] * feat["odds_entropy"]
119    rows.append(feat)
120    prior_dates = [last_date[t] for t in (a, b) if t in last_date]
121    source_cutoff.append(max(prior_dates).isoformat() if prior_dates else "none")
122    pending.append((a, b, row))
123    for a, b, row in pending:
124        for team, suffix in ((a, "a"), (b, "b")):
125            values = {
126                "goals": row[f"goals_{suffix}"], "xg": row[f"xg_{suffix}"],
127                "shots": row[f"shots_{suffix}"], "possession": row[f"possession_{suffix}"],
128                "attendance": row["attendance"], "tv_viewers": row["tv_viewers"],
129            }
130            for field, value in values.items():
131                if pd.notna(value):
132                    histories[team][field].append(float(value))
133            last_date[team] = current_date
134    X = pd.DataFrame(rows)
135    assert not (set(X.columns) & FORBIDDEN_DIRECT), "同场赛后字段进入直接特征"
136    for cutoff, kickoff in zip(source_cutoff, frame["date"]):
137        if cutoff != "none":
138            assert pd.Timestamp(cutoff) < kickoff, "发现 future-row 历史聚合"
139    return frame, X, histories, last_date
140
141
142 def features_with_validation_targets_masked(hist: pd.DataFrame, teams: pd.DataFrame,
143                                           validation_ids: list[str]) -> pd.DataFrame:
144     """模拟整段验证集观看标签均未知，但允许较早比赛的其他赛后事实滚动进入。"""
145     masked = hist.copy()
146     target_rows = masked["match_id"].astype(str).isin(set(validation_ids))
147     assert int(target_rows.sum()) == len(validation_ids)
148     masked.loc[target_rows, "tv_viewers"] = np.nan
149     rebuilt_hist, rebuilt_X, _, _ = build_features(masked, teams)
150     assert rebuilt_hist["match_id"].astype(str).tolist() == hist["match_id"].astype(str).tolist()
151     return rebuilt_X.loc[rebuilt_hist["dataset_split"].eq("train")].reset_index(drop=True)
152
153
154 def build_group_features(groups: pd.DataFrame, base: pd.DataFrame, teams: pd.DataFrame,
155                         histories: dict[str, dict[str, deque]]) -> pd.DataFrame:
156     merged = groups.merge(base, on=["match_id", "group_id", "round_in_group", "team_a", "team_b"], validate="one_to_one")
157     meta = _team_metadata(teams)
158     numeric_meta = [
159         "market_value_musd", "avg_age", "star_index", "fan_base_index",
160         "style_attack", "style_defense", "host_flag", "strength_score",
161     ]
162     rows = []
163     for _, row in merged.iterrows():
164         a, b = str(row["team_a"]), str(row["team_b"])
165         p = np.array([row["p_a_win"], row["p_draw"], row["p_b_win"]], dtype=float)
166         p = p / p.sum()
167         feat: dict[str, object] = {

```

```

168         "competition": "World Cup", "stage": "Group",
169         "timezone_pair": "|".join(sorted([str(meta[a]["home_timezone_region"]), str(meta[b]["home_timezone_region"])])),
170         "strength_rank_mean": (float(meta[a]["strength_rank"]) + float(meta[b]["strength_rank"])) / 2.0,
171         "strength_rank_gap": abs(float(meta[a]["strength_rank"]) - float(meta[b]["strength_rank"])),
172         "elo_mean": (float(meta[a]["elo_rating"]) + float(meta[b]["elo_rating"])) / 2.0,
173         "elo_gap": abs(float(meta[a]["elo_rating"]) - float(meta[b]["elo_rating"])),
174         "p_a": float(p[0]), "p_draw": float(p[1]), "p_b": float(p[2]),
175         "odds_entropy": float(-(p * np.log(np.clip(p, 1e-15, 1.0))).sum()),
176     }
177     for col in numeric_meta:
178         av, bv = float(meta[a][col]), float(meta[b][col])
179         feat[f"{col}_mean"] = (av + bv) / 2.0
180         feat[f"{col}_gap"] = abs(av - bv)
181     feat.update(_history_features(histories[a], "a_hist"))
182     feat.update(_history_features(histories[b], "b_hist"))
183     feat["worldcup_stage"] = 1.0
184     feat["fan_stage_interaction"] = feat["fan_base_index_mean"]
185     feat["star_stage_interaction"] = feat["star_index_mean"]
186     feat["strength_entropy_interaction"] = feat["strength_score_mean"] * feat["odds_entropy"]
187     rows.append(feat)
188     return pd.DataFrame(rows)
189
190
191 def _columns(X: pd.DataFrame):
192     categorical = list(X.select_dtypes(include=["object", "category"]).columns)
193     numeric = [c for c in X.columns if c not in categorical]
194     return numeric, categorical
195
196
197 def _pipeline(model, X: pd.DataFrame, scale: bool) -> Pipeline:
198     numeric, categorical = _columns(X)
199     num_steps = [("impute", SimpleImputer(strategy="median"))]
200     if scale:
201         num_steps.append(("scale", StandardScaler()))
202     prep = ColumnTransformer([
203         ("num", Pipeline(num_steps), numeric),
204         ("cat", Pipeline([
205             ("impute", SimpleImputer(strategy="most_frequent")),
206             ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
207         ]), categorical),
208     ])
209     return Pipeline([("prep", prep), ("model", model)])
210
211
212 def _inner_split(n: int) -> tuple[np.ndarray, np.ndarray]:
213     cut = max(20, int(math.floor(n * 0.8)))
214     cut = min(cut, n - 1)
215     return np.arange(cut), np.arange(cut, n)
216
217
218 def _fit_elastic(X: pd.DataFrame, y: np.ndarray):
219     tr, va = _inner_split(len(X))
220     best = None
221     for alpha in [1e-3, 1e-2, 1e-1, 1.0, 10.0]:
222         for rho in [0.1, 0.5, 0.9]:
223             model = _pipeline(ElasticNet(alpha=alpha, l1_ratio=rho, max_iter=20000, random_state=MASTER_SEED), X, True)
224             model.fit(X.iloc[tr], y[tr])
225             mse = mean_squared_error(y[va], model.predict(X.iloc[va]))
226             if best is None or mse < best[0]:
227                 best = (mse, alpha, rho)
228     final = _pipeline(ElasticNet(alpha=best[1], l1_ratio=best[2], max_iter=20000, random_state=MASTER_SEED), X, True)
229     final.fit(X, y)
230     return final, {"alpha": best[1], "l1_ratio": best[2]}
231
232
233 def _fit_extra(X: pd.DataFrame, y: np.ndarray):
234     tr, va = _inner_split(len(X))
235     best = None
236     for leaf in [3, 5, 8]:
237         for max_features in [0.6, 0.8, 1.0]:
238             est = ExtraTreesRegressor(n_estimators=600, min_samples_leaf=leaf,
239                                     max_features=max_features, random_state=MASTER_SEED, n_jobs=-1)
240             model = _pipeline(est, X, False)
241             model.fit(X.iloc[tr], y[tr])
242             mse = mean_squared_error(y[va], model.predict(X.iloc[va]))
243             if best is None or mse < best[0]:

```

```

244         best = (mse, leaf, max_features)
245     est = ExtraTreesRegressor(n_estimators=600, min_samples_leaf=best[1],
246                             max_features=best[2], random_state=MASTER_SEED, n_jobs=-1)
247     final = _pipeline(est, X, False)
248     final.fit(X, y)
249     return final, {"min_samples_leaf": best[1], "max_features": best[2]}
250
251
252 def _cat_data(X: pd.DataFrame, medians: pd.Series | None = None):
253     numeric, categorical = _columns(X)
254     z = X.copy()
255     if medians is None:
256         medians = z[numeric].median()
257     z[numeric] = z[numeric].fillna(medians).astype(float)
258     z[categorical] = z[categorical].fillna("<missing>").astype(str)
259     return z, categorical, medians
260
261
262 def _fit_cat(X: pd.DataFrame, y: np.ndarray):
263     tr, va = _inner_split(len(X))
264     best = None
265     for depth in [4, 5, 6]:
266         for l2 in [3, 10]:
267             ztr, cats, med = _cat_data(X.iloc[tr])
268             zva, _, _ = _cat_data(X.iloc[va], med)
269             model = CatBoostRegressor(
270                 loss_function="RMSE", eval_metric="RMSE", has_time=True,
271                 iterations=3000, learning_rate=3e-2, depth=depth,
272                 l2_leaf_reg=l2, random_seed=MASTER_SEED, verbose=False,
273                 thread_count=1, allow_writing_files=False,
274             )
275             model.fit(ztr, y[tr], cat_features=cats, eval_set=(zva, y[va]),
276                     early_stopping_rounds=150, verbose=False)
277             mse = mean_squared_error(y[va], model.predict(zva))
278             iters = max(50, int(model.get_best_iteration()) + 1)
279             if best is None or mse < best[0]:
280                 best = (mse, depth, l2, iters)
281     z, cats, med = _cat_data(X)
282     final = CatBoostRegressor(
283         loss_function="RMSE", eval_metric="RMSE", has_time=True,
284         iterations=best[3], learning_rate=3e-2, depth=best[1],
285         l2_leaf_reg=best[2], random_seed=MASTER_SEED, verbose=False,
286         thread_count=1, allow_writing_files=False,
287     )
288     final.fit(z, y, cat_features=cats, verbose=False)
289     return (final, med), {"depth": best[1], "l2_leaf_reg": best[2], "iterations": best[3]}
290
291
292 def _predict(model_name: str, fitted, X: pd.DataFrame) -> np.ndarray:
293     if model_name == "CatBoost":
294         model, med = fitted
295         z, _, _ = _cat_data(X, med)
296         return model.predict(z)
297     return fitted.predict(X)
298
299
300 def _fit(model_name: str, X: pd.DataFrame, y: np.ndarray):
301     if model_name == "ElasticNet":
302         return _fit_elastic(X, y)
303     if model_name == "ExtraTrees":
304         return _fit_extra(X, y)
305     if model_name == "CatBoost":
306         return _fit_cat(X, y)
307     raise ValueError(model_name)
308
309
310 def _fit_fixed(model_name: str, X: pd.DataFrame, y: np.ndarray, params: dict):
311     """用最早训练窗内选出的冻结参数重拟合，避免在每个外折重复整套网格。"""
312     if model_name == "ElasticNet":
313         model = _pipeline(ElasticNet(alpha=params["alpha"], l1_ratio=params["l1_ratio"],
314                                     max_iter=20000, random_state=MASTER_SEED), X, True)
315         model.fit(X, y)
316         return model
317     if model_name == "ExtraTrees":
318         est = ExtraTreesRegressor(n_estimators=600, min_samples_leaf=params["min_samples_leaf"],
319                                 max_features=params["max_features"], random_state=MASTER_SEED, n_jobs=-1)

```

```

320     model = _pipeline(est, X, False)
321     model.fit(X, y)
322     return model
323 if model_name == "CatBoost":
324     z, cats, med = _cat_data(X)
325     model = CatBoostRegressor(
326         loss_function="RMSE", eval_metric="RMSE", has_time=True,
327         iterations=params["iterations"], learning_rate=3e-2, depth=params["depth"],
328         l2_leaf_reg=params["l2_leaf_reg"], random_seed=MASTER_SEED, verbose=False,
329         thread_count=1, allow_writing_files=False,
330     )
331     model.fit(z, y, cat_features=cats, verbose=False)
332     return model, med
333 raise ValueError(model_name)
334
335
336 def solve() -> dict:
337     set_all_seeds(MASTER_SEED)
338     t0 = time.time()
339     sheets = load_all_sheets()
340     hist, X_all, histories, _ = build_features(sheets["historical_matches"], sheets["teams"])
341     train_mask = hist["dataset_split"].eq("train")
342     test_mask = hist["dataset_split"].eq("test")
343     assert int(train_mask.sum()) == 560 and int(test_mask.sum()) == 140
344     assert hist.loc[test_mask, "tv_viewers"].isna().all()
345     train_ids = hist.loc[train_mask, "match_id"].astype(str).tolist()
346     test_ids = hist.loc[test_mask, "match_id"].astype(str).tolist()
347     assert set(train_ids).isdisjoint(test_ids)
348     X_train = X_all.loc[train_mask].reset_index(drop=True)
349     X_test = X_all.loc[test_mask].reset_index(drop=True)
350     y = hist.loc[train_mask, "tv_viewers"].to_numpy(float) / TARGET_SCALE
351
352     folds = list(TimeSeriesSplit(n_splits=5).split(X_train))
353     for tr, va in folds:
354         assert hist.loc[train_mask].iloc[tr]["date"].max() < hist.loc[train_mask].iloc[va]["date"].min()
355     model_names = ["MeanBaseline", "ElasticNet", "ExtraTrees", "CatBoost"]
356     fold_metrics: list[dict] = []
357     oof = {m: np.full(len(y), np.nan) for m in model_names}
358     chosen_params: dict[str, list[dict]] = defaultdict(list)
359     # 所有超参数只用最早外折的训练段选择，之后冻结；test 和任何外折验证标签均不参与。
360     earliest_train = folds[0][0]
361     _, inner_valid = _inner_split(len(earliest_train))
362     inner_valid_global = earliest_train[inner_valid]
363     inner_valid_ids = np.asarray(train_ids)[inner_valid_global].tolist()
364     X_inner_masked = features_with_validation_targets_masked(hist, sheets["teams"], inner_valid_ids)
365     X_earliest = X_inner_masked.iloc[earliest_train].reset_index(drop=True)
366     frozen_params = {}
367     for model_name in model_names[1:]:
368         print(f"[P1] selecting {model_name} on earliest expanding train window", flush=True)
369         _, frozen_params[model_name] = _fit(model_name, X_earliest, y[earliest_train])
370     for fold_no, (tr, va) in enumerate(folds, 1):
371         print(f"[P1] fold {fold_no}/5 train={len(tr)} valid={len(va)}", flush=True)
372         validation_ids = np.asarray(train_ids)[va].tolist()
373         X_fold = features_with_validation_targets_masked(hist, sheets["teams"], validation_ids)
374         mean_pred = np.full(len(va), y[tr].mean())
375         oof["MeanBaseline"][va] = mean_pred
376         for model_name in model_names:
377             if model_name == "MeanBaseline":
378                 pred = mean_pred
379                 params = {}
380             else:
381                 params = frozen_params[model_name]
382                 fitted = _fit_fixed(model_name, X_train.iloc[tr], y[tr], params)
383                 pred = _predict(model_name, fitted, X_fold.iloc[va])
384                 oof[model_name][va] = pred
385                 chosen_params[model_name].append(params)
386         fold_metrics.append({
387             "fold": fold_no, "model": model_name,
388             "mse": float(mean_squared_error(y[va], pred)),
389             "rmse": float(mean_squared_error(y[va], pred) ** 0.5),
390             "mae": float(mean_absolute_error(y[va], pred)),
391             "train_end": hist.loc[train_mask].iloc[tr]["date"].max().isoformat(),
392             "valid_start": hist.loc[train_mask].iloc[va]["date"].min().isoformat(),
393         })
394
395     summary = []

```

```

396 for name in model_names:
397     rows = [r for r in fold_metrics if r["model"] == name]
398     summary.append({"model": name,
399                    "mean_mse": float(np.mean([r["mse"] for r in rows])),
400                    "mean_rmse": float(np.mean([r["rmse"] for r in rows])),
401                    "mean_mae": float(np.mean([r["mae"] for r in rows]))})
402 summary.sort(key=lambda r: (r["mean_mse"], model_names.index(r["model"])))
403 best_mse = summary[0]["mean_mse"]
404 eligible = [r for r in summary if r["mean_mse"] <= best_mse * 1.01]
405 selected = min(eligible, key=lambda r: model_names.index(r["model"]))["model"]
406 print(f"[P1] selected={selected} mean_mse={best_mse:.6f}", flush=True)
407 if selected == "MeanBaseline":
408     fitted = float(y.mean())
409     test_pred = np.full(len(X_test), fitted)
410     final_params = {}
411 else:
412     final_params = frozen_params[selected]
413     fitted = _fit_fixed(selected, X_train, y, final_params)
414     test_pred = _predict(selected, fitted, X_test)
415
416 X_group = build_group_features(sheets["groups_matches"], sheets["base_predictions"], sheets["teams"], histories)
417 if selected == "MeanBaseline":
418     group_pred = np.full(len(X_group), fitted)
419 else:
420     group_pred = _predict(selected, fitted, X_group)
421 test_pred = np.maximum(test_pred, 1e-6)
422 group_pred = np.maximum(group_pred, 1e-6)
423 valid = np.isfinite(oof[selected])
424 residuals = y[valid] - oof[selected][valid]
425 q05, q95 = np.quantile(residuals, [0.05, 0.95])
426
427 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
428 test_out = pd.DataFrame({"match_id_test": test_ids, "predicted_test_tv_viewers": test_pred})
429 group_info = sheets["groups_matches"]
430 group_out = pd.DataFrame({
431     "match_id": group_info["match_id"], "team_a": group_info["team_a"], "team_b": group_info["team_b"],
432     "predicted_tv_viewers": np rint(group_pred * TARGET_SCALE).astype(np.int64),
433 })
434 test_out.to_csv(OUTPUT_DIR / "result_1_test_prediction.csv", index=False, encoding="utf-8-sig")
435 group_out.to_csv(OUTPUT_DIR / "result_1_match_prediction.csv", index=False, encoding="utf-8-sig")
436 assert len(test_out) == 140 and test_out["match_id_test"].is_unique
437 assert len(group_out) == 72 and group_out["match_id"].is_unique
438 assert np.isfinite(test_pred).all() and (test_pred > 0).all()
439
440 lineage = []
441 for col in X_train.columns:
442     history = "_hist_" in col
443     lineage.append({
444         "feature_name": col,
445         "source_table": "historical_matches/teams/base_predictions" if not history else "historical_matches",
446         "source_column": col,
447         "source_time_rule": "strictly_before_kickoff_shift1" if history else "pregame_available",
448         "fitted_on_ids": "outer_fold_train_only; final=560 train ids",
449     })
450 selected_oof = oof[selected][valid]
451 result = {
452     "metadata": {"runtime_metadata(MASTER_SEED)": time.time() - t0},
453     "method": "fixed_split + TimeSeriesSplit(n_splits=5) + ElasticNet/ExtraTrees/CatBoost",
454     "dataset": {"sheets": 15, "total_rows": 2241, "train_rows": 560, "test_rows": 140, "group_rows": 72},
455     "feature_lineage": lineage,
456     "forbidden_direct_features": sorted(FORBIDDEN_DIRECT),
457     "postmatch_direct_features": sorted(POSTMATCH_DIRECT),
458     "intentionally_omitted_pregame_features": sorted(OMITTED_PREGAME),
459     "validation_target_history_policy": "mask every validation-fold tv_viewers label before rebuilding lag features",
460     "fold_metrics": fold_metrics,
461     "model_summary": summary,
462     "selected_model": selected,
463     "selected_params": final_params,
464     "oof": {
465         "match_id": np.asarray(train_ids)[valid].tolist(),
466         "date": hist.loc[train_mask].iloc[np.where(valid)[0]]["date"].astype(str).tolist(),
467         "actual_million": y[valid].tolist(),
468         "predicted_million": selected_oof.tolist(),
469         "residual_million": residuals.tolist(),
470     },
471     "prediction_interval_residual_quantiles": {"q05": float(q05), "q95": float(q95)},

```

```

472     "test_prediction_summary_million": {"min": float(test_pred.min()), "mean": float(test_pred.mean()), "max": float(test_pred.max())},
473     "group_prediction_summary_person": {"min": int(group_out.predicted_tv_viewers.min()), "mean": float(group_out.predicted_tv_viewers.mean()), "max":
int(group_out.predicted_tv_viewers.max())},
474     "outputs": ["output/result_1_test_prediction.csv", "output/result_1_match_prediction.csv"],
475     "seed": MASTER_SEED,
476 }
477 write_json(FIGURES_DIR / "problem_1_results.json", result)
478 print("[P1] validate_capability PASS: 固定split、赛前血缘、同折三模型、140+72交付", flush=True)
479 return result
480
481
482 if __name__ == "__main__":
483     solve()

```

Listing 7: 问题二完整程序

```

1  """问题二：72 场比赛一场馆一时段三维联合 CP-SAT 排程。"""
2  import os, sys
3  _HERE = os.path.dirname(os.path.abspath(__file__))
4  if _HERE not in sys.path:
5      sys.path.insert(0, _HERE)
6
7  import math
8  import time
9  from collections import defaultdict
10 from datetime import datetime
11 from pathlib import Path
12 from zoneinfo import ZoneInfo
13
14 import numpy as np
15 import pandas as pd
16
17 cp_model = None
18
19 from params import (
20     CP_SAT_SCALE, GLOBAL_PRIME_FRACTION, HIGH_SECURITY_THRESHOLD,
21     LARGE_VENUE_FRACTION, MASTER_SEED, MAX_PRIME_COUNT_RANGE,
22     MINIMUM_REST_HOURS, P2_TIME_LIMIT_SECONDS, SECURITY_BASE_COST_USD,
23 )
24 from utils import FIGURES_DIR, OUTPUT_DIR, load_all_sheets, minmax, runtime_metadata, set_all_seeds, write_json
25
26
27 def _round_ticket(ticket: pd.DataFrame) -> dict[int, dict]:
28     return {r: ticket.loc[ticket["match_stage"].eq(f"Group_Match_R{r}")] iloc[0].to_dict() for r in (1, 2, 3)}
29
30
31 def _distance_lookup(distance: pd.DataFrame):
32     return {(str(r.origin_id), str(r.destination_id)): {
33         "distance": float(r.distance_km), "time": float(r.travel_time_hour), "timezone": float(r.timezone_diff)
34     } for r in distance.iteruples(index=False)}
35
36
37 def compute_travel(groups: pd.DataFrame, venues: pd.DataFrame, security: pd.DataFrame,
38     distance: pd.DataFrame) -> dict[tuple[int, int], float]:
39     """严格按上一轮全部安保合格场馆均值计算候选旅行负担。"""
40     if "required_security_level" in groups.columns:
41         g = groups.copy()
42     else:
43         g = groups.merge(security[["match_id", "required_security_level"]], on="match_id", validate="one_to_one")
44     venue_ids = venues["venue_id"].astype(str).tolist()
45     venue_sec = venues.set_index("venue_id")["security_level"].to_dict()
46     lookup = _distance_lookup(distance)
47     team_round_match = {}
48     for i, row in g.iterrows():
49         for team_id in (str(row.team_a_id), str(row.team_b_id)):
50             team_round_match[(team_id, int(row.round_in_group))] = i
51     raw = {}
52     legal_values = {q: [] for q in ("distance", "time", "timezone")}
53     for i, row in g.iterrows():
54         r = int(row.round_in_group)
55         current_req = int(row.required_security_level)
56         for team_id in (str(row.team_a_id), str(row.team_b_id)):
57             for v in venue_ids:
58                 if int(venue_sec[v]) < current_req:

```

```

59         continue
60     if r == 1:
61         vals = lookup[(team_id, v)]
62     else:
63         prev_i = team_round_match[(team_id, r - 1)]
64         prev_req = int(g.iloc[prev_i].required_security_level)
65         origins = [u for u in venue_ids if int(venue_sec[u]) >= prev_req]
66         vals = {q: float(np.mean([lookup[(u, v)][q] for u in origins])) for q in legal_values}
67         raw[(team_id, r, v)] = vals
68         for q in legal_values:
69             legal_values[q].append(vals[q])
70     bounds = {q: (min(values), max(values)) for q, values in legal_values.items()}
71     burden = {}
72     for i, row in g.iterrows():
73         r = int(row.round_in_group)
74         for vi, v in enumerate(venue_ids):
75             if int(venue_sec[v]) < int(row.required_security_level):
76                 continue
77             team_values = []
78             for team_id in (str(row.team_a_id), str(row.team_b_id)):
79                 vals = raw[(team_id, r, v)]
80                 team_values.append(
81                     0.5 * minmax(vals["distance"], *bounds["distance"])
82                     + 0.3 * minmax(vals["time"], *bounds["time"])
83                     + 0.2 * minmax(vals["timezone"], *bounds["timezone"])
84                 )
85             burden[(i, vi)] = float(np.mean(team_values))
86     return burden
87
88
89 def prepare_data():
90     sheets = load_all_sheets()
91     p1 = pd.read_csv(OUTPUT_DIR / "result_1_match_prediction.csv")
92     groups = sheets["groups_matches"].merge(sheets["base_predictions"], on=["match_id", "group_id", "round_in_group", "team_a", "team_b"], validate="one_to_one")
93     groups = groups.merge(sheets["security_requirements"][["match_id", "security_demand_score", "required_security_level"]], on="match_id", validate="one_to_one")
94     groups = groups.merge(p1[["match_id", "predicted_tv_viewers"]], on="match_id", validate="one_to_one")
95     groups = groups.reset_index(drop=True)
96     venues = sheets["venues"].reset_index(drop=True)
97     slots = sheets["time_slots"].reset_index(drop=True)
98     slots["utc"] = pd.to_datetime(slots["reference_utc_time"], utc=True)
99     t0 = slots["utc"].min()
100    slots["utc_hour"] = ((slots["utc"] - t0).dt.total_seconds() / 3600).round().astype(int)
101    limits = sheets["dynamic_resource_limits"].copy()
102    limits["reference_date"] = pd.to_datetime(limits["reference_date"]).dt.date.astype(str)
103    travel = compute_travel(groups, venues, sheets["security_requirements"], sheets["distance_matrix"])
104    return sheets, groups, venues, slots, limits, travel
105
106
107 def build_model(groups, venues, slots, limits, travel, ticket, allowed_slots_by_round=None):
108     global cp_model
109     if cp_model is None:
110         from ortools.sat.python import cp_model as _cp_model
111         cp_model = _cp_model
112     model = cp_model.CpModel()
113     n_i, n_v, n_s = len(groups), len(venues), len(slots)
114     x_ivs = {}
115     by_match, by_venue_slot, by_venue = defaultdict(list), defaultdict(list), defaultdict(list)
116     for i in range(n_i):
117         req = int(groups.loc[i, "required_security_level"])
118         match_round = int(groups.loc[i, "round_in_group"])
119         for v in range(n_v):
120             if int(venues.loc[v, "security_level"]) < req:
121                 continue
122             for s in range(n_s):
123                 if allowed_slots_by_round is not None and s not in allowed_slots_by_round[match_round]:
124                     continue
125                 var = model.NewBoolVar(f"x_ivs_{i}_{v}_{s}")
126                 x_ivs[(i, v, s)] = var
127                 by_match[i].append(var)
128                 by_venue_slot[(v, s)].append(var)
129                 by_venue[v].append(var)
130     for i in range(n_i):
131         assert by_match[i], f"比赛 {i} 无安保合格候选"
132         model.Add(cp_model.LinearExpr.Sum(by_match[i]) == 1)

```

```

133     for vars_ in by_venue_slot.values():
134         model.Add(cp_model.LinearExpr.Sum(vars_) <= 1)
135
136     y_v = {}
137     venue_totals = {}
138     for v in range(n_v):
139         y_v[v] = model.NewBoolVar(f"y_v_{v}")
140         total = cp_model.LinearExpr.Sum(by_venue[v])
141         venue_totals[v] = total
142         lo, hi = int(venues.loc[v, "min_total_matches"]), int(venues.loc[v, "max_total_matches"])
143         model.Add(total >= lo * y_v[v])
144         model.Add(total <= hi * y_v[v])
145         model.Add(y_v[v] <= total)
146         # 题定 lo>=2, 所有场馆必须启用; 此断言使一次性 setup 成本逻辑透明。
147         model.Add(y_v[v] == 1)
148
149     # 场馆当地日负荷。
150     for v in range(n_v):
151         tz = ZoneInfo(str(venues.loc[v, "timezone"]))
152         local_dates = [slots.loc[s, "utc"].to_pydatetime().astimezone(tz).date().isoformat() for s in range(n_s)]
153         for day in sorted(set(local_dates)):
154             vars_ = [x_ivs[(i, v, s)] for (i, vv, s) in x_ivs if vv == v and local_dates[s] == day]
155             model.Add(cp_model.LinearExpr.Sum(vars_) <= int(venues.loc[v, "max_matches_per_day"]))
156
157     # 每队轮次时刻及 60 小时休息。
158     team_round_i = {}
159     for i, row in groups.iterrows():
160         team_round_i[(str(row.team_a_id), int(row.round_in_group))] = i
161         team_round_i[(str(row.team_b_id), int(row.round_in_group))] = i
162     for team in sorted({k[0] for k in team_round_i}):
163         expressions = {}
164         for r in (1, 2, 3):
165             i = team_round_i[(team, r)]
166             expressions[r] = cp_model.LinearExpr.Sum([
167                 int(slots.loc[s, "utc_hour"]) * x_ivs[(i, v, s)]
168                 for (ii, v, s) in x_ivs if ii == i
169             ])
170             model.Add(expressions[2] - expressions[1] >= MINIMUM_REST_HOURS)
171             model.Add(expressions[3] - expressions[2] >= MINIMUM_REST_HOURS)
172
173     prime_n = max(1, int(round(n_s * GLOBAL_PRIME_FRACTION)))
174     prime_slots = set(slots.nlargest(prime_n, "global_prime_score").index.astype(int))
175     large_n = max(1, int(round(n_v * LARGE_VENUE_FRACTION)))
176     large_venues = set(venues.nlargest(large_n, "capacity").index.astype(int))
177     team_prime, team_large = {}, {}
178     for team in sorted({k[0] for k in team_round_i}):
179         match_ids = {team_round_i[(team, r)] for r in (1, 2, 3)}
180         team_prime[team] = cp_model.LinearExpr.Sum([
181             var for (i, v, s), var in x_ivs.items() if i in match_ids and s in prime_slots
182         ])
183         team_large[team] = cp_model.LinearExpr.Sum([
184             var for (i, v, s), var in x_ivs.items() if i in match_ids and v in large_venues
185         ])
186     pmax, pmin = model.NewIntVar(0, 3, "pmax"), model.NewIntVar(0, 3, "pmin")
187     lmax, lmin = model.NewIntVar(0, 3, "lmax"), model.NewIntVar(0, 3, "lmin")
188     for team in team_prime:
189         model.Add(team_prime[team] <= pmax); model.Add(team_prime[team] >= pmin)
190         model.Add(team_large[team] <= lmax); model.Add(team_large[team] >= lmin)
191     model.Add(pmax - pmin <= MAX_PRIME_COUNT_RANGE)
192
193     # 第三轮逐日高安保容量。
194     limit_by_day = limits.set_index("reference_date")["high_security_capacity"].to_dict()
195     for day, cap in limit_by_day.items():
196         slot_ids = set(slots.index[slots["date"].astype(str).eq(day)].astype(int))
197         vars_ = [var for (i, v, s), var in x_ivs.items()
198                 if int(groups.loc[i, "round_in_group"]) == 3
199                 and int(groups.loc[i, "required_security_level"]) >= HIGH_SECURITY_THRESHOLD and s in slot_ids]
200         model.Add(cp_model.LinearExpr.Sum(vars_) <= int(cap))
201     for s in range(n_s):
202         model.Add(cp_model.LinearExpr.Sum([
203             var for v in range(n_v) for var in by_venue_slot[(v, s)]
204         ]) <= int(slots.loc[s, "broadcast_capacity"]))
205
206     rt = _round_ticket(ticket)
207     T_terms, B_terms, C_terms, D_terms = [], [], [], [], []
208     contributions = {}

```

```

209     for (i, v, s), var in x_ivs.items():
210         r = int(groups.loc[i, "round_in_group"])
211         n_att = min(float(groups.loc[i, "expected_attendance_base"]), float(venues.loc[v, "capacity"]))
212         tval = float(rt[r]["base_ticket_price_usd"]) * n_att
213         bval = (float(groups.loc[i, "predicted_tv_viewers"]) * float(rt[r]["broadcast_unit_value_usd"])
214               * float(slots.loc[s, "global_prime_score"]) * float(rt[r]["sponsor_weight"]))
215         cval = float(venues.loc[v, "operation_cost_musd_per_match"]) * 1_000_000 + SECURITY_BASE_COST_USD * int(groups.loc[i, "required_security_level"]) *
216             float(venues.loc[v, "security_cost_index"])
217         dval = float(travel[(i, v)])
218         rval = (0.5 * float(venues.loc[v, "climate_risk"])
219               + 0.3 * n_att / float(venues.loc[v, "capacity"])
220               + 0.2 * int(groups.loc[i, "required_security_level"]) / float(venues.loc[v, "security_level"]))
221         contributions[(i, v, s)] = (tval, bval, cval, dval, rval, n_att)
222         T_terms.append(round(tval) * var); B_terms.append(round(bval) * var)
223         C_terms.append(round(cval) * var); D_terms.append(round(dval * CP_SAT_SCALE) * var)
224         R_terms.append(round(rval * CP_SAT_SCALE) * var)
225     for v, var in y_v.items():
226         C_terms.append(round(float(venues.loc[v, "setup_cost_musd"]) * 1_000_000) * var)
227     exprs = {"T": cp_model.LinearExpr.Sum(T_terms),
228            "B": cp_model.LinearExpr.Sum(B_terms),
229            "C": cp_model.LinearExpr.Sum(C_terms),
230            "Dsum": cp_model.LinearExpr.Sum(D_terms),
231            "Rsum": cp_model.LinearExpr.Sum(R_terms)}
232     aux = {"x_ivs": x_ivs, "y_v": y_v, "pmax": pmax, "pmin": pmin,
233           "lmax": lmax, "lmin": lmin, "prime_slots": prime_slots,
234           "large_venues": large_venues, "contributions": contributions,
235           "team_round_i": team_round_i}
236     return model, exprs, aux
237
238 def _solver(seconds: float, gap: float = 0.0):
239     solver = cp_model.CpSolver()
240     solver.parameters.max_time_in_seconds = float(seconds)
241     solver.parameters.num_search_workers = 1
242     solver.parameters.random_seed = MASTER_SEED
243     solver.parameters.relative_gap_limit = float(gap)
244     return solver
245
246
247 def _status_name(solver, status):
248     return solver.StatusName(status)
249
250
251 def candidate_envelope_bounds(aux, groups: pd.DataFrame, venues: pd.DataFrame):
252     """按附件“全部候选组合”口径构造确定性 Min--Max 包络。
253
254     对每场比赛在其全部安保合格（场馆，时段）候选中分别取最小值和
255     最大值，再跨 72 场求和。任何满足联合硬约束的赛程都是该苗卡尔
256     候选域的子集，故其 T/B/C 必位于该包络内。附件数据中每个场馆的
257     min_total_matches 均大于 0，一次性启用成本因此是所有可行方案共有的
258     常数，并在 C 的两端同时计入。
259     """
260     assert (venues["min_total_matches"].astype(int) > 0).all(), "当前数据并非全部场馆必启用"
261     by_match = defaultdict(list)
262     for (i, _v, _s), values in aux["contributions"].items():
263         by_match[int(i)].append(values)
264     assert set(by_match) == set(range(len(groups))) and all(by_match.values())
265
266     setup = float(venues["setup_cost_musd"].sum()) * 1_000_000
267     bounds = {
268         "T_min": float(sum(min(x[0] for x in by_match[i]) for i in range(len(groups)))),
269         "T_max": float(sum(max(x[0] for x in by_match[i]) for i in range(len(groups)))),
270         "B_min": float(sum(min(x[1] for x in by_match[i]) for i in range(len(groups)))),
271         "B_max": float(sum(max(x[1] for x in by_match[i]) for i in range(len(groups)))),
272         "C_min": float(setup + sum(min(x[2] for x in by_match[i]) for i in range(len(groups)))),
273         "C_max": float(setup + sum(max(x[2] for x in by_match[i]) for i in range(len(groups)))),
274     }
275     assert all(bounds[f"{key}_max"] > bounds[f"{key}_min"] for key in ("T", "B", "C"))
276     meta = {
277         "method": "per-match envelope over all legal match-venue-slot candidates",
278         "candidate_count": int(len(aux["x_ivs"])),
279         "match_count": int(len(groups)),
280         "fixed_setup_cost_usd": setup,
281         "all_venues_mandatory": True,
282         "solver_used": False,
283     }

```

```

284     return bounds, meta
285
286
287 def extract_schedule(solver, groups, venues, slots, aux):
288     rows = []
289     for i, match in groups.iterrows():
290         selected = [(v, s) for (ii, v, s), var in aux["x_ivs"].items() if ii == i and solver.Value(var)]
291         assert len(selected) == 1
292         v, s = selected[0]
293         utc = slots.loc[s, "utc"].to_pydatetime()
294         local = utc.astimezone(ZoneInfo(str(venues.loc[v, "timezone"])))
295         tval, bval, _, dval, rval, n_att = aux["contributions"][(i, v, s)]
296         rows.append({
297             "match_id": str(match.match_id), "group_id": str(match.group_id),
298             "round_in_group": int(match.round_in_group), "team_a": str(match.team_a), "team_b": str(match.team_b),
299             "team_a_id": str(match.team_a_id), "team_b_id": str(match.team_b_id),
300             "venue_id": str(venues.loc[v, "venue_id"]), "venue_index": int(v),
301             "slot_id": str(slots.loc[s, "slot_id"]), "slot_index": int(s),
302             "city": str(venues.loc[v, "city"]), "country": str(venues.loc[v, "country"]),
303             "reference_date": str(slots.loc[s, "date"]),
304             "reference_kickoff_time": str(slots.loc[s, "reference_kickoff_time"]),
305             "local_datetime": local.isoformat(), "utc_datetime": utc.isoformat(),
306             "required_security_level": int(match.required_security_level),
307             "venue_security_level": int(venues.loc[v, "security_level"]),
308             "venue_capacity": int(venues.loc[v, "capacity"]),
309             "expected_attendance": float(n_att), "expected_tv_viewers": int(match.predicted_tv_viewers),
310             "ticket_revenue_usd": float(tval), "broadcast_value_usd": float(bval),
311             "travel_cost_index": float(dval), "risk_index": float(rval),
312             "is_prime": int(s in aux["prime_slots"]), "is_large_venue": int(v in aux["large_venues"]),
313         })
314     return rows
315
316
317 def evaluate_rows(rows, groups, venues, bounds):
318     df = pd.DataFrame(rows)
319     venue_used = set(df["venue_id"])
320     setup = venues.loc[venues["venue_id"].isin(venue_used), "setup_cost_musd"].sum() * 1_000_000
321     vmap = venues.set_index("venue_id")
322     op = 0.0
323     for r in rows:
324         v = vmap.loc[r["venue_id"]]
325         op += float(v.operation_cost_musd_per_match) * 1_000_000 + SECURITY_BASE_COST_USD * r["required_security_level"] * float(v.security_cost_index)
326     T0, B0, C0 = float(df.ticket_revenue_usd.sum()), float(df.broadcast_value_usd.sum()), float(setup + op)
327     T = float(minmax(T0, bounds["T_min"], bounds["T_max"]))
328     B = float(minmax(B0, bounds["B_min"], bounds["B_max"]))
329     C = float(minmax(C0, bounds["C_min"], bounds["C_max"]))
330     U = float(groups["uncertainty_index"].mean())
331     H = float(groups["attractiveness_index"].mean() / 100.0)
332     D, R = float(df.travel_cost_index.mean()), float(df.risk_index.mean())
333     teams = sorted(set(df.team_a_id) | set(df.team_b_id))
334     prime_counts, large_counts = [], []
335     for t in teams:
336         z = df[(df.team_a_id == t) | (df.team_b_id == t)]
337         prime_counts.append(int(z.is_prime.sum())); large_counts.append(int(z.is_large_venue.sum()))
338     F = 0.5 * (max(prime_counts) - min(prime_counts)) / 3.0 + 0.5 * (max(large_counts) - min(large_counts)) / 3.0
339     Z2 = 0.25*T + 0.25*B + 0.15*U + 0.10*H - 0.08*C - 0.07*D - 0.06*F - 0.04*R
340     return {"T_raw": T0, "B_raw": B0, "C_raw": C0, "T": T, "B": B, "U": U, "H": H,
341             "C": C, "D": D, "F": float(F), "R": R, "Z2": float(Z2),
342             "prime_count_range": max(prime_counts)-min(prime_counts),
343             "large_count_range": max(large_counts)-min(large_counts)}
344
345
346 def validate_schedule(rows, groups, venues, slots, limits):
347     df = pd.DataFrame(rows)
348     violations = []
349     if len(df) != 72 or not df.match_id.is_unique: violations.append("每场唯一")
350     if df.duplicated(["venue_id", "utc_datetime"]).any(): violations.append("场馆UTC冲突")
351     for team in sorted(set(df.team_a_id) | set(df.team_b_id)):
352         z = df[(df.team_a_id == team) | (df.team_b_id == team)].sort_values("round_in_group")
353         hours = pd.to_datetime(z.utc_datetime, utc=True).diff().dropna().dt.total_seconds() / 3600
354         if len(z) != 3 or (hours < MINIMUM_REST_HOURS - 1e-9).any(): violations.append(f"休息:{team}")
355     vmap = venues.set_index("venue_id")
356     for vid, z in df.groupby("venue_id"):
357         if not (int(vmap.loc[vid, "min_total_matches"]) <= len(z) <= int(vmap.loc[vid, "max_total_matches"])):
358             violations.append(f"场馆总量:{vid}")
359     local_days = z.local_datetime.astype(str).str.slice(0, 10)

```

```

360     if local_days.value_counts().max() > int(vmap.loc[vid, "max_matches_per_day"]): violations.append(f"当地日:{vid}")
361     if (df.required_security_level > df.venue_security_level).any(): violations.append("安保资格")
362     slot_caps = slots.set_index("slot_id")["broadcast_capacity"].to_dict()
363     for sid, z in df.groupby("slot_id"):
364         if len(z) > int(slot_caps[sid]): violations.append(f"转播容量:{sid}")
365     lim = limits.set_index("reference_date")["high_security_capacity"].to_dict()
366     high = df[(df.round_in_group == 3) & (df.required_security_level >= HIGH_SECURITY_THRESHOLD)]
367     for day, z in high.groupby("reference_date"):
368         if len(z) > int(lim[str(day)]): violations.append(f"三轮高安保:{day}")
369     prime = []
370     for team in sorted(set(df.team_a_id) | set(df.team_b_id)):
371         z = df[(df.team_a_id == team) | (df.team_b_id == team)]
372         prime.append(int(z.is_prime.sum()))
373     if max(prime) - min(prime) > MAX_PRIME_COUNT_RANGE: violations.append("黄金公平")
374     assert not violations, "P2约束失败: " + ", ".join(violations[:5])
375     rest_values = []
376     for team in sorted(set(df.team_a_id) | set(df.team_b_id)):
377         z = df[(df.team_a_id == team) | (df.team_b_id == team)].sort_values("round_in_group")
378         rest_values.extend((pd.to_datetime(z.utc_datetime, utc=True).diff().dropna().dt.total_seconds()/3600).tolist())
379     return {"pass": True, "n_constraints": 9, "n_violations": 0,
380           "min_rest_hours": float(min(rest_values))}
381
382
383 def solve() -> dict:
384     set_all_seeds(MASTER_SEED)
385     start = time.time()
386     sheets, groups, venues, slots, limits, travel = prepare_data()
387     model, exprs, aux = build_model(groups, venues, slots, limits, travel, sheets["ticket_broadcast"])
388     print(f"[P2] x_ivs={len(aux['x_ivs'])} 开始寻找解析可行基线", flush=True)
389     # 仅为 full-domain 求解器构造 incumbent hint: 最终模型仍包含全部 80 个时段。
390     hint_windows = {1: set(range(0, 20)), 2: set(range(28, 48)), 3: set(range(60, 80))}
391     hint_model, _, hint_aux = build_model(groups, venues, slots, limits, travel, sheets["ticket_broadcast"], hint_windows)
392     hint_model.Maximize(0)
393     baseline_solver = _solver(180.0)
394     status = baseline_solver.Solve(hint_model)
395     print(f"[P2] hint status={baseline_solver.StatusName(status)}", flush=True)
396     assert status in (cp_model.OPTIMAL, cp_model.FEASIBLE), "P2无可行基线"
397     feasible_baseline = extract_schedule(baseline_solver, groups, venues, slots, hint_aux)
398     validate_schedule(feasible_baseline, groups, venues, slots, limits)
399     chosen_hint = {(r["match_id"], r["venue_id"], r["slot_id"]) for r in feasible_baseline}
400     for (i, v, s), var in aux["x_ivs"].items():
401         key = (str(groups.loc[i, "match_id"]), str(venues.loc[v, "venue_id"]), str(slots.loc[s, "slot_id"]))
402         if key in chosen_hint:
403             model.AddHint(var, 1)
404
405     bounds, normalization_meta = candidate_envelope_bounds(aux, groups, venues)
406     print(f"[P2] deterministic candidate envelope {bounds}", flush=True)
407
408     # 将浮点归一化目标一次性缩放为整数; U/H 为固定常数, 求解后再加回。
409     coeffs = []
410     for key, var in aux["x_ivs"].items():
411         t, b, c, d, r, _ = aux["contributions"][key]
412         coef = (0.25 * t / (bounds["T_max"] - bounds["T_min"])
413               + 0.25 * b / (bounds["B_max"] - bounds["B_min"])
414               - 0.08 * c / (bounds["C_max"] - bounds["C_min"])
415               - 0.07 * d / 72.0 - 0.04 * r / 72.0)
416         coeffs.append(round(coef * CP_SAT_SCALE) * var)
417     for v, var in aux["y_v"].items():
418         setup = float(venues.loc[v, "setup_cost_musd"]) * 1_000_000
419         coef = -0.08 * setup / (bounds["C_max"] - bounds["C_min"])
420         coeffs.append(round(coef * CP_SAT_SCALE) * var)
421     coeffs.append(round(-0.06 * 0.5 / 3.0 * CP_SAT_SCALE) * (aux["pmax"] - aux["pmin"] + aux["lmax"] - aux["lmin"]))
422     model.Maximize(cp_model.LinearExpr.Sum(coeffs))
423     solver = _solver(P2_TIME_LIMIT_SECONDS, gap=5e-3)
424     print(f"[P2] main CP-SAT solve", flush=True)
425     status = solver.Solve(model)
426     assert status in (cp_model.OPTIMAL, cp_model.FEASIBLE), f"P2主求解失败 {solver.StatusName(status)}"
427     rows = extract_schedule(solver, groups, venues, slots, aux)
428     audit = validate_schedule(rows, groups, venues, slots, limits)
429     metrics = evaluate_rows(rows, groups, venues, bounds)
430     baseline_metrics = evaluate_rows(feasible_baseline, groups, venues, bounds)
431     assert all(-1e-8 <= metrics[k] <= 1+1e-8 for k in ["T", "B", "U", "H", "C", "D", "F", "R"])
432     assert -0.25-1e-8 <= metrics["Z2"] <= 0.75+1e-8
433
434     out = pd.DataFrame(rows)
435     out["fairness_penalty"] = metrics["F"]

```

```

436 out["total_objective_value"] = np.nan
437 out.loc[0, "total_objective_value"] = metrics["Z2"]
438 template_cols = pd.read_csv(Path(__file__).resolve().parent.parent / "user_data" / "result_2_template.csv").columns
439 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
440 out[list(template_cols)].to_csv(OUTPUT_DIR / "result_2_group_schedule.csv", index=False, encoding="utf-8-sig")
441 result = {
442     "metadata": {**runtime_metadata(MASTER_SEED), "elapsed_seconds": time.time()-start},
443     "method": "three-dimensional match-venue-slot CP-SAT",
444     "solver": {"status": solver.StatusName(status), "incumbent_scaled": solver.ObjectiveValue(),
445               "best_bound_scaled": solver.BestObjectiveBound(), "wall_time": solver.WallTime(),
446               "workers": 1, "relative_gap_limit": 0.005},
447     "normalization_bounds": bounds, "normalization_meta": normalization_meta,
448     "metrics": metrics, "schedule": rows, "constraint_audit": audit,
449     "feasible_baseline": {"schedule": feasible_baseline, "metrics": baseline_metrics,
450                           "constraint_audit": {"pass": True, "n_violations": 0}},
451     "venue_load": out.groupby("venue_id").size().to_dict(),
452     "seed": MASTER_SEED,
453     "outputs": ["output/result_2_group_schedule.csv"],
454 }
455 write_json(FIGURES_DIR / "problem_2_results.json", result)
456 print(f"[P2] validate_capability PASS Z2={metrics['Z2']:.6f} status={solver.StatusName(status)}", flush=True)
457 return result
458
459
460 if __name__ == "__main__":
461     solve()

```

Listing 8: 问题三完整程序

```

1  """问题三：共同截面晋级模拟与四资源日级多选择 CP-SAT."""
2  import os, sys
3  _HERE = os.path.dirname(os.path.abspath(__file__))
4  if _HERE not in sys.path:
5      sys.path.insert(0, _HERE)
6
7  import time
8  from collections import defaultdict
9  from pathlib import Path
10
11  import numpy as np
12  import pandas as pd
13  from ortools.sat.python import cp_model
14
15  from params import (
16      ATTENDANCE_FLOOR_RATIO, CP_SAT_SCALE, DELTA_INJURY_COEF,
17      DELTA_STATE_COEF, DIRECT_INJURY_COEF, GOAL_MEAN_CLIP, MASTER_SEED,
18      MC_DRAWS, P3_TIME_LIMIT_SECONDS, RAW_FEEDBACK_CLIP, RED_CARD_DECAY,
19      STATE_WEIGHTS, STATE_XG_SCALE, TEAM_FEEDBACK_CLIP,
20  )
21  from utils import FIGURES_DIR, OUTPUT_DIR, load_all_sheets, minmax, runtime_metadata, set_all_seeds, write_json
22
23
24  def team_snapshot(live: pd.DataFrame, teams: list[str]) -> dict[str, dict]:
25      stats = {t: {"games": 0, "points": 0.0, "gf": 0.0, "ga": 0.0,
26                  "xgf": 0.0, "xga": 0.0, "red": 0.0, "injuries": []} for t in teams}
27      for r in live.itertuples(index=False):
28          ga, gb = int(r.goals_a), int(r.goals_b)
29          pa, pb = (3, 0) if ga > gb else ((0, 3) if ga < gb else (1, 1))
30          for t, gf, gc, xgf, xga, red, pts in [
31              (r.team_a, ga, gb, r.xg_a, r.xg_b, r.red_cards_a, pa),
32              (r.team_b, gb, ga, r.xg_b, r.xg_a, r.red_cards_b, pb),
33          ]:
34              s = stats[str(t)]
35              s["games"] += 1; s["points"] += pts; s["gf"] += gf; s["ga"] += gc
36              s["xgf"] += float(xgf); s["xga"] += float(xga); s["red"] += float(red)
37              s["injuries"].append(float(r.injury_impact_level))
38      for t, s in stats.items():
39          assert s["games"] == 2, f"{t} 前两轮场数不等于2"
40          g = s["games"]
41          h = float(np.clip(np.mean(s["injuries"]), 0, 3))
42          state = (STATE_WEIGHTS["points"] * s["points"] / (3 * g)
43                  + STATE_WEIGHTS["xg_form"] * (0.5 + 0.5 * np.tanh(((s["xgf"]-s["xga"])/g) / STATE_XG_SCALE))
44                  + STATE_WEIGHTS["discipline"] * np.exp(-RED_CARD_DECAY * s["red"] / g))
45          s["injury"] = h; s["state"] = float(np.clip(state, 0, 1)); s["gd"] = s["gf"] - s["ga"]

```

```

46     return stats
47
48
49 def updated_lambdas(round3: pd.DataFrame, snapshot: dict[str, dict]):
50     la, lb = [], []
51     for r in round3.itertuples(index=False):
52         a, b = snapshot[str(r.team_a)], snapshot[str(r.team_b)]
53         delta = DELTA_STATE_COEF * (a["state"] - b["state"]) + DELTA_INJURY_COEF * (b["injury"] - a["injury"])
54         la.append(float(np.clip(r.expected_goals_a * np.exp(delta - DIRECT_INJURY_COEF*a["injury"]), *GOAL_MEAN_CLIP)))
55         lb.append(float(np.clip(r.expected_goals_b * np.exp(-delta - DIRECT_INJURY_COEF*b["injury"]), *GOAL_MEAN_CLIP)))
56     return np.asarray(la), np.asarray(lb)
57
58
59 def _blocks(order, metrics):
60     start = 0
61     while start < len(order):
62         end = start + 1
63         while end < len(order) and metrics[order[end]] == metrics[order[start]]:
64             end += 1
65         yield start, end, order[start:end]
66         start = end
67
68
69 def simulate_qualification(round3: pd.DataFrame, membership: pd.DataFrame,
70                             snapshot: dict[str, dict], lam_a: np.ndarray, lam_b: np.ndarray,
71                             seed: int, keep_scores: bool = True):
72     """一个固定 snapshot 上一次性生成 MC_DRAWS*24, 并对完全同分块按比例分配。"""
73     rng = np.random.Generator(np.random.PCG64(seed))
74     score_a = rng.poisson(lam=np.broadcast_to(lam_a, (MC_DRAWS, len(round3))))
75     score_b = rng.poisson(lam=np.broadcast_to(lam_b, (MC_DRAWS, len(round3))))
76     teams = membership.sort_values(["group_id", "team_id"])["team_name"].astype(str).tolist()
77     team_idx = {t: i for i, t in enumerate(teams)}
78     groups = {str(g): [team_idx[str(t)] for t in z.team_name] for g, z in membership.groupby("group_id", sort=True)}
79     match_idx = [(team_idx[str(r.team_a)], team_idx[str(r.team_b)]) for r in round3.itertuples(index=False)]
80     base_pts = np.array([snapshot[t]["points"] for t in teams], float)
81     base_gd = np.array([snapshot[t]["gd"] for t in teams], float)
82     base_gf = np.array([snapshot[t]["gf"] for t in teams], float)
83     adv = np.zeros((MC_DRAWS, len(teams)), dtype=np.float64)
84     mass = np.zeros((MC_DRAWS, len(teams)), dtype=np.float64)
85     for m in range(MC_DRAWS):
86         pts, gd, gf = base_pts.copy(), base_gd.copy(), base_gf.copy()
87         for k, (a, b) in enumerate(match_idx):
88             xa, xb = int(score_a[m, k]), int(score_b[m, k])
89             gf[a] += xa; gf[b] += xb; gd[a] += xa-xb; gd[b] += xb-xa
90             if xa > xb: pts[a] += 3
91             elif xa < xb: pts[b] += 3
92             else: pts[a] += 1; pts[b] += 1
93         candidate_weight = np.zeros((len(teams), float))
94         metrics = [(float(pts[i]), float(gd[i]), float(gf[i])) for i in range(len(teams))]
95         for members in groups.values():
96             order = sorted(members, key=lambda i: metrics[i], reverse=True)
97             for start, end, block in _blocks(order, metrics):
98                 direct_slots = max(0, min(end, 2) - start)
99                 if direct_slots:
100                     adv[m, block] += direct_slots / len(block)
101                 if start <= 2 < end:
102                     candidate_weight[block] = 1.0 / len(block)
103                 candidates = [i for i, w in enumerate(candidate_weight) if w > 0]
104                 order = sorted(candidates, key=lambda i: metrics[i], reverse=True)
105                 remaining = 8.0
106                 for _, block in _blocks(order, metrics):
107                     block_mass = float(candidate_weight[block].sum())
108                     take = min(remaining, block_mass)
109                     if take > 0:
110                         adv[m, block] += candidate_weight[block] * (take / block_mass)
111                         remaining -= take
112                 if remaining <= 1e-12:
113                     break
114             mass[m] = float(adv[m].sum())
115     assert np.max(np.abs(mass - 32.0)) <= 1e-6, "晋级名额质量不守恒"
116     result = {"teams": teams, "probability": adv.mean(axis=0), "mass": mass}
117     if keep_scores:
118         result.update({"score_a": score_a, "score_b": score_b, "adv": adv})
119     return result
120
121

```

```

122 def conditional_probabilities(sim, round3: pd.DataFrame):
123     idx = {t: i for i, t in enumerate(sim["teams"])}
124     rows = []
125     for k, r in enumerate(round3.itertuples(index=False)):
126         draw = sim["score_a"][:, k] == sim["score_b"][:, k]
127         awin = sim["score_a"][:, k] > sim["score_b"][:, k]
128         bwin = sim["score_b"][:, k] > sim["score_a"][:, k]
129         ia, ib = idx[str(r.team_a)], idx[str(r.team_b)]
130         def avg(mask, j): return float(sim["adv"][mask, j].mean()) if int(mask.sum()) else np.nan
131         rows.append({
132             "match_id": str(r.match_id), "p_a_draw": avg(draw, ia), "p_b_draw": avg(draw, ib),
133             "p_a_cond_win": avg(awin, ia), "p_b_cond_win": avg(bwin, ib),
134             "n_draw": int(draw.sum()), "n_a_win": int(awin.sum()), "n_b_win": int(bwin.sum()),
135         })
136     return pd.DataFrame(rows)
137
138
139 def feedback_by_team(live, base, p1_group):
140     z = live.merge(base[["match_id", "expected_attendance_base"]], on="match_id", validate="one_to_one")
141     z = z.merge(p1_group[["match_id", "predicted_tv_viewers"]], on="match_id", validate="one_to_one")
142     values = defaultdict(lambda: {"N": [], "V": []})
143     for r in z.itertuples(index=False):
144         rn = float(np.clip(r.attendance / r.expected_attendance_base, *RAW_FEEDBACK_CLIP))
145         rv = float(np.clip(r.tv_viewers / r.predicted_tv_viewers, *RAW_FEEDBACK_CLIP))
146         for t in (str(r.team_a), str(r.team_b)):
147             values[t]["N"].append(rn); values[t]["V"].append(rv)
148     return {t: {k: float(np.clip(np.mean(v), *TEAM_FEEDBACK_CLIP)) for k, v in d.items()} for t, d in values.items()}
149
150
151 def price_common_upper(epsilon: float, day_increase: float) -> float:
152     """共同价格域解析上界: 与静态/动态状态无关."""
153     return float(min(day_increase, 0.88 ** (1 / epsilon) - 1))
154
155
156 def resource_maps(costs: pd.DataFrame):
157     return {(str(r.resource_type), int(r.resource_level)): {
158         "cost": float(r.unit_cost_index), "demand": float(r.demand_multiplier), "risk": float(r.risk_multiplier)
159     } for r in costs.itertuples(index=False)}
160
161
162 def derive_match_environment(round3, schedule, venues, ticket, limits, snapshot, feedback,
163                             team_prob, cond):
164     sched = schedule[schedule["round_in_group"].eq(3)].copy()
165     z = round3.merge(sched[["match_id", "venue_id", "slot_id", "reference_date"]], on="match_id", validate="one_to_one")
166     z = z.merge(venues[["venue_id", "capacity", "security_level"]], on="venue_id", validate="many_to_one")
167     z = z.merge(cond, on="match_id", validate="one_to_one")
168     throw = ticket.loc[ticket["match_stage"].eq("Group_Match_R3")].iloc[0]
169     pmap = team_prob
170     rows = []
171     for r in z.itertuples(index=False):
172         a, b = snapshot[str(r.team_a)], snapshot[str(r.team_b)]
173         pa, pb = pmap[str(r.team_a)], pmap[str(r.team_b)]
174         Q = (4*pa*(1-pa) + 4*pb*(1-pb)) / 2.0
175         fN = float(np.sqrt(feedback[str(r.team_a)]["N"] * feedback[str(r.team_b)]["N"]))
176         fV = float(np.sqrt(feedback[str(r.team_a)]["V"] * feedback[str(r.team_b)]["V"]))
177         S = (a["state"] + b["state"]) / 2.0; H = (a["injury"] + b["injury"]) / 6.0
178         A0 = float(r.attractiveness_index) / 100.0
179         F = float(np.clip((0.5*fN + 0.5*fV - 0.75) / 0.50, 0, 1))
180         A = float(np.clip(0.50*A0 + 0.25*Q + 0.12*F + 0.08*S + 0.05*(1-H), 0, 1))
181         Ntilde = float(r.expected_attendance_base * fN * (0.88+0.27*Q) * (0.94+0.12*S) * (1-0.10*H))
182         Vtilde = float(r.predicted_tv_viewers * fV * (0.87+0.30*Q) * (0.90+0.20*A) * (1-0.06*H))
183         Rstake = float(0.5*(2*pa-1)**2 + 0.5*(2*pb-1)**2)
184         D = min(float(r.p_a_draw), float(r.p_b_draw))
185         G = 0.5*max(float(r.p_a_cond_win)-float(r.p_a_draw), 0) + 0.5*max(float(r.p_b_cond_win)-float(r.p_b_draw), 0)
186         Rcol = float(np.clip(D * (1-np.clip(G, 0, 1)) * (1-0.35*Rstake), 0, 1))
187         occupancy = float(np.clip(Ntilde / float(r.capacity), 0, 1))
188         dsec = float(np.clip(0.45*r.security_demand_score + 0.25*occupancy + 0.15*A + 0.15*(Rstake+Rcol)/2, 0, 1))
189         rows.append({
190             **r._asdict(), "p_a": pa, "p_b": pb, "Q": Q, "fN": fN, "fV": fV,
191             "S": S, "H": H, "A0": A0, "A": A, "Ntilde": Ntilde, "Vtilde": Vtilde,
192             "Rstake": Rstake, "Rcol": Rcol, "dsec": dsec,
193             "base_ticket_price": float(throw.base_ticket_price_usd), "epsilon": float(throw.price_elasticity),
194             "broadcast_unit_value": float(throw.broadcast_unit_value_usd),
195         })
196     return pd.DataFrame(rows)
197

```

```

198
199 def make_action_table(env: pd.DataFrame, costs: pd.DataFrame, limits: pd.DataFrame, dynamic: bool):
200     maps = resource_maps(costs)
201     lim = limits.copy(); lim["reference_date"] = lim["reference_date"].astype(str)
202     daymap = lim.set_index("reference_date").to_dict(orient="index")
203     action_table = []
204     for i, r in env.iterrows():
205         if dynamic:
206             Nbase, Vbase, attractiveness = float(r.Ntilde), float(r.Vtilde), float(r.A)
207             stake, col, dsec = float(r.Rstake), float(r.Rcol), float(r.dsec)
208         else:
209             Nbase, Vbase, attractiveness = float(r.expected_attendance_base), float(r.predicted_tv_viewers), float(r.A0)
210             stake, col, dsec = 1 - float(r.uncertainty_index), 0.0, float(r.security_demand_score)
211     day_limit = daymap[str(r.reference_date)]
212     delta = price_common_upper(float(r.epsilon), float(day_limit["max_ticket_increase_rate"]))
213     for b in (1, 2, 3):
214         for q in range(int(r.required_security_level), int(r.security_level)+1):
215             for l in (1, 2, 3):
216                 bm, qm, lm = maps[("broadcast", b)], maps[("security", q)], maps[("transport", l)]
217                 at_zero = min(float(r.capacity), Nbase * lm["demand"])
218                 attendance = min(float(r.capacity), Nbase * lm["demand"] * (1+delta) ** float(r.epsilon))
219                 assert attendance + 1e-8 >= ATTENDANCE_FLOOR_RATIO * at_zero
220                 tv = Vbase * bm["demand"]
221                 TV = float(r.base_ticket_price) * (1+delta) * attendance
222                 BV = float(r.broadcast_unit_value) * tv
223                 risk = 0.40*stake + 0.40*col + 0.20*dsec*qm["risk"]
224                 breakdown = {"broadcast": bm["cost"], "security": qm["cost"], "transport": lm["cost"]}
225                 action_table.append({
226                     "match_index": i, "match_id": str(r.match_id), "reference_date": str(r.reference_date),
227                     "b": b, "q": q, "l": l, "delta": delta, "attendance": attendance,
228                     "attendance_at_zero": at_zero, "tv": tv, "TV": TV, "BV": BV,
229                     "A": attractiveness, "C": sum(breakdown.values()), "R": float(np.clip(risk, 0, 1)),
230                     "cost_breakdown": breakdown,
231                 })
232     return action_table
233
234
235 def action_bounds(actions):
236     return {k: (min(a[k] for a in actions), max(a[k] for a in actions)) for k in ("TV", "BV", "A", "C", "R")}
237
238
239 def action_value(a, bounds):
240     return (0.35*minmax(a["TV"], *bounds["TV"]) + 0.35*minmax(a["BV"], *bounds["BV"])
241             + 0.10*minmax(a["A"], *bounds["A"]) - 0.10*minmax(a["C"], *bounds["C"])
242             - 0.10*minmax(a["R"], *bounds["R"]))
243
244
245 def optimize_actions(action_table, limits, bounds, label):
246     model = cp_model.CpModel()
247     z_ia = [model.NewBoolVar(f"z_ia_{label}_{j}") for j in range(len(action_table))]
248     by_match, by_day = defaultdict(list), defaultdict(list)
249     for j, a in enumerate(action_table):
250         by_match[a["match_index"]].append(j); by_day[a["reference_date"]].append(j)
251     for idx in sorted(by_match): model.Add(sum(z_ia[j] for j in by_match[idx]) == 1)
252     lim = limits.copy(); lim["reference_date"] = lim["reference_date"].astype(str)
253     daily_resource_budget = lim.set_index("reference_date").to_dict(orient="index")
254     for day, js in by_day.items():
255         caps = daily_resource_budget[day]
256         model.Add(sum(z_ia[j] for j in js if action_table[j]["b"] == 3) <= int(caps["high_broadcast_capacity"]))
257         model.Add(sum(z_ia[j] for j in js if action_table[j]["q"] >= 3) <= int(caps["high_security_capacity"]))
258         model.Add(sum(z_ia[j] for j in js if action_table[j]["l"] == 3) <= int(caps["enhanced_transport_capacity"]))
259         model.Add(sum(round(action_table[j]["C"] * 1000) * z_ia[j] for j in js) <= round(float(caps["daily_resource_budget_index"])*1000))
260     model.Maximize(sum(round(action_value(a, bounds)*CP_SAT_SCALE)*z_ia[j] for j, a in enumerate(action_table)))
261     solver = cp_model.CpSolver(); solver.parameters.max_time_in_seconds = min(P3_TIME_LIMIT_SECONDS, 300)
262     solver.parameters.relative_gap_limit = 1e-3; solver.parameters.num_search_workers = 1; solver.parameters.random_seed = MASTER_SEED
263     status = solver.Solve(model)
264     assert status in (cp_model.OPTIMAL, cp_model.FEASIBLE)
265     selected = [action_table[j] for j, var in enumerate(z_ia) if solver.Value(var)]
266     assert len(selected) == 24
267     return selected, {"status": solver.StatusName(status), "incumbent": solver.ObjectiveValue()/CP_SAT_SCALE,
268                     "best_bound": solver.BestObjectiveBound()/CP_SAT_SCALE, "wall_time": solver.WallTime()}
269
270
271 def daily_audit(selected, limits):
272     lim = limits.copy(); lim["reference_date"] = lim["reference_date"].astype(str)
273     lim = lim.set_index("reference_date").to_dict(orient="index")

```

```

274 rows = []
275 for day in sorted([a["reference_date"] for a in selected]):
276     z = [a for a in selected if a["reference_date"] == day]
277     use = {"broadcast3": sum(a["b"] == 3 for a in z), "security3plus": sum(a["q"] >= 3 for a in z),
278           "transport3": sum(a["l"] == 3 for a in z), "budget": sum(a["C"] for a in z)}
279     cap = lim[day]
280     ok = (use["broadcast3"] <= cap["high_broadcast_capacity"] and use["security3plus"] <= cap["high_security_capacity"]
281           and use["transport3"] <= cap["enhanced_transport_capacity"] and use["budget"] <= cap["daily_resource_budget_index"]+1e-9)
282     rows.append({"date": day, **use, "caps": cap, "pass": bool(ok)})
283 assert all(r["pass"] for r in rows)
284 return rows
285
286
287 def solve() -> dict:
288     set_all_seeds(MASTER_SEED); start = time.time(); sheets = load_all_sheets()
289     groups = sheets["groups_matches"]
290     base = groups.merge(sheets["base_predictions"], on=["match_id", "group_id", "round_in_group", "team_a", "team_b"], validate="one_to_one")
291     base = base.merge(sheets["security_requirements"][["match_id", "security_demand_score", "required_security_level"]], on="match_id", validate="one_to_one")
292
293     p1 = pd.read_csv(OUTPUT_DIR / "result_1_match_prediction.csv")
294     base = base.merge(p1[["match_id", "predicted_tv_viewers"]], on="match_id", validate="one_to_one")
295     round3 = base[base["round_in_group"].eq(3)].reset_index(drop=True)
296     teams = sheets["group_membership"][["team_name"]].astype(str).tolist()
297     snapshot = team_snapshot(sheets["live_group_results"], teams)
298     lam_a, lam_b = updated_lambdas(round3, snapshot)
299     print(f"[P3] joint Monte Carlo shape={MC_DRAWS},{len(round3)}", flush=True)
300     sim = simulate_qualification(round3, sheets["group_membership"], snapshot, lam_a, lam_b, MASTER_SEED)
301     team_prob = {t: float(sim["probability"][i]) for i, t in enumerate(sim["teams"])}
302     assert abs(sum(team_prob.values()) - 32.0) <= 1e-5 and all(0 <= p <= 1 for p in team_prob.values())
303     cond = conditional_probabilities(sim, round3)
304     feedback = feedback_by_team(sheets["live_group_results"], sheets["base_predictions"], p1)
305     schedule = pd.read_csv(OUTPUT_DIR / "result_2_group_schedule.csv")
306     env = derive_match_environment(round3, schedule, sheets["venues"], sheets["ticket_broadcast"],
307                                   sheets["dynamic_resource_limits"], snapshot, feedback, team_prob, cond)
308     static_actions = make_action_table(env, sheets["dynamic_resource_costs"], sheets["dynamic_resource_limits"], False)
309     dynamic_actions = make_action_table(env, sheets["dynamic_resource_costs"], sheets["dynamic_resource_limits"], True)
310     static_bounds = action_bounds(static_actions)
311     static_selected, static_solver = optimize_actions(static_actions, sheets["dynamic_resource_limits"], static_bounds, "static")
312     static_action_snapshot = {(a["match_id"]): (a["b"], a["q"], a["l"], a["delta"]) for a in static_selected}
313     shared_eval_bounds = action_bounds(dynamic_actions)
314     dynamic_selected, dynamic_solver = optimize_actions(dynamic_actions, sheets["dynamic_resource_limits"], shared_eval_bounds, "dynamic")
315     dyn_lookup = {(a["match_id"], a["b"], a["q"], a["l"]): a for a in dynamic_actions}
316     static_reeval = [dyn_lookup[(mid, *decision[:3])] for mid, decision in static_action_snapshot.items()]
317     assert static_action_snapshot == {(a["match_id"]): (a["b"], a["q"], a["l"], a["delta"]) for a in static_selected}
318     z_static = sum(action_value(a, shared_eval_bounds) for a in static_reeval)
319     z_dynamic = sum(action_value(a, shared_eval_bounds) for a in dynamic_selected)
320     assert z_dynamic + 1e-4 >= z_static
321     daily_static = daily_audit(static_reeval, sheets["dynamic_resource_limits"])
322     daily_dynamic = daily_audit(dynamic_selected, sheets["dynamic_resource_limits"])
323
324     envmap = env.set_index("match_id").to_dict(orient="index")
325     smap = {a["match_id"]: a for a in static_reeval}; dmap = {a["match_id"]: a for a in dynamic_selected}
326     rows = []
327     for mid in env["match_id"]:
328         e, s, d = envmap[mid], smap[mid], dmap[mid]
329         sv, dv = action_value(s, shared_eval_bounds), action_value(d, shared_eval_bounds)
330         rows.append({
331             "match_id": mid, "group_id": e["group_id"], "team_a": e["team_a"], "team_b": e["team_b"],
332             "venue_id": e["venue_id"], "slot_id": e["slot_id"], "reference_date": e["reference_date"],
333             "updated_p_team_a_advance": e["p_a"], "updated_p_team_b_advance": e["p_b"],
334             "updated_expected_attendance": d["attendance"], "updated_expected_tv_viewers": d["tv"],
335             "stakeless_risk": e["Rstake"], "collusion_risk": e["Rcol"], "updated_attractiveness": e["A"],
336             "recommended_broadcast_priority": d["b"], "recommended_security_level": d["q"],
337             "recommended_transport_level": d["l"], "recommended_ticket_adjustment": d["delta"],
338             "updated_ticket_revenue_usd": d["TV"], "updated_broadcast_value_usd": d["BV"],
339             "resource_cost_index": d["C"], "risk_exposure_index": d["R"],
340             "static_net_value": sv, "dynamic_net_value": dv,
341             "improvement_rate": None if abs(sv) <= 1e-15 else (dv-sv)/abs(sv),
342             "static_decision": {"b": s["b"], "q": s["q"], "l": s["l"], "delta": s["delta"]},
343             "dynamic_decision": {"b": d["b"], "q": d["q"], "l": d["l"], "delta": d["delta"]},
344             "attendance_at_zero": d["attendance_at_zero"], "capacity": e["capacity"],
345             "required_security_level": e["required_security_level"], "venue_security_level": e["security_level"],
346         })
347     out = pd.DataFrame(rows)
348     template_cols = pd.read_csv(Path(__file__).resolve().parent.parent / "user_data" / "result_3_template.csv").columns
349     out[list(template_cols)].to_csv(OUTPUT_DIR / "result_3_dynamic_strategy.csv", index=False, encoding="utf-8-sig")

```

```

349     assert len(out) == 24 and out.match_id.is_unique
350     assert all(r["venue_id"] == schedule.set_index("match_id").loc[r["match_id"], "venue_id"] for r in rows)
351     assert all(r["updated_expected_attendance"] + 1e-8 >= ATTENDANCE_FLOOR_RATIO*r["attendance_at_zero"] for r in rows)
352     assert all(r["required_security_level"] <= r["recommended_security_level"] <= r["venue_security_level"] for r in rows)
353
354     # 四个附加固定种子各跑完整 20000 联合样本，并把更新状态传入完整动态重优化。
355     stability = []
356     base_dynamic_snapshot = {a["match_id"]:(a["b"],a["q"],a["l"],a["delta"]) for a in dynamic_selected}
357     decision_stability = [{"seed":MASTER_SEED,"Z3_static":float(z_static),"Z3_dynamic":float(z_dynamic),
358                           "improvement":float(z_dynamic-z_static),"decision_changes_vs_base":0,
359                           "solver_status":dynamic_solver["status"]}]]
360     for seed in range(MASTER_SEED+1, MASTER_SEED+5):
361         print(f"[P3] stability seed={seed}", flush=True)
362         ss = simulate_qualification(round3, sheets["group_membership"], snapshot, lam_a, lam_b, seed, keep_scores=True)
363         stability.append(ss["probability"].astype(float))
364         seed_prob = {t:float(ss["probability"][i]) for i,t in enumerate(ss["teams"])}
365         seed_cond = conditional_probabilities(ss, round3)
366         seed_env = derive_match_environment(round3, schedule, sheets["venues"], sheets["ticket_broadcast"],
367                                           sheets["dynamic_resource_limits"], snapshot, feedback,
368                                           seed_prob, seed_cond)
369         seed_actions = make_action_table(seed_env, sheets["dynamic_resource_costs"], sheets["dynamic_resource_limits"], True)
370         seed_bounds = action_bounds(seed_actions)
371         seed_selected, seed_solver = optimize_actions(seed_actions, sheets["dynamic_resource_limits"], seed_bounds, f"dynamic_{seed}")
372         seed_lookup = {(a["match_id"],a["b"],a["q"],a["l"]):a for a in seed_actions}
373         seed_static = [seed_lookup[(mid,*decision[:3])] for mid,decision in static_action_snapshot.items()]
374         seed_z_static = sum(action_value(a,seed_bounds) for a in seed_static)
375         seed_z_dynamic = sum(action_value(a,seed_bounds) for a in seed_selected)
376         assert seed_z_dynamic + 1e-4 >= seed_z_static
377         seed_snapshot = {a["match_id"]:(a["b"],a["q"],a["l"],a["delta"]) for a in seed_selected}
378         changes = sum(seed_snapshot[mid] != base_dynamic_snapshot[mid] for mid in base_dynamic_snapshot)
379         decision_stability.append({"seed":seed,"Z3_static":float(seed_z_static),"Z3_dynamic":float(seed_z_dynamic),
380                                   "improvement":float(seed_z_dynamic-seed_z_static),
381                                   "decision_changes_vs_base":int(changes),"solver_status":seed_solver["status"]})
382     prob_stack = np.vstack([sim["probability"].astype(float), *stability])
383     result = {
384         "metadata": {**runtime_metadata(MASTER_SEED), "elapsed_seconds": time.time()-start},
385         "method": "PCG64 joint Poisson Monte Carlo + action_table + daily multiple-choice CP-SAT",
386         "mc_shape": [MC_DRAWS, 24], "common_snapshot": True,
387         "qualifier_mass_by_draw": sim["mass"].astype(float).tolist(),
388         "team_advancement": [{t:"team":t,"probability":team_prob[t],"five_seed_std":float(prob_stack[:,i].std()) for i,t in enumerate(sim["teams"])]},
389         "five_seed_decision_stability":decision_stability,
390         "lambdas": [{"match_id":round3.loc[i,"match_id"],"lambda_a":float(lam_a[i]),"lambda_b":float(lam_b[i]) for i in range(24)],
391         "conditional_probabilities": cond.to_dict(orient="records"),
392         "action_table": dynamic_actions, "shared_eval_bounds": shared_eval_bounds,
393         "static_action_snapshot": static_action_snapshot,
394         "static_solver": static_solver, "dynamic_solver": dynamic_solver,
395         "Z3_static_reveal": float(z_static), "Z3_dynamic": float(z_dynamic), "improvement": float(z_dynamic-z_static),
396         "rows": rows, "daily_static_audit": daily_static, "daily_dynamic_audit": daily_dynamic,
397         "source_unit_value": {"broadcast_cost": "dynamic_resource_costs", "security_cost": "dynamic_resource_costs", "transport_cost": "dynamic_resource_costs"},
398         "seed": MASTER_SEED, "outputs": ["output/result_3_dynamic_strategy.csv"],
399     }
400     write_json(FIGURES_DIR / "problem_3_results.json", result)
401     print(f"[P3] validate_capability PASS Z3_dynamic={z_dynamic:.6f} static={z_static:.6f}", flush=True)
402     return result
403
404
405 if __name__ == "__main__":
406     solve()

```

Listing 9: 问题四完整程序

```

1  """问题四: 2022 卡塔尔世界杯真实赛程与 P2 优化赛程同口径评价."""
2  import os, sys
3  _HERE = os.path.dirname(os.path.abspath(__file__))
4  if _HERE not in sys.path:
5      sys.path.insert(0, _HERE)
6
7  import json
8  import math
9  import time
10 from pathlib import Path
11 from zoneinfo import ZoneInfo
12

```

```

13 import numpy as np
14 import pandas as pd
15 from sklearn.neighbors import NearestNeighbors
16 from sklearn.preprocessing import StandardScaler
17
18 from params import MASTER_SEED, SECURITY_BASE_COST_USD
19 from problem2 import compute_travel
20 from utils import FIGURES_DIR, OUTPUT_DIR, ROOT, load_all_sheets, minmax, runtime_metadata, set_all_seeds, write_json
21
22 FIFA_SCHEDULE_URL = "https://inside.fifa.com/tournaments/mens/worldcup/qatar2022/media-releases/final-match-schedule-for-the-fifa-world-cup-qatar-2022-tm-
now-available-on"
23 FIFA_FIXTURE_URL = "https://www.fifa.com/en/tournaments/mens/worldcup/qatar2022/scores-fixtures"
24 RAW_VALIDATION_URL = "https://raw.githubusercontent.com/openfootball/worldcup.json/master/2022/worldcup.json"
25 TEAM_ELO_SOURCE_URL = "https://www.international-football.net/elo-ratings-table?confed=&day=12&month=11&old-team=&year=2022"
26 FIFA_STADIUM_SOURCE_URL = "https://inside.fifa.com/tournament-organisation/world-cup-2022-in-numbers/fifa-world-cup-qatar-2022-at-a-glance"
27 FIFA_STADIUM_ADDRESS_URL = "https://www.fifa.com/en/articles/stadium-addresses-en"
28 ACCESSED_AT = "2026-08-14"
29 ACTUAL_CLIMATE_SCENARIO = 0.12
30
31 STADIUMS = {
32     "Al Bayt Stadium": ("Al Khor", 68895, 25.652, 51.488),
33     "Al Thumama Stadium": ("Doha", 44400, 25.235, 51.532),
34     "Khalifa International Stadium": ("Al Rayyan", 45857, 25.263, 51.448),
35     "Ahmad bin Ali Stadium": ("Al Rayyan", 45032, 25.330, 51.342),
36     "Ahmad Bin Ali Stadium": ("Al Rayyan", 45032, 25.330, 51.342),
37     "Al Janoub Stadium": ("Al Wakrah", 44325, 25.160, 51.574),
38     "Education City Stadium": ("Al Rayyan", 44667, 25.310, 51.424),
39     "Stadium 974": ("Doha", 44089, 25.289, 51.567),
40     "Lusail Stadium": ("Lusail", 88966, 25.420, 51.490),
41     "Lusail Iconic Stadium": ("Lusail", 88966, 25.420, 51.490),
42 }
43
44
45 def _clean_ground(value: str) -> str:
46     name = value.split(",")[0].strip()
47     return name
48
49
50 def collect_actual_schedule() -> pd.DataFrame:
51     """从已下载的开放结构化副本取数，并单列 FIFA 官方核验来源。"""
52     raw_path = ROOT / "user_data" / "worldcup_2022_openfootball.json"
53     raw = json.loads(raw_path.read_text(encoding="utf-8"))["matches"]
54     games = [x for x in raw if str(x.get("group", "")).startswith("Group ")]
55     assert len(games) == 48
56     df = pd.DataFrame(games)
57     df["group_id"] = df["group"].str.replace("Group ", "", regex=False)
58     df["local_dt"] = pd.to_datetime(df["date"] + " " + df["time"]).dt.tz_localize(ZoneInfo("Asia/Qatar"))
59     df = df.sort_values(["group_id", "local_dt", "team1", "team2"], kind="mergesort").reset_index(drop=True)
60     df["round_in_group"] = df.groupby("group_id").cumcount().floordiv(2) + 1
61     rows = []
62     for idx, r in df.iterrows():
63         venue = _clean_ground(str(r.ground)); assert venue in STADIUMS, venue
64         city, cap, lat, lon = STADIUMS[venue]
65         score_obj = r["score"]
66         score = score_obj.get("ft", [None, None]) if isinstance(score_obj, dict) else score_obj
67         rows.append({
68             "match_id": f"Q22-{idx+1:02d}", "competition": "FIFA World Cup Qatar 2022",
69             "stage": "Group", "group_id": str(r.group_id), "round_in_group": int(r.round_in_group),
70             "team_a": str(r.team1), "team_b": str(r.team2), "venue": venue,
71             "city": city, "country": "Qatar", "date": str(r.date),
72             "local_kickoff_time": str(r.time), "local_datetime": r.local_dt.isoformat(),
73             "utc_datetime": r.local_dt.tz_convert("UTC").isoformat(), "capacity": cap,
74             "latitude": lat, "longitude": lon, "goals_a": int(score[0]), "goals_b": int(score[1]),
75             "source_url": RAW_VALIDATION_URL, "fixture_source_url": FIFA_FIXTURE_URL,
76             "official_schedule_url": FIFA_SCHEDULE_URL, "stadium_source_url": FIFA_STADIUM_SOURCE_URL,
77             "stadium_address_url": FIFA_STADIUM_ADDRESS_URL, "retrieval_date": ACCESSED_AT,
78         })
79     out = pd.DataFrame(rows)
80     assert len(out) == 48 and out.match_id.is_unique and out.source_url.str.len().gt(0).all()
81     for team in sorted(set(out.team_a) | set(out.team_b)):
82         assert len(out[(out.team_a == team) | (out.team_b == team)]) == 3
83     return out
84
85
86 def load_actual_team_strength(actual: pd.DataFrame) -> pd.DataFrame:
87     """读取世界杯开赛前 Elo 快照；不允许以题内球队中位数静默补齐。"""

```

```

88 path = ROOT / "user_data" / "worldcup_2022_pre_tournament_elo.csv"
89 ratings = pd.read_csv(path)
90 assert ratings["team_name"].is_unique and len(ratings) == 32
91 actual_teams = set(actual["team_a"]) | set(actual["team_b"])
92 assert actual_teams == set(ratings["team_name"]), sorted(actual_teams ^ set(ratings["team_name"]))
93 assert ratings["elo_rating"].between(1000, 2500).all()
94 assert ratings["rating_date"].eq("2022-11-12").all()
95 assert ratings["source_url"].eq(TEAM_ELO_SOURCE_URL).all()
96 return ratings.set_index("team_name")
97
98
99 def _match_features(elo_a: float, elo_b: float, pa: float, pd: float, pb: float,
100 round_in_group: int) -> list[float]:
101     """只使用现实与题内两侧都能在赛前取得同构特征。"""
102     return [pa, pd, pb, 1-max(pa, pd), pb], (elo_a+elo_b)/2,
103         abs(elo_a-elo_b), float(round_in_group)]
104
105
106 def attach_match_proxies(actual, sheets, p1_group):
107     teams, groups, base = sheets["teams"], sheets["groups_matches"], sheets["base_predictions"]
108     actual_strength = load_actual_team_strength(actual)
109     synthetic_strength = teams.set_index("team_name")["elo_rating"]
110     attach = groups.merge(base, on=["match_id", "group_id", "round_in_group", "team_a", "team_b"], validate="one_to_one")
111     attach = attach.merge(sheets["security_requirements"][["match_id", "security_demand_score", "required_security_level"]], on="match_id", validate="
        one_to_one")
112     attach = attach.merge(p1_group[["match_id", "predicted_tv_viewers"]], on="match_id", validate="one_to_one")
113     X_train = []
114     for r in attach.itertuples(index=False):
115         elo_a, elo_b = float(synthetic_strength.loc[str(r.team_a)]), float(synthetic_strength.loc[str(r.team_b)])
116         X_train.append(_match_features(elo_a, elo_b, float(r.p_a_win), float(r.p_draw),
117             float(r.p_b_win), int(r.round_in_group)))
118     X_actual = []
119     pregame = []
120     for r in actual.itertuples(index=False):
121         elo_a = float(actual_strength.loc[str(r.team_a)], "elo_rating")
122         elo_b = float(actual_strength.loc[str(r.team_b)], "elo_rating")
123         # 仅用赛前 Elo 差构造 Bradley-Terry 概率, 不读取实际赛果。
124         win_a = 1/(1+10**((elo_b-elo_a)/400)); draw = 0.24
125         pa, pb = (1-draw)*win_a, (1-draw)*(1-win_a)
126         X_actual.append(_match_features(elo_a, elo_b, pa, draw, pb, int(r.round_in_group)))
127         pregame.append((pa, draw, pb))
128     scaler = StandardScaler().fit(np.asarray(X_train))
129     nn = NearestNeighbors(n_neighbors=2).fit(scaler.transform(np.asarray(X_train)))
130     _, neigh = nn.kneighbors(scaler.transform(np.asarray(X_actual)))
131     rows = actual.copy()
132     cols = ["expected_attendance_base", "attractiveness_index", "uncertainty_index", "commercial_value_index",
133         "security_demand_score", "required_security_level", "predicted_tv_viewers"]
134     for c in cols:
135         rows[c] = [attach.iloc[j][c] for j in neigh[:,0]]
136         rows[f"proxy2_{c}"] = [attach.iloc[j][c] for j in neigh[:,1]]
137     rows[["p_a_win", "p_draw", "p_b_win"]] = np.asarray(pregame)
138     # U 属于可由同一组三向赛前概率直接计算的指标, 不能从另一场比赛继承。
139     rows["uncertainty_index"] = 1.0 - rows[["p_a_win", "p_draw", "p_b_win"]].max(axis=1)
140     rows["proxy_match_id"] = [attach.iloc[j].match_id for j in neigh[:,0]]
141     rows["proxy2_match_id"] = [attach.iloc[j].match_id for j in neigh[:,1]]
142     rows["is_proxy"] = True
143     return rows
144
145
146 def attach_venue_proxies(actual, venues):
147     """仅按有官方来源的容量映射; 安保等级作为可行域而不是距离特征。"""
148     actual_feat = actual[["capacity"]].to_numpy(float)
149     train_feat = venues[["capacity"]].to_numpy(float)
150     scaler = StandardScaler().fit(train_feat)
151     train_scaled = scaler.transform(train_feat)
152     actual_scaled = scaler.transform(actual_feat)
153     neigh = []
154     for i, row in enumerate(actual.itertuples(index=False)):
155         primary_legal = np.flatnonzero(venues["security_level"].to_numpy(int) >= int(row.required_security_level))
156         second_legal = np.flatnonzero(venues["security_level"].to_numpy(int) >= int(row.proxy2_required_security_level))
157         assert len(primary_legal) >= 1 and len(second_legal) >= 2
158         primary_dist = np.square(train_scaled[primary_legal] - actual_scaled[i]).sum(axis=1)
159         second_dist = np.square(train_scaled[second_legal] - actual_scaled[i]).sum(axis=1)
160         primary = primary_legal[np.argsort(primary_dist, kind="mergesort")[0]]
161         second = second_legal[np.argsort(second_dist, kind="mergesort")[1]]
162         neigh.append([primary, second])

```

```

163     neigh = np.asarray(neigh, dtype=int)
164     out = actual.copy()
165     for c in ["venue_id", "setup_cost_musd", "operation_cost_musd_per_match", "security_level", "security_cost_index"]:
166         out[f"proxy_{c}"] = [venues.iloc[j][c] for j in neigh[:,0]]
167         out[f"proxy2_{c}"] = [venues.iloc[j][c] for j in neigh[:,1]]
168     return out
169
170
171 def _haversine(lat1, lon1, lat2, lon2):
172     p1,p2=np.radians(lat1),np.radians(lat2); dp=np.radians(lat2-lat1); dl=np.radians(lon2-lon1)
173     a=np.sin(dp/2)**2+np.cos(p1)*np.cos(p2)*np.sin(dl/2)**2
174     return 6371*2*np.arctan2(np.sqrt(a),np.sqrt(1-a))
175
176
177 def map_prime(rows, slots):
178     """按赛事相对进度日和 UTC 钟点唯一映射，消除跨日期同距的任意并列。"""
179     slots = slots.copy()
180     slots["utc"] = pd.to_datetime(slots.reference_utc_time, utc=True)
181     slots["slot_date"] = pd.to_datetime(slots["date"]).dt.date
182     actual_dates = sorted(pd.to_datetime(rows["date"]).dt.date.unique())
183     slot_dates = sorted(slots["slot_date"].unique())
184     actual_day_index = {day: idx for idx, day in enumerate(actual_dates)}
185     values=[]; flags=[]; mapped_ids=[]
186     prime_ids = set(slots.nlargest(20, "global_prime_score")["slot_id"].astype(str))
187     for r in rows.itertuples(index=False):
188         utc = pd.Timestamp(r.utc_datetime)
189         day = pd.Timestamp(r.date).date()
190         progress = actual_day_index[day] / max(len(actual_dates)-1, 1)
191         mapped_day = slot_dates[int(round(progress * (len(slot_dates)-1)))]
192         candidates = slots[slots["slot_date"].eq(mapped_day)].copy()
193         clock = utc.hour + utc.minute/60
194         candidate_clock = candidates["utc"].dt.hour + candidates["utc"].dt.minute/60
195         raw_delta = (candidate_clock-clock).abs()
196         candidates["_clock_delta"] = np.minimum(raw_delta, 24-raw_delta)
197         nearest = candidates.sort_values(["_clock_delta", "slot_id", kind="mergesort"]).iloc[0]
198         values.append(float(nearest["global_prime_score"]))
199         flags.append(str(nearest["slot_id"]) in prime_ids)
200         mapped_ids.append(str(nearest["slot_id"]))
201     return np.asarray(values, dtype=float), np.asarray(flags, dtype=bool), np.asarray(mapped_ids, dtype=object)
202
203
204 def build_p2_travel_context(sheets):
205     """构造与 P2 完全相同的候选域、安保规则、三分项边界和旅行负担表。"""
206     groups = sheets["groups_matches"].merge(
207         sheets["security_requirements"][["match_id", "required_security_level"]],
208         on="match_id", validate="one_to_one",
209     ).reset_index(drop=True)
210     venues = sheets["venues"].reset_index(drop=True)
211     burden = compute_travel(groups, venues, sheets["security_requirements"], sheets["distance_matrix"])
212     return {
213         "burden": burden,
214         "match_index": {str(m): int(i) for i, m in enumerate(groups["match_id"])},
215         "venue_index": {str(v): int(i) for i, v in enumerate(venues["venue_id"])},
216     }
217
218
219 def assign_p2_proxy_travel(rows, context):
220     """将现实赛程的比赛/场馆整体映射到 P2 候选域后，复算同定义 D。"""
221     values = []
222     for row in rows.itertuples(index=False):
223         i = context["match_index"][str(row.proxy_match_id)]
224         v = context["venue_index"][str(row.proxy_venue_id)]
225         key = (i, v)
226         assert key in context["burden"], (row.proxy_match_id, row.proxy_venue_id)
227         values.append(float(context["burden"][key]))
228     return np.asarray(values, dtype=float)
229
230
231 def prepare_actual_rows(actual, sheets, travel_context):
232     prime_score, prime, mapped_slot = map_prime(actual, sheets["time_slots"])
233     z=actual.copy(); z["global_prime_score"]=prime_score; z["is_prime"]=prime.astype(int)
234     z["prime_proxy_slot_id"] = mapped_slot
235     large_stadiums=set(z.groupby("venue")["capacity"].first().nlargest(2).index)
236     z["is_large_venue"]=z.venue.isin(large_stadiums).astype(int)
237     # 真实场馆路径只保留为结构层公里量，不再冒充 P2 的复合旅行指标 D。
238     travel=np.zeros(len(z)); team_last={}

```

```

239     for idx,r in z.sort_values("utc_datetime").iterrows():
240         vals=[]
241         for team in (r.team_a,r.team_b):
242             if team in team_last:
243                 prev=team_last[team]; vals.append(_haversine(prev[0],prev[1],r.latitude,r.longitude))
244             else: vals.append(0.0)
245             team_last[team]=(r.latitude,r.longitude)
246         travel[idx]=np.mean(vals)
247     z["path_travel_km_per_team"]=travel
248     z["travel_cost_index"] = assign_p2_proxy_travel(z, travel_context)
249     z["expected_attendance"]=np.minimum(z.expected_attendance_base,z.capacity)
250     z["ticket_revenue_usd"]=[(80 if r==1 else 95 if r==2 else 110)*n for r,n in zip(z.round_in_group,z.expected_attendance)]
251     unit={1:0.035,2:0.040,3:0.052}; sponsor={1:0.85,2:0.90,3:1.10}
252     z["broadcast_value_usd"]=[tv*unit[r]*gp*sponsor[r] for tv,r,gp in zip(z.predicted_tv_viewers,z.round_in_group,z.global_prime_score)]
253     z["risk_index"]=0.5*ACTUAL_CLIMATE_SCENARIO+0.3*z.expected_attendance/z.capacity+0.2*z.required_security_level/z.proxy_security_level
254     return z
255
256
257 def prepare_optimized_rows(schedule, sheets):
258     z=schedule.copy(); slots=sheets["time_slots"].set_index("slot_id"); venues=sheets["venues"].set_index("venue_id")
259     prime_ids=set(sheets["time_slots"].nlargest(20,"global_prime_score").slot_id)
260     large_ids=set(sheets["venues"].nlargest(4,"capacity").venue_id)
261     z["is_prime"]=z.slot_id.isin(prime_ids).astype(int); z["is_large_venue"]=z.venue_id.isin(large_ids).astype(int)
262     z["setup_cost_musd"]=[venues.loc[v,"setup_cost_musd"] for v in z.venue_id]
263     z["operation_cost_musd_per_match"]=[venues.loc[v,"operation_cost_musd_per_match"] for v in z.venue_id]
264     z["security_cost_index"]=[venues.loc[v,"security_cost_index"] for v in z.venue_id]
265     z["uncertainty_index"]=sheets["base_predictions"].set_index("match_id").loc[z.match_id,"uncertainty_index"].to_numpy()
266     z["attractiveness_index"]=sheets["base_predictions"].set_index("match_id").loc[z.match_id,"attractiveness_index"].to_numpy()
267     z["team_a_id"]=sheets["groups_matches"].set_index("match_id").loc[z.match_id,"team_a_id"].to_numpy()
268     z["team_b_id"]=sheets["groups_matches"].set_index("match_id").loc[z.match_id,"team_b_id"].to_numpy()
269     for c in ["capacity", "latitude", "longitude", "timezone"]:
270         z[f"venue_{c}"] = [venues.loc[v, c] for v in z.venue_id]
271     return z
272
273
274 STRUCTURAL_DEFINITIONS = {
275     "rest_min_hours": {"category": "休息", "unit": "小时", "preferred_direction": "higher",
276         "denominator": "全部球队相邻两轮间隔", "definition": "球队相邻比赛 UTC 间隔的最小值", "evidence": "direct"},
277     "rest_mean_hours": {"category": "休息", "unit": "小时", "preferred_direction": "higher",
278         "denominator": "全部球队相邻两轮间隔", "definition": "球队相邻比赛 UTC 间隔的平均值", "evidence": "direct"},
279     "timezone_crossings": {"category": "时区", "unit": "次", "preferred_direction": "lower",
280         "denominator": "全部球队相邻两轮间隔", "definition": "相邻比赛场馆 UTC 偏移发生变化的次数", "evidence": "direct"},
281     "timezone_crossing_rate": {"category": "时区", "unit": "比例", "preferred_direction": "lower",
282         "denominator": "球队转场次数", "definition": "跨时区次数除以球队转场次数", "evidence": "direct"},
283     "travel_mean_km": {"category": "旅行", "unit": "公里/次", "preferred_direction": "lower",
284         "denominator": "球队转场次数", "definition": "相邻比赛实际场馆大圆距离的平均值", "evidence": "direct"},
285     "venue_change_rate": {"category": "旅行", "unit": "比例", "preferred_direction": "lower",
286         "denominator": "球队转场次数", "definition": "相邻轮更换场馆的球队转场比例", "evidence": "direct"},
287     "venue_utilization_rate": {"category": "场馆", "unit": "比例", "preferred_direction": "higher",
288         "denominator": "启用场馆数乘比赛日期数", "definition": "发生比赛的场馆日占可用场馆日比例", "evidence": "direct"},
289     "venue_daily_peak": {"category": "场馆", "unit": "场/场馆日", "preferred_direction": "lower",
290         "denominator": "单个场馆当地自然日", "definition": "单一场馆单日承办场次最大值", "evidence": "direct"},
291     "capacity_occupancy_rate": {"category": "容量", "unit": "比例", "preferred_direction": "higher",
292         "denominator": "逐场容量后取场均", "definition": "预计现场观众与场馆容量之比的场均值", "evidence": "proxy_demand"},
293     "prime_coverage_rate": {"category": "黄金时段", "unit": "比例", "preferred_direction": "higher",
294         "denominator": "比赛场次", "definition": "按赛事进度日和 UTC 钟点映射后落入前四分位时段的比例", "evidence": "time_mapping"},
295     "expected_attendance_per_match": {"category": "观众", "unit": "人/场", "preferred_direction": "higher",
296         "denominator": "比赛场次", "definition": "预计现场观众总量除以比赛场次", "evidence": "proxy_demand"},
297 }
298
299
300 def structural_metrics(rows: pd.DataFrame, kind: str) -> dict:
301     """对两套赛程调用同一结构函数：总量均附明确规模分母。"""
302     ta = "team_a" if kind == "actual" else "team_a_id"
303     tb = "team_b" if kind == "actual" else "team_b_id"
304     venue = "venue" if kind == "actual" else "venue_id"
305     capacity = "capacity" if kind == "actual" else "venue_capacity"
306     lat = "latitude" if kind == "actual" else "venue_latitude"
307     lon = "longitude" if kind == "actual" else "venue_longitude"
308     z = rows.copy()
309     z["_utc"] = pd.to_datetime(z["_utc_datetime"], utc=True)
310     z["_local_date"] = [pd.Timestamp(x).date().isoformat() for x in z["_local_datetime"]]
311     z["_utc_offset"] = [pd.Timestamp(x).utcoffset().total_seconds() / 3600 for x in z["_local_datetime"]]
312
313     rest, travel, timezone_shift = [], [], []
314     venue_changes = 0

```

```

315 teams = sorted(set(z[ta]) | set(z[tb]))
316 for team in teams:
317     q = z[(z[ta] == team) | (z[tb] == team)].sort_values("_utc")
318     records = q.to_dict(orient="records")
319     for previous, current in zip(records[:-1], records[1:]):
320         rest.append((current["_utc"] - previous["_utc"]).total_seconds() / 3600)
321         travel.append(haversine(previous[lat], previous[lon], current[lat], current[lon]))
322         timezone_shift.append(abs(current["_utc_offset"] - previous["_utc_offset"]))
323         venue_changes += int(previous[venue] != current[venue])
324
325 transitions = len(rest)
326 assert transitions > 0
327 venue_days = z[[venue, "_local_date"]].drop_duplicates()
328 n_venues = int(z[venue].nunique()); n_dates = int(z["_local_date"].nunique())
329 crossings = int(np.sum(np.asarray(timezone_shift) > 1e-9))
330 metrics = {
331     "rest_min_hours": float(np.min(rest)),
332     "rest_mean_hours": float(np.mean(rest)),
333     "timezone_crossings": float(crossings),
334     "timezone_crossing_rate": float(crossings / transitions),
335     "travel_mean_km": float(np.mean(travel)),
336     "venue_change_rate": float(venue_changes / transitions),
337     "venue_utilization_rate": float(len(venue_days) / (n_venues * n_dates)),
338     "venue_daily_peak": float(z.groupby([venue, "_local_date"]).size().max()),
339     "capacity_occupancy_rate": float(np.mean(np.minimum(z["expected_attendance"].to_numpy(float) / z[capacity].to_numpy(float), 1.0))),
340     "prime_coverage_rate": float(z["is_prime"].mean()),
341     "expected_attendance_per_match": float(z["expected_attendance"].mean()),
342     "match_count": int(len(z)), "team_transition_count": int(transitions),
343     "active_venue_count": n_venues, "match_date_count": n_dates,
344 }
345 assert set(STRUCTURAL_DEFINITIONS).issubset(metrics)
346 return metrics
347
348 def _comparison_row(name, category, actual, optimized, direction):
349     a, o = float(actual), float(optimized)
350     diff = o - a
351     imp = None if abs(a) <= 1e-15 else ((o-a)/abs(a) if direction == "higher" else (a-o)/abs(a))
352     return {"indicator_name": name, "indicator_category": category,
353           "actual_schedule_value": a, "optimized_schedule_value": o,
354           "absolute_difference": diff, "relative_improvement": imp,
355           "preferred_direction": direction,
356           "evaluation_result": "优化较优" if imp is not None and imp > 0 else "实际较优或持平"}
357
358
359 def structural_comparison_rows(actual_m, optimized_m):
360     return [_comparison_row(k, f"结构-{spec['category']}", actual_m[k], optimized_m[k], spec["preferred_direction"])
361            for k, spec in STRUCTURAL_DEFINITIONS.items()]
362
363
364 def validate_p4_c2(actual, optimized, actual_structure, optimized_structure, structure_comp):
365     """P4-C2 机器网：任一题定结构维度、单位、方向或分母缺失即失败。"""
366     required = set(STRUCTURAL_DEFINITIONS)
367     assert required == set(row["indicator_name"] for row in structure_comp)
368     for metrics in (actual_structure, optimized_structure):
369         assert required.issubset(metrics)
370         assert np.isfinite([metrics[k] for k in required]).all()
371     for spec in STRUCTURAL_DEFINITIONS.values():
372         assert spec["unit"] and spec["preferred_direction"] in {"higher", "lower"} and spec["denominator"]
373     recalculated_u = 1.0 - actual[["p_a_win", "p_draw", "p_b_win"]].max(axis=1).to_numpy(float)
374     assert np.allclose(actual["uncertainty_index"].to_numpy(float), recalculated_u, atol=1e-12)
375     assert "path_travel_km_per_team" in actual and "travel_cost_index" in actual
376     assert not np.allclose(actual["path_travel_km_per_team"], actual["travel_cost_index"])
377     assert len(actual) == 48 and len(optimized) == 72
378     return {"status": "PASS", "required_dimensions": sorted(required),
379           "actual_matches": 48, "optimized_matches": 72,
380           "travel_contract": "physical_km_separate_from_P2_composite_D"}
381
382
383 def raw_schedule_metrics(rows: pd.DataFrame, kind: str) -> dict:
384     """先计算未归一化的同口径指标，供共同边界一次性拟合。"""
385     n=len(rows); preferred_direction={"T":"higher","B":"higher","U":"higher","H":"higher","C":"lower","D":"lower","F":"lower","R":"lower"}
386     scale_denominator={"T":n,"B":n,"C":n,"D":n,"R":n,"U":n,"H":n,"F":"three_matches_per_team"}
387     T0=float(rows.ticket_revenue_usd.sum()/n); B0=float(rows.broadcast_value_usd.sum()/n)
388     if kind=="actual":
389         byvenue=rows.groupby("venue"); setup=sum(float(g.proxy_setup_cost_musd.iloc[0])*1e6 for _,g in byvenue)
390         op=sum(float(r.proxy_operation_cost_musd_per_match)*1e6+SECURITY_BASE_COST_USD*r.required_security_level*r.proxy_security_cost_index for r in rows.

```

```

    itertuples())
391 else:
392     byvenue=rows.groupby("venue_id"); setup=sum(float(g.setup_cost_musd.iloc[0])*1e6 for _,g in byvenue)
393     op=sum(float(r.operation_cost_musd_per_match)*1e6+SECURITY_BASE_COST_USD*r.required_security_level*r.security_cost_index for r in rows.itertuples()
    )
394 C0=(setup+op)/n; U=float(rows.uncertainty_index.mean()); H=float(rows.attractiveness_index.mean()/100)
395 D=float(rows.travel_cost_index.mean()); R=float(rows.risk_index.mean())
396 ta="team_a" if kind=="actual" else "team_a_id"; tb="team_b" if kind=="actual" else "team_b_id"
397 teams=sorted(set(rows[ta])|set(rows[tb])); pc=[];lc=[]
398 for team in teams:
399     q=rows[(rows[ta]==team)|(rows[tb]==team)]; pc.append(int(q.is_prime.sum()));lc.append(int(q.is_large_venue.sum()))
400 F=0.5*(max(pc)-min(pc))/3+0.5*(max(lc)-min(lc))/3
401 return {"T":T0,"B":B0,"U":U,"H":H,"C":C0,"D":D,"F":float(F),"R":R,
402         "preferred_direction":preferred_direction,"scale_denominator":scale_denominator}
403
404
405 def common_comparison_bounds(base_bounds: dict, *raw_metrics: dict) -> dict:
406     """用 P2 边界与全部已报告 P4 场景的原始值构造共同包络。"""
407     bounds = {}
408     for key in ("T", "B", "C"):
409         observed = [float(metrics[key]) for metrics in raw_metrics]
410         lower = min(float(base_bounds[key][0]), *observed)
411         upper = max(float(base_bounds[key][1]), *observed)
412         assert upper > lower
413         bounds[key] = (lower, upper)
414     return bounds
415
416
417 def evaluate_schedule(raw: dict, comparison_bounds: dict) -> dict:
418     """在两方案和敏感性场景共用的边界下执行 Min-Max, 结果严格属于 [0,1]。"""
419     vals={"T":float(minmax(raw["T"],*comparison_bounds["T"])),
420          "B":float(minmax(raw["B"],*comparison_bounds["B"])),
421          "U":float(raw["U"]), "H":float(raw["H"]),
422          "C":float(minmax(raw["C"],*comparison_bounds["C"])),
423          "D":float(raw["D"]), "F":float(raw["F"]), "R":float(raw["R"])}
424     assert all(-1e-12 <= vals[k] <= 1+1e-12 for k in ("T","B","U","H","C","D","F","R")), vals
425     vals["Z4"]=(0.25*vals["T"]+0.25*vals["B"]+0.15*vals["U"]+0.10*vals["H"]
426               -0.08*vals["C"]-0.07*vals["D"]-0.06*vals["F"]-0.04*vals["R"])
427     vals.update({"T_raw_per_match":float(raw["T"]), "B_raw_per_match":float(raw["B"]),
428               "C_raw_per_match":float(raw["C"]),
429               "preferred_direction":raw["preferred_direction"],
430               "scale_denominator":raw["scale_denominator"],
431               "normalization_bounds":{k:list(v) for k,v in comparison_bounds.items()},
432               "scenario_extrapolation":False,"extrapolated_indicators":[]})
433     return vals
434
435
436 def comparison_rows(actual_m, optimized_m):
437     info={"T":("价值","higher"),"B":("价值","higher"),"U":("竞技","higher"),"H":("吸引力","higher"),
438          "C":("成本","lower"),"D":("旅行","lower"),"F":("公平","lower"),"R":("风险","lower"),"Z4":("综合","higher")}
439     rows=[]
440     for k,(cat,direction) in info.items():
441         rows.append(_comparison_row(k, f"代理-{cat}", actual_m[k], optimized_m[k], direction))
442     return rows
443
444
445 def solve() -> dict:
446     set_all_seeds(MASTER_SEED); start=time.time(); sheets=load_all_sheets()
447     actual_schedule=collect_actual_schedule(); p1=pd.read_csv(OUTPUT_DIR/"result_1_match_prediction.csv")
448     actual_proxy=attach_match_proxies(actual_schedule,sheets,p1); actual_proxy=attach_venue_proxies(actual_proxy,sheets["venues"])
449     travel_context = build_p2_travel_context(sheets)
450     actual=prepare_actual_rows(actual_proxy,sheets,travel_context)
451     actual_proxy2=actual_proxy.copy()
452     for c in ["expected_attendance_base","attractiveness_index","commercial_value_index",
453             "security_demand_score","required_security_level","predicted_tv_viewers"]:
454         actual_proxy2[c]=actual_proxy2[f"proxy2_{c}"]
455     for c in ["venue_id","setup_cost_musd","operation_cost_musd_per_match","security_level","security_cost_index"]:
456         actual_proxy2[f"proxy_{c}"]=actual_proxy2[f"proxy2_{c}"]
457     actual_proxy2["proxy_match_id"] = actual_proxy2["proxy2_match_id"]
458     actual_proxy2["proxy_venue_id"] = actual_proxy2["proxy2_venue_id"]
459     actual_second=prepare_actual_rows(actual_proxy2,sheets,travel_context)
460     schedule=pd.read_csv(OUTPUT_DIR/"result_2_group_schedule.csv"); optimized=prepare_optimized_rows(schedule,sheets)
461     p2=json.loads((FIGURES_DIR/"problem_2_results.json").read_text(encoding="utf-8"))["normalization_bounds"]
462     base_bounds={"T":(p2["T_min"]/72,p2["T_max"]/72),"B":(p2["B_min"]/72,p2["B_max"]/72),"C":(p2["C_min"]/72,p2["C_max"]/72)}
463     actual_raw=raw_schedule_metrics(actual,"actual")
464     optimized_raw=raw_schedule_metrics(optimized,"optimized")

```

```

465 actual_second_raw=raw_schedule_metrics(actual_second,"actual")
466 comparison_bounds=common_comparison_bounds(base_bounds,actual_raw,optimized_raw,actual_second_raw)
467 actual_m=evaluate_schedule(actual_raw,comparison_bounds)
468 optimized_m=evaluate_schedule(optimized_raw,comparison_bounds)
469 actual_second_m=evaluate_schedule(actual_second_raw,comparison_bounds)
470 actual_structure=structural_metrics(actual,"actual"); optimized_structure=structural_metrics(optimized,"optimized")
471 proxy_comp=comparison_rows(actual_m,optimized_m)
472 structure_comp=structural_comparison_rows(actual_structure,optimized_structure)
473 capability_validation=validate_p4_c2(actual,optimized,actual_structure,optimized_structure,structure_comp)
474 # 16个权重扰动：绝对值重归一且符号保留。
475 base_w=np.array([.25,.25,.15,.10,-.08,-.07,-.06,-.04]); keys=["T","B","U","H","C","D","F","R"]
476 robust=[]; base_direction=np.sign(optimized_m["Z4"]-actual_m["Z4"])
477 for j in range(8):
478     for factor in (0.9,1.1):
479         w=base_w.copy();w[j]*=factor;w=np.sign(w)*np.abs(w)/np.abs(w).sum()
480         za=sum(w[k]*actual_m[keys[k]] for k in range(8));zo=sum(w[k]*optimized_m[keys[k]] for k in range(8))
481         robust.append(("perturbed":keys[j],"factor":factor,"actual":float(za),"optimized":float(zo),"direction":int(np.sign(zo-za))))
482 direction_rate=float(np.mean([r["direction"]==base_direction for r in robust]))
483 neighbor_direction=int(np.sign(optimized_m["Z4"]-actual_second_m["Z4"]))
484 neighbor_preserved=bool(neighbor_direction==base_direction)
485 overall_stable=bool(direction_rate>=0.9 and neighbor_preserved)
486 if not overall_stable:
487     next(row for row in proxy_comp if row["indicator_name"]=="Z4")["evaluation_result"]="代理敏感，不作总体判定"
488 comp=structure_comp+proxy_comp
489 actual_cols=pd.read_csv(ROOT/"user_data"/"actual_schedule_template.csv").columns
490 OUTPUT_DIR.mkdir(parents=True,exist_ok=True)
491 actual[list(actual_cols)].to_csv(OUTPUT_DIR/"actual_schedule_2022.csv",index=False,encoding="utf-8-sig")
492 pd.DataFrame(comp).to_csv(OUTPUT_DIR/"result_4_schedule_comparison.csv",index=False,encoding="utf-8-sig")
493 assert len(actual)==48 and actual.source_url.eq(RAW_VALIDATION_URL).all()
494 assert all(np.isfinite([r["actual_schedule_value"],r["optimized_schedule_value"]]).all() for r in comp)
495 assert all(0 <= actual_m[k] <= 1 and 0 <= optimized_m[k] <= 1 for k in ("T","B","U","H","C","D","F","R"))
496 proxy_diagnostics={
497     "actual_team_elo_coverage":"32/32",
498     "primary_match_proxy_unique":int(actual["proxy_match_id"].nunique()),
499     "primary_match_proxy_max_reuse":int(actual["proxy_match_id"].value_counts().max()),
500     "primary_venue_proxy_unique":int(actual["proxy_venue_id"].nunique()),
501     "primary_venue_proxy_max_reuse":int(actual["proxy_venue_id"].value_counts().max()),
502     "prime_proxy_slot_unique":int(actual["prime_proxy_slot_id"].nunique()),
503 }
504 result={
505     "metadata":{"**runtime_metadata(MASTER_SEED),"elapsed_seconds":time.time()-start,"accessed_at":ACCESSED_AT},
506     "method":"physical structure is evaluated directly; the conditional proxy layer uses pre-tournament Elo-complete match mapping, capacity-only venue
mapping, deterministic tournament-progress time mapping, and one common envelope Min-Max domain",
507     "sources":{"machine_read_schedule":RAW_VALIDATION_URL,"official_schedule_verification":FIFA_SCHEDULE_URL,
508               "official_scores_verification":FIFA_FIXTURE_URL,"pre_tournament_elo":TEAM_ELO_SOURCE_URL,
509               "official_stadium_capacity":FIFA_STADIUM_SOURCE_URL,"official_stadium_addresses":FIFA_STADIUM_ADDRESS_URL},
510     "source_contract":{"machine_ingestion":"OpenFootball structured JSON snapshot",
511                      "official_verification":"FIFA schedule, scores, stadium capacity and address pages"},
512     "actual_schedule_rows":actual.to_dict(orient="records"),"actual_metrics":actual_m,"optimized_metrics":optimized_m,
513     "common_normalization_bounds":{"k:list(v) for k,v in comparison_bounds.items()}},
514     "structural_definitions":STRUCTURAL_DEFINITIONS,"actual_structural_metrics":actual_structure,
515     "optimized_structural_metrics":optimized_structure,"structural_comparison":structure_comp,
516     "capability_validation":{"P4-C2":capability_validation},
517     "proxy_diagnostics":proxy_diagnostics,
518     "proxy_comparison":proxy_comp,"comparison":comp,"weight_robustness":robust,"direction_robustness_rate":direction_rate,
519     "weight_direction_stable":bool(direction_rate>=0.9),
520     "overall_conclusion_stable":overall_stable,
521     "overall_conclusion":"代理映射敏感，仅报告指标级取舍 if not overall_stable else "综合方向在已检验权重和邻居代理下稳定，但仍为条件性结论",
522     "second_neighbor_sensitivity":{"actual_Z4":float(actual_second_m["Z4"]),
523                                  "baseline_direction":int(base_direction),
524                                  "direction_preserved":neighbor_preserved},
525     "proxy_note":"结构层报告物理量；代理层用完整赛前Elo映射比赛、仅按容量和安保可行域映射场馆，黄金时段按赛事进度日与UTC钟点唯一映射；T/B/C在全部报告场景共用的包络边界上归一化。",
526     "unobserved_constraints":["抽签结果","转播合同","场馆可用性","当地治理","商业谈判"],
527     "seed":MASTER_SEED,"outputs":["output/actual_schedule_2022.csv","output/result_4_schedule_comparison.csv"],
528 }
529 write_json(FIGURES_DIR/"problem_4_results.json",result)
530 print(f"[P4] validate_capability PASS actual_Z4={actual_m['Z4']:.6f} optimized_Z4={optimized_m['Z4']:.6f} robust={direction_rate:.1%}",flush=True)
531 return result
532
533
534 if __name__=="__main__":
535     solve()

```

Listing 10: 灵敏度分析完整程序

```

1  """模型检验与灵敏度：P2权重响应面、P3状态/伤病参数3*3重算。"""
2  import os, sys
3  _HERE = os.path.dirname(os.path.abspath(__file__))
4  if _HERE not in sys.path:
5      sys.path.insert(0, _HERE)
6
7  import json
8  import numpy as np
9  import pandas as pd
10
11  import problem3 as p3
12  from params import MASTER_SEED
13  from utils import FIGURES_DIR, OUTPUT_DIR, load_all_sheets, write_json
14
15
16  def p2_fixed_schedule_surface(metrics):
17      """固定最优赛程的权重响应，不冒充每格重新优化。"""
18      commercial=np.linspace(0.9,1.1,5); penalty=np.linspace(0.9,1.1,5); rows=[]
19      for c in commercial:
20          for p in penalty:
21              raw=np.array([.25*c,.25*c,.15,.10,.08*p,.07*p,.06*p,.04*p]); raw=raw/raw.sum()
22              z=raw[0]*metrics["T"]+raw[1]*metrics["B"]+raw[2]*metrics["U"]+raw[3]*metrics["H"]-raw[4]*metrics["C"]-raw[5]*metrics["D"]-raw[6]*metrics["F"]-
                raw[7]*metrics["R"]
23              rows.append({"commercial_multiplier":float(c),"penalty_multiplier":float(p),"fixed_schedule_Z2":float(z)})
24      return rows
25
26
27  def p3_surface():
28      sheets=load_all_sheets(); groups=sheets["groups_matches"]
29      base=groups.merge(sheets["base_predictions"],on=["match_id","group_id","round_in_group","team_a","team_b"],validate="one_to_one")
30      base=base.merge(sheets["security_requirements"][["match_id","security_demand_score","required_security_level"]],on="match_id",validate="one_to_one")
31      p1=pd.read_csv(OUTPUT_DIR/"result_1_match_prediction.csv");base=base.merge(p1[["match_id","predicted_tv_viewers"]],on="match_id",validate="one_to_one")
32      round3=base[base.round_in_group.eq(3)].reset_index(drop=True);teams=sheets["group_membership"].team_name.astype(str).tolist()
33      snap=p3.team_snapshot(sheets["live_group_results"],teams);feedback=p3.feedback_by_team(sheets["live_group_results"],sheets["base_predictions"],p1)
34      schedule=pd.read_csv(OUTPUT_DIR/"result_2_group_schedule.csv");rows=[]
35      state0,inj0=p3.DELTA_STATE_COEF,p3.DELTA_INJURY_COEF
36      for sf in (0.9,1.0,1.1):
37          for hf in (0.9,1.0,1.1):
38              p3.DELTA_STATE_COEF=state0*sf;p3.DELTA_INJURY_COEF=inj0*hf
39              la,lb=p3.updated_lambdas(round3,snap);sim=p3.simulate_qualification(round3,sheets["group_membership"],snap,la,lb,MASTER_SEED,keep_scores=True)
40              probs={t:float(sim["probability"][i]) for i,t in enumerate(sim["teams"])};cond=p3.conditional_probabilities(sim,round3)
41              env=p3.derive_match_environment(round3,schedule,sheets["venues"],sheets["ticket_broadcast"],sheets["dynamic_resource_limits"],snap,feedback,
                probs,cond)
42              actions=p3.make_action_table(env,sheets["dynamic_resource_costs"],sheets["dynamic_resource_limits"],True);bounds=p3.action_bounds(actions)
43              chosen,_=p3.optimize_actions(actions,sheets["dynamic_resource_limits"],bounds,f"sens_{sf}_{hf}")
44              z=sum(p3.action_value(a,bounds) for a in chosen)
45              rows.append({"state_coefficient_multiplier":sf,"injury_coefficient_multiplier":hf,"Z3":float(z),"mean_advancement_probability":float(np.mean(
                list(probs.values())))})
46              print(f"[sensitivity] state={sf:.1f} injury={hf:.1f} Z3={z:.6f}",flush=True)
47      p3.DELTA_STATE_COEF,p3.DELTA_INJURY_COEF=state0,inj0
48      return rows
49
50
51  def main():
52      p2=json.loads((FIGURES_DIR/"problem_2_results.json").read_text(encoding="utf-8"))
53      p4=json.loads((FIGURES_DIR/"problem_4_results.json").read_text(encoding="utf-8"))
54      result={"p2_weight_surface":p2_fixed_schedule_surface(p2["metrics"]),"p2_note":"固定主赛程重评，不冒充逐格重优化",
55              "p3_state_injury_surface":p3_surface(),"p4_weight_robustness":p4["weight_robustness"],"seed":MASTER_SEED}
56      write_json(FIGURES_DIR/"sensitivity_results.json",result)
57      print("[sensitivity] PASS")
58
59
60  if __name__=="__main__":
61      main()

```

Listing 11: 正式输出约束复核程序

```

1  """从最终 JSON/CSV 独立重算四问硬约束；仅输出结论和最多5个定位。"""
2  import os, sys
3  _HERE = os.path.dirname(os.path.abspath(__file__))
4  if _HERE not in sys.path:
5      sys.path.insert(0, _HERE)

```

```

6
7 import json
8 import re
9 from pathlib import Path
10
11 import numpy as np
12 import pandas as pd
13
14 from params import ATTENDANCE_FLOOR_RATIO, GOAL_MEAN_CLIP, MINIMUM_REST_HOURS
15 from utils import FIGURES_DIR, OUTPUT_DIR, load_all_sheets
16
17 _BASE_KW = re.compile(r"baseline|naive|greedy|就近|等权|均分|不调整|lower.?bound|下界", re.I)
18
19
20 def _discover_baselines(obj, prefix=""):
21     found=[]
22     if isinstance(obj,dict):
23         for k,v in obj.items():
24             path=f"{prefix}.{k}" if prefix else k
25             if _BASE_KW.search(k): found.append(path)
26             if isinstance(v,dict): found.extend(_discover_baselines(v,path))
27     return found
28
29
30 def main() -> None:
31     sheets=load_all_sheets(); audited=set(); failures=[]; n_constraints=0
32     p1=json.loads((FIGURES_DIR/"problem_1_results.json").read_text(encoding="utf-8"))
33     p2=json.loads((FIGURES_DIR/"problem_2_results.json").read_text(encoding="utf-8"))
34     p3=json.loads((FIGURES_DIR/"problem_3_results.json").read_text(encoding="utf-8"))
35     p4=json.loads((FIGURES_DIR/"problem_4_results.json").read_text(encoding="utf-8"))
36     discovered=_discover_baselines({"p1":p1,"p2":p2,"p3":p3,"p4":p4})
37
38     # P1: 切分、交付和正值。
39     test=pd.read_csv(OUTPUT_DIR/"result_1_test_prediction.csv"); group=pd.read_csv(OUTPUT_DIR/"result_1_match_prediction.csv")
40     validation_policy=str(p1.get("validation_target_history_policy", ""))
41     ok=(len(test)==140 and test.match_id_test.is_unique and (test.predicted_test_tv_viewers>0).all()
42         and len(group)==72 and group.match_id.is_unique and (group.predicted_tv_viewers>0).all()
43         and "mask" in validation_policy.lower())
44     n_constraints+=6
45     if not ok: failures.append("P1 split/delivery/range")
46     print(f"[P1] PASS={ok} n_violations={0 if ok else 1} rows_test={len(test)} rows_group={len(group)}")
47
48     # P2 最优解与自动发现的可行基线使用同一审计函数。
49     from problem2 import prepare_data, validate_schedule, evaluate_rows
50     _,groups,venues,slots,limits,_=prepare_data(); bounds=p2["normalization_bounds"]
51     for name,rows in [{"p2.schedule",p2["schedule"]}, {"p2.feasible_baseline",p2["feasible_baseline"]}]:
52         try:
53             a=validate_schedule(rows,groups,venues,slots,limits); m=evaluate_rows(rows,groups,venues,bounds)
54             ok=a["pass"] and all(-1e-9<=m[k]<=1+1e-9 for k in ["T","B","U","H","C","D","F","R"])
55         except Exception as e:
56             ok=False; failures.append(f"{name}: {e}")
57         n_constraints+=10; audited.add(name)
58         print(f"[{name}] PASS={ok} n_violations={0 if ok else 1}")
59
60     # P3: 守恒、概率/lambda、固定赛程、安保、票价需求和日容量。
61     mass=np.asarray(p3["qualifier_mass_by_draw"],float); probs=np.asarray([x["probability"] for x in p3["team_advancement"]]);
62     lamb=np.asarray([y for x in p3["lambdas"] for y in (x["lambda_a"],x["lambda_b"])]); rows=p3["rows"]
63     sched=pd.read_csv(OUTPUT_DIR/"result_2_group_schedule.csv").set_index("match_id")
64     seed_runs=p3.get("five_seed_decision_stability", [])
65     checks=[np.max(np.abs(mass-32))<=1e-6, np.all((0<=probs)&(probs<=1)), abs(probs.sum()-32)<=1e-5,
66         np.all((GOAL_MEAN_CLIP[0]<=lamb)&(lamb<=GOAL_MEAN_CLIP[1])),
67         all((r["venue_id"],r["slot_id"])==(sched.loc[r["match_id"],"venue_id"],sched.loc[r["match_id"],"slot_id"]) for r in rows),
68         all(r["required_security_level"]<=r["recommended_security_level"]<=r["venue_security_level"] for r in rows),
69         all(r["updated_expected_attendance"]+1e-8==ATTENDANCE_FLOOR_RATIO*r["attendance_at_zero"] for r in rows),
70         all(x["pass"] for x in p3["daily_static_audit"]),all(x["pass"] for x in p3["daily_dynamic_audit"]),
71         p3["Z3_dynamic"]+1e-4>=p3["Z3_static_reeval"],
72         len(seed_runs)==5 and all(x["solver_status"]=="OPTIMAL" and x["improvement"]>=-1e-8 for x in seed_runs)]
73     ok=all(checks); n_constraints+=len(checks)
74     if not ok: failures.append("P3 hard checks")
75     print(f"[P3] PASS={ok} n_violations={sum(not x for x in checks)} max_mass_error=(np.max(np.abs(mass-32)):.3e)")
76
77     # P4: 真实行、来源、UTC、共同归一化边界及代理敏感性。
78     ar=p4["actual_schedule_rows"]; utc_ok=True
79     for r in ar:
80         local=pd.Timestamp(r["local_datetime"]); utc=pd.Timestamp(r["utc_datetime"])
81         utc_ok &= local.tz_convert("UTC")==utc

```

```

82 normalized_names=["T","B","U","H","C","D","F","R"]
83 normalized=[p4[side][name] for side in ("actual_metrics","optimized_metrics") for name in normalized_names]
84 common_bounds=p4["common_normalization_bounds"]
85 shared_bounds=all(p4[side]["normalization_bounds"]==common_bounds for side in ("actual_metrics","optimized_metrics"))
86 checks=[len(ar)==48,
87         all("openfootball" in r["source_url"].lower() for r in ar),
88         all(bool(r["official_schedule_url"]) and bool(r["stadium_source_url"]) for r in ar),
89         utc_ok,
90         all(np.isfinite(x["actual_schedule_value"]) and np.isfinite(x["optimized_schedule_value"]) for x in p4["comparison"]),
91         all((-1e-9<=x<=1+1e-9 for x in normalized),
92         shared_bounds,
93         p4["proxy_diagnostics"]["actual_team_elo_coverage"]=="32/32",
94         p4["direction_robustness_rate"]>=0.9 and p4["second_neighbor_sensitivity"]["direction_preserved"]]
95 ok=all(checks); n_constraints+=len(checks)
96 if not ok: failures.append("P4 row/source/UTC/finite")
97 print(f"[P4] PASS={ok} n_violations={sum(not x for x in checks)} actual_rows={len(ar)}")
98
99 # 自动发现的 baseline 必须有明确审计对象；只把叶级/方案级路径纳入登记断言。
100 required=[x for x in discovered if x.endswith("feasible_baseline")]
101 missing=[x for x in required if x not in audited]
102 if missing: failures.append(f"基线漏审: {missing}")
103 print(f"[baseline_registry] discovered={required} audited={sorted(audited & set(required))} missing={missing}")
104 if failures:
105     for x in failures[:5]: print(f" VIOLATION {x}")
106     raise SystemExit(1)
107 print(f"[constraint_audit] PASS n_constraints={n_constraints} n_violations=0")
108
109
110 if __name__=="__main__":
111     main()

```

Listing 12: 图表统一生成入口

```

1 """Regenerate every publication figure and LaTeX table from persisted results."""
2 from __future__ import annotations
3
4 import subprocess
5 import sys
6 from pathlib import Path
7
8
9 ROOT = Path(__file__).resolve().parents[1]
10 FIGURES = ROOT / "figures"
11
12
13 def run(path: Path) -> None:
14     print(f"[figures] running {path.relative_to(ROOT)}", flush=True)
15     subprocess.run([sys.executable, str(path)], cwd=ROOT, check=True)
16
17
18 def main() -> None:
19     run(FIGURES / "prep_plot_data.py")
20     for path in sorted(FIGURES.glob("gen_fig_*.py")):
21         run(path)
22     run(FIGURES / "generate_tables.py")
23     print("[figures] PASS", flush=True)
24
25
26 if __name__ == "__main__":
27     main()

```

Listing 13: 图表数据整理程序

```

1 """Prepare figure-only datasets from the persisted model outputs and source workbook.
2
3 All derived values are deterministic. The script never changes the modelling
4 outputs; it only gathers the exact fields needed by the publication figures.
5 """
6 from __future__ import annotations
7
8 import json

```

```

9 import sys
10 from collections import Counter, defaultdict
11 from pathlib import Path
12
13 import numpy as np
14 import pandas as pd
15 from sklearn.inspection import permutation_importance
16
17 ROOT = Path(__file__).resolve().parents[1]
18 sys.path.insert(0, str(ROOT))
19 sys.path.insert(0, str(ROOT / "code"))
20
21 from problem1 import _fit_fixed, _predict, build_features # noqa: E402
22 from problem3 import simulate_qualification, team_snapshot # noqa: E402
23
24
25 def read_json(name: str):
26     return json.loads((ROOT / "figures" / name).read_text(encoding="utf-8"))
27
28
29 def jsonable(value):
30     if isinstance(value, dict):
31         return {str(k): jsonable(v) for k, v in value.items()}
32     if isinstance(value, (list, tuple)):
33         return [jsonable(v) for v in value]
34     if isinstance(value, np.ndarray):
35         return value.tolist()
36     if isinstance(value, (np.integer,)):
37         return int(value)
38     if isinstance(value, (np.floating,)):
39         return float(value)
40     if isinstance(value, (pd.Timestamp,)):
41         return value.isoformat()
42     if pd.isna(value):
43         return None
44     return value
45
46
47 def load_workbook() -> dict[str, pd.DataFrame]:
48     candidates = sorted((ROOT / "user_data").glob("*.xlsx"))
49     if len(candidates) != 1:
50         raise RuntimeError(f"Expected exactly one source workbook, found {len(candidates)}")
51     return pd.read_excel(candidates[0], sheet_name=None)
52
53
54 def q1_data(sheets, p1):
55     hist, features, _, _ = build_features(sheets["historical_matches"], sheets["teams"])
56     train_mask = hist["dataset_split"].eq("train")
57     train = hist.loc[train_mask].reset_index(drop=True)
58     x_train = features.loc[train_mask].reset_index(drop=True)
59     y = train["tv_viewers"].to_numpy(float) / 1_000_000.0
60
61     pair = {
62         "观看人数": y,
63         "平均Elo": x_train["elo_mean"].to_numpy(float),
64         "平均排名": x_train["strength_rank_mean"].to_numpy(float),
65         "平均球迷基础": x_train["fan_base_index_mean"].to_numpy(float),
66         "赔率熵": x_train["odds_entropy"].to_numpy(float),
67     }
68
69     # The final selected model is re-fitted on the first 80% of the fixed train
70     # set, and permutation effects are evaluated only on the later 20%.
71     cut = int(np.floor(len(x_train) * 0.8))
72     model_name = p1["selected_model"]
73     if model_name == "MeanBaseline":
74         importance_rows = []
75     else:
76         fitted = _fit_fixed(model_name, x_train.iloc[:cut], y[:cut], p1["selected_params"])
77         x_valid = x_train.iloc[cut:].reset_index(drop=True)
78         y_valid = y[cut:]
79         imp = permutation_importance(
80             fitted,
81             x_valid,
82             y_valid,
83             scoring="neg_mean_squared_error",
84             n_repeats=12,

```

```

85     random_state=int(p1["seed"]),
86 )
87 order = np.argsort(imp.importances_mean[::-1][:12])
88 original_prediction = _predict(model_name, fitted, x_valid)
89 rng = np.random.default_rng(int(p1["seed"]))
90 importance_rows = []
91 for index in order:
92     feature = str(x_valid.columns[index])
93     shuffled = x_valid.copy()
94     shuffled[feature] = rng.permutation(shuffled[feature].to_numpy())
95     shuffled_prediction = _predict(model_name, fitted, shuffled)
96     effect = original_prediction - shuffled_prediction
97     series = x_valid[feature]
98     if pd.api.types.is_numeric_dtype(series):
99         feature_value = series.rank(pct=True).fillna(0.5).to_numpy(float)
100     else:
101         codes = pd.Categorical(series.astype(str)).codes.astype(float)
102         feature_value = codes / max(float(codes.max()), 1.0)
103     importance_rows.append({
104         "feature": feature,
105         "delta_mse": float(imp.importances_mean[index]),
106         "delta_mse_std": float(imp.importances_std[index]),
107         "effect": effect.astype(float).tolist(),
108         "feature_value": feature_value.astype(float).tolist(),
109     })
110
111 return {
112     "target_million": y.tolist(),
113     "stage": x_train["stage"].astype(str).tolist(),
114     "timezone_pair": x_train["timezone_pair"].astype(str).tolist(),
115     "pairplot": {k: np.asarray(v, float).tolist() for k, v in pair.items()},
116     "fold_metrics": p1["fold_metrics"],
117     "model_summary": p1["model_summary"],
118     "oof": p1["oof"],
119     "importance": importance_rows,
120     "selected_model": model_name,
121 }
122
123
124 def q2_constraint_rows(sheets, p2):
125     schedule = pd.DataFrame(p2["schedule"])
126     venues = sheets["venues"].set_index("venue_id")
127     slots = sheets["time_slots"].set_index("slot_id")
128     limits = sheets["dynamic_resource_limits"].copy()
129     limits["reference_date"] = limits["reference_date"].astype(str)
130     limit_map = limits.set_index("reference_date")
131
132     loads = schedule.groupby("venue_id").size()
133     lower_slacks = [loads[v] - int(venues.loc[v, "min_total_matches"]) for v in loads.index]
134     upper_slacks = [int(venues.loc[v, "max_total_matches"]) - loads[v] for v in loads.index]
135     local_day_use = schedule.groupby(["venue_id", schedule["local_datetime"].str[:10]]).size()
136     local_day_ratios = [
137         float(use) / max(float(venues.loc[venue_id, "max_matches_per_day"]), 1.0)
138         for (venue_id, _), use in local_day_use.items()
139     ]
140     local_day_max_ratio = max(local_day_ratios)
141     security_slack = (schedule["venue_security_level"] - schedule["required_security_level"]).min()
142     slot_use = schedule.groupby("slot_id").size()
143     broadcast_slacks = [int(slots.loc[s, "broadcast_capacity"]) - slot_use[s] for s in slot_use.index]
144     high = schedule[(schedule["round_in_group"] == 3) & (schedule["required_security_level"] >= 3)]
145     ratios = []
146     for day, rows in high.groupby("reference_date"):
147         cap = int(limit_map.loc[str(day), "high_security_capacity"])
148         ratios.append((cap - len(rows)) / max(cap, 1))
149     prime_margin = (2 - int(p2["metrics"]["prime_count_range"])) / 2
150
151 return [
152     {"name": "最小轮间休息", "actual": float(p2["constraint_audit"]["min_rest_hours"]), "limit": 60.0,
153      "unit": "小时", "margin_pct": 100 * (float(p2["constraint_audit"]["min_rest_hours"]) - 60.0) / 60.0},
154     {"name": "场馆承办下界", "actual": float(min(lower_slacks)), "limit": 0.0,
155      "unit": "剩余场", "margin_pct": 100 * min(lower_slacks) / max(float(venues["max_total_matches"].max()), 1.0)},
156     {"name": "场馆承办上界", "actual": float(min(upper_slacks)), "limit": 0.0,
157      "unit": "剩余场", "margin_pct": 100 * min(upper_slacks) / max(float(venues["max_total_matches"].max()), 1.0)},
158     {"name": "场馆单日本地容量", "actual": float(local_day_max_ratio), "limit": 1.0,
159      "unit": "最大利用率", "margin_pct": 100 * (1.0 - local_day_max_ratio)},
160     {"name": "场馆安保等级", "actual": float(security_slack), "limit": 0.0,

```

```

161     "unit": "等级余量", "margin_pct": 100 * security_slack / max(float(venues["security_level"].max()), 1.0)},
162     {"name": "时段转播容量", "actual": float(min(broadcast_slacks)), "limit": 0.0,
163     "unit": "剩余场", "margin_pct": 100 * min(broadcast_slacks) / max(float(slots["broadcast_capacity"].max()), 1.0)},
164     {"name": "第三轮高安保日容量", "actual": float(min(ratios) if ratios else 1.0), "limit": 0.0,
165     "unit": "比例余量", "margin_pct": 100 * (min(ratios) if ratios else 1.0)},
166     {"name": "黄金时段公平偏差", "actual": float(p2["metrics"]["prime_count_range"]), "limit": 2.0,
167     "unit": "场", "margin_pct": 100 * prime_margin},
168 ]
169
170
171 def q2_data(sheets, p2, sensitivity):
172     schedule = pd.DataFrame(p2["schedule"])
173     venues = sheets["venues"].copy()
174     venue_rows = venues[["venue_id", "city", "country", "latitude", "longitude", "capacity"]].to_dict("records")
175     venue_lookup = venues.set_index("venue_id")
176
177     transitions = Counter()
178     all_teams = sorted(set(schedule["team_a_id"]) | set(schedule["team_b_id"]))
179     for team in all_teams:
180         rows = schedule[(schedule["team_a_id"] == team) | (schedule["team_b_id"] == team)].sort_values("round_in_group")
181         ids = rows["venue_id"].astype(str).tolist()
182         for left, right in zip(ids[:-1], ids[1:]):
183             if left != right:
184                 transitions[(left, right)] += 1
185     edge_rows = []
186     for (left, right), count in transitions.items():
187         edge_rows.append({
188             "source": left,
189             "target": right,
190             "count": count,
191             "source_lon": float(venue_lookup.loc[left, "longitude"]),
192             "source_lat": float(venue_lookup.loc[left, "latitude"]),
193             "target_lon": float(venue_lookup.loc[right, "longitude"]),
194             "target_lat": float(venue_lookup.loc[right, "latitude"]),
195         })
196
197     weights = {"T": 0.25, "B": 0.25, "U": 0.15, "H": 0.10,
198               "C": -0.08, "D": -0.07, "F": -0.06, "R": -0.04}
199     objective = [
200         {"indicator": key, "value": float(weights[key] * p2["metrics"][key]),
201         "kind": "固定项" if key in {"U", "H"} else "决策项"}
202         for key in weights
203     ]
204     return {
205         "schedule": p2["schedule"],
206         "venues": venue_rows,
207         "venue_load": p2["venue_load"],
208         "travel_edges": edge_rows,
209         "constraint_margins": q2_constraint_rows(sheets, p2),
210         "objective_contributions": objective,
211         "objective_total": float(p2["metrics"]["Z2"]),
212         "weight_surface": sensitivity["p2_weight_surface"],
213         "time_slots": sheets["time_slots"][["slot_id", "date", "reference_kickoff_time", "broadcast_capacity"]].to_dict("records"),
214     }
215
216
217 def q3_baseline_advancement(sheets, p3):
218     groups = sheets["groups_matches"]
219     base = groups.merge(
220         sheets["base_predictions"],
221         on=["match_id", "group_id", "round_in_group", "team_a", "team_b"],
222         validate="one_to_one",
223     )
224     round3 = base[base["round_in_group"].eq(3)].reset_index(drop=True)
225     teams = sheets["group_membership"]["team_name"].astype(str).tolist()
226     snapshot = team_snapshot(sheets["live_group_results"], teams)
227     baseline = simulate_qualification(
228         round3,
229         sheets["group_membership"],
230         snapshot,
231         round3["expected_goals_a"].to_numpy(float),
232         round3["expected_goals_b"].to_numpy(float),
233         int(p3["seed"]),
234         keep_scores=False,
235     )
236     updated = {row["team"]: row["probability"] for row in p3["team_advancement"]}

```

```

237     return [
238         {"team": team, "baseline": float(baseline["probability"][i]), "updated": float(updated[team])}
239         for i, team in enumerate(baseline["teams"])
240     ]
241
242
243 def q3_data(sheets, p3, sensitivity):
244     transitions = []
245     resource_names = {"b": "转播", "q": "安保", "l": "交通"}
246     for resource, label in resource_names.items():
247         counts = Counter()
248         for row in p3["rows"]:
249             counts[(int(row["static_decision"][resource]), int(row["dynamic_decision"][resource]))] += 1
250         transitions.extend([
251             {"resource": label, "source": left, "target": right, "count": count}
252             for (left, right), count in sorted(counts.items())
253         ])
254
255     bubble_rows = []
256     for row in p3["rows"]:
257         importance = 0.5 * (
258             4 * row["updated_p_team_a_advance"] * (1 - row["updated_p_team_a_advance"])
259             + 4 * row["updated_p_team_b_advance"] * (1 - row["updated_p_team_b_advance"])
260         )
261         bubble_rows.append({
262             "match_id": row["match_id"],
263             "risk": float(row["risk_exposure_index"]),
264             "commercial_musd": float((row["updated_ticket_revenue_usd"] + row["updated_broadcast_value_usd"]) / 1_000_000),
265             "importance": float(importance),
266             "resource_cost": float(row["resource_cost_index"]),
267         })
268     return {
269         "advancement_shift": q3_baseline_advancement(sheets, p3),
270         "resource_transitions": transitions,
271         "rows": p3["rows"],
272         "risk_value": bubble_rows,
273         "sensitivity_surface": sensitivity["p3_state_injury_surface"],
274         "daily_static_audit": p3["daily_static_audit"],
275         "daily_dynamic_audit": p3["daily_dynamic_audit"],
276         "action_table": p3["action_table"],
277         "Z3_static": p3["Z3_static_reeval"],
278         "Z3_dynamic": p3["Z3_dynamic"],
279     }
280
281
282 def q4_data(p4):
283     proxy_metrics = [row for row in p4["proxy_comparison"] if row["indicator_name"] != "Z4"]
284     structural_metrics = [row for row in p4["structural_comparison"] if row["indicator_name"] != "timezone_crossings"]
285     differences = []
286     radar = []
287     for row in structural_metrics:
288         actual = float(row["actual_schedule_value"])
289         optimized = float(row["optimized_schedule_value"])
290         higher = row["preferred_direction"] == "higher"
291         denominator = max(abs(actual), abs(optimized), 1e-12)
292         preferred_change = (optimized - actual if higher else actual - optimized) / denominator
293         differences.append({**row, "preferred_change": float(preferred_change)})
294     for row in proxy_metrics:
295         actual = float(row["actual_schedule_value"])
296         optimized = float(row["optimized_schedule_value"])
297         higher = row["preferred_direction"] == "higher"
298         # P4 已在共同边界上把八项代理指标归一到 [0,1]; 雷达图直接使用该共同尺度,
299         # 成本/旅行/公平/风险只做方向反转, 不再逐指标二次缩放。
300         if higher:
301             actual_score, optimized_score = actual, optimized
302         else:
303             actual_score, optimized_score = 1-actual, 1-optimized
304         radar.append({"indicator": row["indicator_name"], "actual": float(actual_score), "optimized": float(optimized_score)})
305     return {
306         "differences": differences,
307         "radar": radar,
308         "comparison": p4["comparison"],
309         "weight_robustness": p4["weight_robustness"],
310         "proxy_note": p4["proxy_note"],
311     }
312

```

```

313
314 def main():
315     print("[prep] loading source workbook and persisted results", flush=True)
316     sheets = load_workbook()
317     p1 = read_json("problem_1_results.json")
318     p2 = read_json("problem_2_results.json")
319     p3 = read_json("problem_3_results.json")
320     p4 = read_json("problem_4_results.json")
321     sensitivity = read_json("sensitivity_results.json")
322     output = {
323         "metadata": {
324             "source_results": [
325                 "problem_1_results.json", "problem_2_results.json", "problem_3_results.json",
326                 "problem_4_results.json", "sensitivity_results.json",
327             ],
328             "seed": p1["seed"],
329             "baseline_advancement_definition": "same observed two-round snapshot, pre-update round-three scoring rates",
330         },
331         "q1": q1_data(sheets, p1),
332         "q2": q2_data(sheets, p2, sensitivity),
333         "q3": q3_data(sheets, p3, sensitivity),
334         "q4": q4_data(p4),
335     }
336     target = ROOT / "figures" / "_plot_data.json"
337     target.write_text(json.dumps(jsonable(output), ensure_ascii=False, indent=2), encoding="utf-8")
338     print(f"[prep] wrote {target} ({target.stat().st_size} bytes)", flush=True)
339
340
341 if __name__ == "__main__":
342     main()

```

Listing 14: 统一绘图实现

```

1  """Recipe-adapted publication plots for the CUMCM C results.
2
3  The twenty tiny ``gen_fig.py`` entry points call the functions in this module.
4  All numerical content is read from ``figures/_plot_data.json``.
5  """
6  from __future__ import annotations
7
8  import json
9  import math
10 from collections import Counter, defaultdict
11 from pathlib import Path
12
13 import matplotlib.dates as mdates
14 import matplotlib.patches as patches
15 import matplotlib.pyplot as plt
16 import numpy as np
17 import pandas as pd
18 from matplotlib.colors import LinearSegmentedColormap, Normalize
19 from matplotlib.lines import Line2D
20 from matplotlib.path import Path as MplPath
21 from matplotlib.patches import PathPatch
22 from matplotlib.ticker import FormatStrFormatter
23 from scipy import stats
24 from scipy.stats import gaussian_kde
25
26 import _utils.plot_utils as pu
27
28 pu.setup_style("nature")
29 ROOT = Path(__file__).resolve().parents[1]
30 DATA_PATH = ROOT / "figures" / "_plot_data.json"
31
32 FEATURE_ZH = {
33     "competition": "赛事类别",
34     "stage": "赛事阶段",
35     "timezone_pair": "时区组合",
36     "strength_rank_mean": "平均实力排名",
37     "strength_rank_gap": "实力排名差",
38     "elo_mean": "平均Elo",
39     "elo_gap": "Elo差",
40     "p_a": "A队胜率",
41     "p_draw": "平局概率",

```

```

42     "p_b": "B队胜率",
43     "odds_entropy": "赔率熵",
44     "market_value_musd_mean": "平均身价",
45     "market_value_musd_gap": "身价差",
46     "avg_age_mean": "平均年龄",
47     "avg_age_gap": "年龄差",
48     "star_index_mean": "平均球星指数",
49     "star_index_gap": "球星指数差",
50     "fan_base_index_mean": "平均球迷基础",
51     "fan_base_index_gap": "球迷基础差",
52     "style_attack_mean": "平均进攻风格",
53     "style_attack_gap": "进攻风格差",
54     "style_defense_mean": "平均防守风格",
55     "style_defense_gap": "防守风格差",
56     "host_flag_mean": "东道主比例",
57     "host_flag_gap": "东道主差异",
58     "strength_score_mean": "平均实力得分",
59     "strength_score_gap": "实力得分差",
60     "worldcup_stage": "世界杯标记",
61     "fan_stage_interaction": "球迷基础-世界杯交互",
62     "star_stage_interaction": "球星-世界杯交互",
63     "strength_entropy_interaction": "实力-赔率熵交互",
64 }
65 METRIC_ZH = {
66     "T": "票务价值T", "B": "传播价值B", "U": "不确定性U", "H": "吸引力H",
67     "C": "组织成本C", "D": "旅行负担D", "F": "公平惩罚F", "R": "执行风险R",
68     "rest_min_hours": "最小休息", "rest_mean_hours": "平均休息",
69     "timezone_crossing_rate": "跨时区率", "travel_mean_km": "场馆旅行",
70     "venue_change_rate": "换馆率", "venue_utilization_rate": "场馆日利用率",
71     "venue_daily_peak": "场馆日峰值", "capacity_occupancy_rate": "容量匹配",
72     "prime_coverage_rate": "黄金覆盖", "expected_attendance_per_match": "场均观众",
73 }
74
75
76 def _data():
77     return json.loads(DATA_PATH.read_text(encoding="utf-8"))
78
79
80 def _output(name: str):
81     return str(ROOT / f"figures" / f"{name}.pdf")
82
83
84 def _seq_cmap():
85     return LinearSegmentedColormap.from_list(
86         "nature_sequence", [pu.COLORS["light"], pu.PALETTE[1], pu.PALETTE[0]]
87     )
88
89
90 def _panel_label(ax, label):
91     ax.set_title(label, loc="left", pad=3, fontsize=10, fontweight="bold")
92
93
94 def fig_q1_target_raincloud(d):
95     values = np.asarray(d["q1"]["target_million"], float)
96     rng = np.random.default_rng(int(d["metadata"]["seed"]))
97     fig, ax = plt.subplots(figsize=(8.2, 3.6))
98     parts = ax.violinplot(values, positions=[0.15], vert=False, widths=0.6,
99                          showmeans=False, showmedians=False, showextrema=False)
100     for body in parts["bodies"]:
101         vertices = body.get_paths()[0].vertices
102         vertices[:, 1] = np.maximum(vertices[:, 1], 1), 0.15)
103         body.set_facecolor(pu.PALETTE[0])
104         body.set_edgecolor(pu.PALETTE[0])
105         body.set_alpha(0.30)
106         body.set_linewidth(1.2)
107     jitter = -0.18 + rng.normal(0, 0.025, len(values))
108     ax.scatter(values, jitter, s=10, color=pu.PALETTE[1], alpha=0.34,
109               edgecolor=pu.COLORS["light"], linewidth=0.25, rasterized=True)
110     ax.boxplot(values, positions=[-0.02], vert=False, widths=0.12, patch_artist=True,
111               showfliers=False,
112               boxprops={"facecolor": pu.COLORS["light"], "edgecolor": pu.PALETTE[0], "linewidth": 1.2},
113               whiskerprops={"color": pu.PALETTE[0]}, capprops={"color": pu.PALETTE[0]},
114               medianprops={"color": pu.PALETTE[3], "linewidth": 1.6})
115     mean = float(values.mean())
116     ax.scatter([mean], [-0.02], marker="D", s=46, color=pu.PALETTE[3],
117               edgecolor=pu.COLORS["light"], linewidth=0.7, zorder=5)

```

```

118     ax.annotate(f"均值 {mean:.1f}", xy=(mean, -0.02), xytext=(8, 16),
119               textcoords="offset points", fontsize=8, color=pu.COLORS["text"],
120               arrowprops={"arrowstyle": "-", "color": pu.COLORS["ref_line"], "lw": 0.8})
121     ax.set_xlabel("观看人数 (百万人)")
122     ax.set_yticks([])
123     ax.set_ylim(-0.34, 0.52)
124     ax.set_xlim(values.min() - 5, values.max() + 5)
125     pu.save_fig(fig, _output("fig_q1_target_raincloud"))
126
127
128 def fig_q1_feature_pairplot(d):
129     frame = pd.DataFrame(d["q1"]["pairplot"])
130     cols = list(frame.columns)
131     n = len(cols)
132     fig, axes = plt.subplots(n, n, figsize=(8, 8), squeeze=False)
133     cmap = _seq_cmap()
134     for i, ycol in enumerate(cols):
135         for j, xcol in enumerate(cols):
136             ax = axes[i, j]
137             x = frame[xcol].to_numpy(float)
138             y = frame[ycol].to_numpy(float)
139             if i == j:
140                 ax.hist(x, bins=22, density=True, color=pu.PALETTE[i % len(pu.PALETTE)],
141                       alpha=0.35, edgecolor=pu.COLORS["light"], linewidth=0.4)
142                 if np.std(x) > 1e-12:
143                     grid = np.linspace(x.min(), x.max(), 160)
144                     kde = gaussian_kde(x)
145                     ax.plot(grid, kde(grid), color=pu.PALETTE[i % len(pu.PALETTE)], lw=1.1)
146                 ax.set_yticks([])
147             elif i > j:
148                 ax.scatter(x, y, s=6, color=pu.PALETTE[0], alpha=0.22, edgecolor="none", rasterized=True)
149                 if np.std(x) > 1e-12:
150                     coef = np.polyfit(x, y, 1)
151                     grid = np.linspace(x.min(), x.max(), 80)
152                     ax.plot(grid, np.polyval(coef, grid), color=pu.PALETTE[1], lw=0.9)
153             else:
154                 corr = float(np.corrcoef(x, y)[0, 1])
155                 background_score = (corr + 1) / 2
156                 ax.set_facecolor(cmap(background_score))
157                 ax.text(0.5, 0.53, f"r={corr:.2f}", ha="center", va="center",
158                       transform=ax.transAxes, fontsize=9, fontweight="bold",
159                       color=pu.COLORS["light"] if background_score > 0.52 else pu.COLORS["text"])
160                 ax.text(0.5, 0.32, "正相关" if corr >= 0 else "负相关", ha="center", va="center",
161                       transform=ax.transAxes, fontsize=6.5,
162                       color=pu.COLORS["light"] if background_score > 0.52 else pu.COLORS["text"])
163                 ax.set_xticks([]); ax.set_yticks([])
164             if i == n - 1:
165                 ax.set_xlabel(xcol, fontsize=7.5)
166                 ax.tick_params(axis="x", labelsize=6)
167             else:
168                 ax.set_xticklabels([])
169             if j == 0:
170                 ax.set_ylabel(ycol, fontsize=7.5)
171                 ax.tick_params(axis="y", labelsize=6)
172             else:
173                 ax.set_yticklabels([])
174     fig.subplots_adjust(wspace=0.08, hspace=0.08)
175     pu.save_fig(fig, _output("fig_q1_feature_pairplot"))
176
177
178 def _violin_panel(ax, labels, values, max_groups=6):
179     counts = Counter(labels)
180     chosen = [name for name, _ in counts.most_common(max_groups)]
181     data = [np.asarray([v for label, v in zip(labels, values) if label == name], float) for name in chosen]
182     positions = np.arange(len(chosen))
183     parts = ax.violinplot(data, positions=positions, widths=0.82, showmeans=False,
184                          showmedians=False, showextrema=False)
185     for i, body in enumerate(parts["bodies"]):
186         body.set_facecolor(pu.PALETTE[i % len(pu.PALETTE)])
187         body.set_edgecolor(pu.PALETTE[i % len(pu.PALETTE)])
188         body.set_alpha(0.30)
189         body.set_linewidth(1.1)
190     for i, arr in enumerate(data):
191         q1, med, q3 = np.quantile(arr, [0.25, 0.5, 0.75])
192         ax.vlines(i, q1, q3, color=pu.PALETTE[i % len(pu.PALETTE)], lw=3.5, zorder=4)
193         ax.scatter([i], [med], s=25, color=pu.COLORS["text"], edgecolor=pu.COLORS["light"], zorder=5)

```

```

194     return chosen
195
196
197 def fig_q1_stage_timezone_violin(d):
198     values = np.asarray(d["q1"]["target_million"], float)
199     stages = d["q1"]["stage"]
200     zones = [z.replace("|", "/") for z in d["q1"]["timezone_pair"]]
201     fig, axes = plt.subplots(1, 2, figsize=(11, 4.5), sharey=True)
202     stage_labels = _violin_panel(axes[0], stages, values, 6)
203     zone_labels = _violin_panel(axes[1], zones, values, 6)
204     _panel_label(axes[0], "(a)")
205     _panel_label(axes[1], "(b)")
206     axes[0].set_xticks(range(len(stage_labels)), stage_labels, rotation=25, ha="right")
207     axes[1].set_xticks(range(len(zone_labels)), zone_labels, rotation=25, ha="right")
208     axes[0].set_ylabel("观看人数（百万人）")
209     axes[0].set_xlabel("赛事阶段（按样本量前6类）")
210     axes[1].set_xlabel("主队/客队时区组合（按样本量前6类）")
211     pu.save_fig(fig, _output("fig_q1_stage_timezone_violin"))
212
213
214 def fig_q1_model_accuracy_heatmap(d):
215     rows = d["q1"]["fold_metrics"]
216     models = sorted([r["model"] for r in rows], key=lambda x: ["MeanBaseline", "ElasticNet", "ExtraTrees", "CatBoost"].index(x))
217     summary = {r["model"]: r for r in d["q1"]["model_summary"]}
218     labels = [f"折{i} MSE" for i in range(1, 6)] + [f"平均RMSE", "平均MAE"]
219     values = np.zeros((len(models), len(labels)))
220     for i, model in enumerate(models):
221         by_fold = {int(r["fold"]): r for r in rows if r["model"] == model}
222         values[i, :5] = [by_fold[k]["mse"] for k in range(1, 6)]
223         values[i, 5] = summary[model]["mean_rmse"]
224         values[i, 6] = summary[model]["mean_mae"]
225     scores = np.zeros_like(values)
226     for j in range(values.shape[1]):
227         lo, hi = values[:, j].min(), values[:, j].max()
228         scores[:, j] = 1 - (values[:, j] - lo) / max(hi - lo, 1e-12)
229     display_models = {"MeanBaseline": "均值", "ElasticNet": "弹性网", "ExtraTrees": "极端树", "CatBoost": "提升树"}
230     fig, ax = plt.subplots(figsize=(10.0, 4.9))
231     image = ax.imshow(scores, cmap=seq_cmap(), vmin=0, vmax=1, aspect="auto")
232     for i in range(values.shape[0]):
233         for j in range(values.shape[1]):
234             color = pu.COLORS["light"] if scores[i, j] > 0.63 else pu.COLORS["text"]
235             text = f"{values[i, j]:.1f}" if j < 5 else f"{values[i, j]:.2f}"
236             weight = "bold" if values[i, j] == values[:, j].min() else "normal"
237             ax.text(j, i, text, ha="center", va="center", fontsize=7.5, color=color, fontweight=weight)
238     selected = d["q1"]["selected_model"]
239     selected_i = models.index(selected)
240     ax.add_patch(patches.Rectangle((-0.49, selected_i - 0.49), len(labels) - 0.02, 0.98,
241                                   fill=False, edgecolor=pu.PALETTE[3], linewidth=1.8))
242     ax.set_xticks(range(len(labels)), labels, rotation=22, ha="right")
243     ax.set_yticks(range(len(models)), [display_models[m] for m in models])
244     ax.set_xlabel("时序验证误差指标（颜色越深表示列内相对越优）", labelpad=8)
245     ax.xaxis.set_label_position("top")
246     ax.xaxis.label.set_size(8)
247     ax.set_ylabel("")
248     cbar = fig.colorbar(image, ax=ax, fraction=0.03, pad=0.02)
249     cbar.ax.set_title("相对得分", fontsize=7, pad=3)
250     fig.subplots_adjust(left=0.15, right=0.90, bottom=0.24, top=0.96)
251     pu.save_fig(fig, _output("fig_q1_model_accuracy_heatmap"))
252
253
254 def fig_q1_prediction_fit(d):
255     oof = d["q1"]["oof"]
256     actual = np.asarray(oof["actual_million"], float)
257     predicted = np.asarray(oof["predicted_million"], float)
258     residual = actual - predicted
259     r2 = 1 - np.sum(residual ** 2) / np.sum((actual - actual.mean()) ** 2)
260     rmse = float(np.sqrt(np.mean(residual ** 2)))
261     fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.4), gridspec_kw={"width_ratios": [1.25, 1]})
262     ax = axes[0]
263     ax.scatter(actual, predicted, s=18, color=pu.PALETTE[0], alpha=0.50,
264               edgecolor=pu.COLORS["light"], linewidth=0.35, rasterized=True)
265     lo = min(actual.min(), predicted.min()); hi = max(actual.max(), predicted.max())
266     ax.plot([lo, hi], [lo, hi], color=pu.COLORS["ref_line"], lw=1.2, linestyle="--", label="45度参考线")
267     ax.set_xlim(lo - 4, hi + 4); ax.set_ylim(lo - 4, hi + 4)
268     ax.set_xlabel("真实观看人数（百万人）"); ax.set_ylabel("预测观看人数（百万人）")
269     ax.text(0.05, 0.92, f"$R^2$={r2:.3f}\nRMSE={rmse:.2f} 百万", transform=ax.transAxes,

```

```

270         va="top", fontsize=8.5, color=pu.COLORS["text"],
271         bbox={"boxstyle": "round,pad=0.35", "facecolor": pu.COLORS["bg_box"], "edgecolor": pu.PALETTE[0], "alpha": 0.9})
272     threshold = np.quantile(actual, 0.82)
273     inset = ax.inset_axes([0.57, 0.08, 0.38, 0.35])
274     mask = actual >= threshold
275     inset.scatter(actual[mask], predicted[mask], s=12, color=pu.PALETTE[1], alpha=0.65, edgecolor="none")
276     inset.plot([actual[mask].min(), actual[mask].max()], [actual[mask].min(), actual[mask].max()],
277               color=pu.COLORS["ref_line"], lw=0.8, linestyle="--")
278     inset.tick_params(labelsize=5.5)
279     inset.set_xlabel("离观者区真实值", fontsize=6)
280     inset.set_ylabel("预测值", fontsize=6)
281     _panel_label(ax, "(a)")
282
283     ax = axes[1]
284     ax.hist(residual, bins=22, density=True, color=pu.PALETTE[1], alpha=0.32,
285            edgecolor=pu.PALETTE[1], linewidth=0.6)
286     grid = np.linspace(residual.min(), residual.max(), 220)
287     kde = gaussian_kde(residual)
288     ax.fill_between(grid, kde(grid), color=pu.PALETTE[0], alpha=0.18)
289     ax.plot(grid, kde(grid), color=pu.PALETTE[0], lw=1.4, label="核密度")
290     ax.axvline(0, color=pu.COLORS["ref_line"], linestyle="--", lw=1.0)
291     ax.set_xlabel("残差 (百万人)"); ax.set_ylabel("概率密度")
292     ax.legend(loc="best")
293     _panel_label(ax, "(b)")
294     pu.save_fig(fig, _output("fig_q1_prediction_fit"))
295
296
297 def fig_q1_residual_diagnostics(d):
298     oof = d["q1"]["oof"]
299     fitted = np.asarray(oof["predicted_million"], float)
300     residual = np.asarray(oof["residual_million"], float)
301     dates = pd.to_datetime(oof["date"], utc=True)
302     fig, axes = plt.subplots(2, 2, figsize=(10, 7.4))
303     axes[0, 0].scatter(fitted, residual, s=13, color=pu.PALETTE[0], alpha=0.45, edgecolor="none", rasterized=True)
304     coef = np.polyfit(fitted, residual, 2)
305     grid = np.linspace(fitted.min(), fitted.max(), 160)
306     axes[0, 0].plot(grid, np.polyval(coef, grid), color=pu.PALETTE[1], lw=1.3)
307     axes[0, 0].axhline(0, color=pu.COLORS["ref_line"], linestyle="--", lw=0.9)
308     axes[0, 0].set_xlabel("拟合值 (百万人)"); axes[0, 0].set_ylabel("残差 (百万人)")
309
310     theoretical, ordered = stats.probplot(residual, dist="norm", fit=False)
311     axes[0, 1].scatter(theoretical, ordered, s=13, color=pu.PALETTE[2], alpha=0.55, edgecolor="none")
312     line = np.polyfit(theoretical, ordered, 1)
313     axes[0, 1].plot(theoretical, np.polyval(line, theoretical), color=pu.PALETTE[3], lw=1.1)
314     axes[0, 1].set_xlabel("理论正态分位数"); axes[0, 1].set_ylabel("样本残差分位数 (百万人)")
315
316     axes[1, 0].hist(residual, bins=24, density=True, color=pu.PALETTE[1], alpha=0.33,
317                    edgecolor=pu.PALETTE[1], linewidth=0.5)
318     xgrid = np.linspace(residual.min(), residual.max(), 200)
319     axes[1, 0].plot(xgrid, stats.norm.pdf(xgrid, residual.mean(), residual.std(ddof=1)),
320                   color=pu.PALETTE[3], lw=1.3)
321     axes[1, 0].set_xlabel("残差 (百万人)"); axes[1, 0].set_ylabel("概率密度")
322
323     axes[1, 1].plot(dates, residual, color=pu.PALETTE[0], lw=0.75, alpha=0.65)
324     axes[1, 1].scatter(dates, residual, s=8, color=pu.PALETTE[0], alpha=0.42, edgecolor="none")
325     axes[1, 1].axhline(0, color=pu.COLORS["ref_line"], linestyle="--", lw=0.9)
326     axes[1, 1].axis.set_major_locator(mdates.AutoDateLocator(minticks=4, maxticks=7))
327     axes[1, 1].axis.set_major_formatter(mdates.DateFormatter("%Y"))
328     axes[1, 1].set_xlabel("验证样本比赛日期"); axes[1, 1].set_ylabel("残差 (百万人)")
329     for i, ax in enumerate(axes.flat):
330         _panel_label(ax, f"({chr(97 + i)})")
331     pu.save_fig(fig, _output("fig_q1_residual_diagnostics"))
332
333
334 def fig_q1_shap_summary(d):
335     rows = d["q1"]["importance"]
336     if not rows:
337         raise RuntimeError("No permutation importance data available")
338     rows = sorted(rows, key=lambda row: row["delta_mse"])
339     bar_rows = rows[-12:]
340     swarm_rows = bar_rows[-8:]
341     fig, axes = plt.subplots(1, 2, figsize=(11, 5.4), gridspec_kw={"width_ratios": [0.9, 1.35]})
342     labels = [FEATURE_ZH.get(row["feature"], row["feature"]) for row in bar_rows]
343     values = np.asarray([row["delta_mse"] for row in bar_rows], float)
344     colors = [pu.PALETTE[(i + 1) % len(pu.PALETTE)] for i in range(len(values))]
345     bars = axes[0].barh(np.arange(len(values)), values, color=colors, alpha=0.30,

```

```

346         edgecolor=colors, linewidth=1.1)
347     for bar, value in zip(bars, values):
348         axes[0].text(max(value, 0) + max(values.max(), 1e-6) * 0.02,
349                     bar.get_y() + bar.get_height() / 2, f"{value:.3f}",
350                     va="center", fontsize=7, color=pu.COLORS["text"])
351     axes[0].set_yticks(range(len(labels)), labels)
352     axes[0].set_xlabel("置换重要性 (验证集MSE增量)")
353     _panel_label(axes[0], "(a)")
354
355     rng = np.random.default_rng(int(d["metadata"]["seed"]))
356     for yidx, row in enumerate(swarm_rows):
357         effect = np.asarray(row["effect"], float)
358         feature_value = np.asarray(row["feature_value"], float)
359         order = np.argsort(np.abs(effect))[:-1][:100]
360         jitter = rng.normal(0, 0.09, len(order))
361         axes[1].scatter(effect[order], yidx + jitter, c=feature_value[order], cmap="coolwarm",
362                         vmin=0, vmax=1, s=13, alpha=0.62,
363                         edgecolor=pu.COLORS["light"], linewidth=0.2, rasterized=True)
364     axes[1].axvline(0, color=pu.COLORS["ref_line"], linestyle="--", lw=0.9)
365     axes[1].set_yticks(range(len(swarm_rows)), [FEATURE_ZH.get(r["feature"], r["feature"]) for r in swarm_rows])
366     axes[1].set_xlabel("单样本置换预测影响 (百万人)")
367     _panel_label(axes[1], "(b)")
368     legend_handles = [
369         Line2D([0], [0], marker="o", linestyle="none", color=pu.PALETTE[0], label="特征值较低", markersize=5),
370         Line2D([0], [0], marker="o", linestyle="none", color=pu.PALETTE[1], label="特征值较高", markersize=5),
371     ]
372     axes[1].legend(handles=legend_handles, loc="best", frameon=True)
373     pu.save_fig(fig, _output("fig_q1_shap_summary"))
374
375
376 def fig_q2_schedule_gantt(d):
377     frame = pd.DataFrame(d["q2"]["schedule"])
378     frame["utc"] = pd.to_datetime(frame["utc_datetime"], utc=True).dt.tz_convert(None)
379     venues = sorted(frame["venue_id"].unique())
380     ypos = {venue: i for i, venue in enumerate(venues)}
381     fig = plt.figure(figsize=(11, 6.8))
382     layout = fig.add_gridspec(1, 2, width_ratios=[18, 2], wspace=0.03)
383     ax = fig.add_subplot(layout[0, 0])
384     legend_ax = fig.add_subplot(layout[0, 1])
385     for row in frame.itertuples(index=False):
386         start = mdates.date2num(row.utc.to_pydatetime())
387         color = pu.PALETTE[(int(row.round_in_group) - 1) % len(pu.PALETTE)]
388         ax.barh(ypos[row.venue_id], 2 / 24, left=start, height=0.55,
389               color=color, alpha=0.42, edgecolor=color, linewidth=0.8)
390     ax.set_yticks(range(len(venues)), venues)
391     ax.xaxis_date()
392     ax.xaxis.set_major_locator(mdates.DayLocator(interval=2))
393     ax.xaxis.set_major_formatter(mdates.DateFormatter("%m-%d"))
394     ax.set_xlabel("比赛开球时间 (UTC, 2026年)")
395     ax.set_ylabel("场馆编号")
396     handles = [patches.Patch(facecolor=pu.PALETTE[i], edgecolor=pu.PALETTE[i], alpha=0.42, label=f"小组第{i + 1}轮") for i in range(3)]
397     legend_ax.legend(handles=handles, loc="center", frameon=True, fontsize=8)
398     legend_ax.set_xticks([]); legend_ax.set_yticks([])
399     for spine in legend_ax.spines.values():
400         spine.set_visible(False)
401     pu.save_fig(fig, _output("fig_q2_schedule_gantt"))
402
403
404 def fig_q2_venue_travel_map(d):
405     venues = pd.DataFrame(d["q2"]["venues"])
406     edges = sorted(d["q2"]["travel_edges"], key=lambda row: row["count"])
407     loads = d["q2"]["venue_load"]
408     fig, ax = plt.subplots(figsize=(9.8, 5.7))
409     ax.set_facecolor(pu.COLORS["bg_box"])
410     for edge in edges:
411         ax.plot([edge["source_lon"], edge["target_lon"]], [edge["source_lat"], edge["target_lat"]],
412               color=pu.PALETTE[1], alpha=0.12 + 0.08 * edge["count"],
413               lw=0.45 + 0.55 * edge["count"], solid_capstyle="round", zorder=1)
414     sizes = np.asarray([loads[str(v)] for v in venues["venue_id"]], float)
415     ax.scatter(venues["longitude"], venues["latitude"], s=45 + 18 * sizes,
416             color=pu.PALETTE[0], alpha=0.78, edgecolor=pu.COLORS["light"], linewidth=0.9, zorder=3)
417     for row in venues.itertuples(index=False):
418         ax.annotate(str(row.venue_id), (row.longitude, row.latitude), xytext=(4, 4),
419                 textcoords="offset points", fontsize=6.5, color=pu.COLORS["text"], zorder=4,
420                 bbox={"boxstyle": "round,pad=0.15", "facecolor": pu.COLORS["light"], "edgecolor": pu.COLORS["grid"], "alpha": 0.85})
421     ax.text(-127, 51.0, "加拿大", color=pu.COLORS["ref_line"], fontsize=9)

```

```

422 ax.text(-108, 31.0, "墨西哥", color=pu.COLORS["ref_line"], fontsize=9)
423 ax.text(-101, 42.5, "美国", color=pu.COLORS["ref_line"], fontsize=10)
424 ax.set_xlim(-130, -70); ax.set_ylim(18, 56)
425 ax.set_xlabel("经度 (度)"); ax.set_ylabel("纬度 (度)")
426 ax.text(0.02, 0.04, "节点面积: 承办场次; 连线宽度: 球队跨轮移动次数",
427       transform=ax.transAxes, fontsize=7.5, color=pu.COLORS["text"],
428       bbox={"boxstyle": "round,pad=0.3", "facecolor": pu.COLORS["light"], "edgecolor": pu.PALETTE[0], "alpha": 0.9})
429 pu.save_fig(fig, _output("fig_q2_venue_travel_map"))
430
431
432 def fig_q2_venue_slot_matrix(d):
433     frame = pd.DataFrame(d["q2"]["schedule"])
434     dates = sorted(frame["reference_date"].astype(str).unique())
435     venues = sorted(frame["venue_id"].astype(str).unique())
436     matrix = np.zeros((len(venues), len(dates)), int)
437     vi = {v: i for i, v in enumerate(venues)}; di = {date: i for i, date in enumerate(dates)}
438     for row in frame.itertuples(index=False):
439         matrix[vi[str(row.venue_id)], di[str(row.reference_date)]] += 1
440     fig, ax = plt.subplots(figsize=(11, 5.8))
441     image = ax.imshow(matrix, cmap=seq_cmap(), vmin=0, vmax=max(matrix.max(), 1), aspect="auto")
442     for i in range(matrix.shape[0]):
443         for j in range(matrix.shape[1]):
444             if matrix[i, j] > 0:
445                 norm = matrix[i, j] / max(matrix.max(), 1)
446                 ax.text(j, i, str(matrix[i, j]), ha="center", va="center", fontsize=6.2,
447                       color=pu.COLORS["light"] if norm > 0.6 else pu.COLORS["text"])
448     ax.set_yticks(range(len(venues)), venues)
449     ax.set_xticks(range(len(dates)), [date[5:] for date in dates], rotation=40, ha="right")
450     ax.set_xlabel("参考日期 (2026年, 月-日)", labelpad=8); ax.set_ylabel("场馆编号")
451     ax.xaxis.set_label_position("top")
452     ax.xaxis.label.set_size(8)
453     cbar = fig.colorbar(image, ax=ax, fraction=0.025, pad=0.02)
454     cbar.set_label("场次")
455     fig.subplots_adjust(left=0.10, right=0.92, bottom=0.25, top=0.97)
456     pu.save_fig(fig, _output("fig_q2_venue_slot_matrix"))
457
458
459 def fig_q2_objective_waterfall(d):
460     rows = d["q2"]["objective_contributions"]
461     values = np.asarray([r["value"] for r in rows], float)
462     labels = [r["indicator"] for r in rows]
463     starts = np.r_[0.0, np.cumsum(values[:-1])]
464     fig, ax = plt.subplots(figsize=(9.2, 4.7))
465     for i, (row, value, start) in enumerate(zip(rows, values, starts)):
466         bottom = min(start, start + value)
467         color = pu.PALETTE[2] if value >= 0 else pu.PALETTE[1]
468         hatch = "/" if row["kind"] == "固定项" else None
469         ax.bar(i, abs(value), bottom=bottom, width=0.68, color=color, alpha=0.30,
470              edgecolor=color, linewidth=1.2, hatch=hatch)
471         end = start + value
472         ax.text(i, end + (0.018 if value >= 0 else -0.025), f"{value:+.3f}", ha="center",
473               va="bottom" if value >= 0 else "top", fontsize=7.2, color=pu.COLORS["text"])
474         if i < len(values) - 1:
475             ax.plot([i + 0.34, i + 0.66], [end, end], color=pu.COLORS["ref_line"], lw=0.8)
476     total = float(d["q2"]["objective_total"])
477     ax.bar(len(values), total, width=0.68, color=pu.PALETTE[0], alpha=0.35,
478          edgecolor=pu.PALETTE[0], linewidth=1.4)
479     ax.text(len(values), total + 0.018, f"{total:.3f}", ha="center", va="bottom", fontsize=8,
480           color=pu.COLORS["text"], fontweight="bold")
481     ax.axhline(0, color=pu.COLORS["ref_line"], lw=0.9)
482     ax.set_xticks(range(len(labels) + 1), labels + ["Z2"])
483     ax.set_xlabel("目标函数指标 (斜线为赛程固定项)")
484     ax.set_ylabel("对目标函数的加权贡献")
485     pu.save_fig(fig, _output("fig_q2_objective_waterfall"))
486
487
488 def fig_q2_constraint_margins_lollipop(d):
489     rows = sorted(d["q2"]["constraint_margins"], key=lambda row: row["margin_pct"])
490     labels = [row["name"] for row in rows]
491     values = np.asarray([row["margin_pct"] for row in rows], float)
492     y = np.arange(len(rows))
493     fig, ax = plt.subplots(figsize=(8.4, 5.2))
494     for yi, value in zip(y, values):
495         color = pu.PALETTE[1] if value <= 1e-10 else pu.PALETTE[0]
496         ax.hlines(yi, 0, value, color=color, lw=2.1, alpha=0.75)
497         ax.scatter(value, yi, s=55 + 2 * value, color=color, edgecolor=pu.COLORS["light"], linewidth=0.8, zorder=3)

```

```

498     ax.text(value + max(values.max(), 5) * 0.025, yi, "紧约束" if value <= 1e-10 else f"{value:.1f}%",
499             va="center", fontsize=7.5, color=pu.COLORS["text"])
500     ax.axvline(0, color=pu.COLORS["ref_line"], lw=0.9)
501     ax.set_yticks(y, labels)
502     ax.set_xlabel("最小归一化可行裕度 (%)")
503     ax.set_ylabel("约束族")
504     ax.set_xlim(-1, max(values.max() * 1.20, 8))
505     pu.save_fig(fig, _output("fig_q2_constraint_margins_lollipop"))
506
507
508 def fig_q2_weight_sensitivity_contour(d):
509     frame = pd.DataFrame(d["q2"]["weight_surface"])
510     xs = sorted(frame["commercial_multiplier"].unique())
511     ys = sorted(frame["penalty_multiplier"].unique())
512     matrix = frame.pivot(index="penalty_multiplier", columns="commercial_multiplier", values="fixed_schedule_Z2").loc[ys, xs].to_numpy(float)
513     xx, yy = np.meshgrid(xs, ys)
514     fig, ax = plt.subplots(figsize=(7.5, 5.5))
515     filled = ax.contourf(xx, yy, matrix, levels=14, cmap="YlOrRd")
516     lines = ax.contour(xx, yy, matrix, levels=8, colors=[pu.COLORS["text"]], linewidths=0.55, alpha=0.70)
517     ax.clabel(lines, inline=True, fontsize=7, fmt="%.4f")
518     index = np.unravel_index(np.argmax(matrix), matrix.shape)
519     ax.scatter([xx[index]], [yy[index]], marker="*", s=120, color=pu.PALETTE[3],
520               edgecolor=pu.COLORS["light"], linewidth=0.7, zorder=4)
521     ax.annotate(f"固定方案最高值 {matrix[index]:.4f}", xy=(xx[index], yy[index]), xytext=(-104, 25),
522               textcoords="offset points", fontsize=7.5, color=pu.COLORS["text"],
523               bbox={"boxstyle": "round,pad=0.25", "facecolor": pu.COLORS["light"], "edgecolor": pu.PALETTE[3], "alpha": 0.88},
524               arrowprops={"arrowstyle": "->", "color": pu.PALETTE[3], "lw": 0.8})
525     ax.set_xlabel("商业权重倍率", labelpad=-2)
526     ax.set_ylabel("惩罚权重倍率")
527     cbar = fig.colorbar(filled, ax=ax, fraction=0.04, pad=0.03)
528     cbar.ax.set_title("Z2", fontsize=8, pad=3)
529     ticks = np.linspace(matrix.min(), matrix.max(), 5)
530     cbar.set_ticks(ticks)
531     cbar.set_ticklabels([f"{value:.3f}" for value in ticks])
532     cbar.ax.yaxis.set_ticks_position("left")
533     cbar.ax.tick_params(labelsize=7, pad=2)
534     fig.subplots_adjust(left=0.14, right=0.88, bottom=0.17, top=0.97)
535     pu.save_fig(fig, _output("fig_q2_weight_sensitivity_contour"))
536
537
538 def fig_q3_advancement_shift(d):
539     rows = sorted(d["q3"]["advancement_shift"], key=lambda row: row["updated"] - row["baseline"])
540     y = np.arange(len(rows))
541     before = np.asarray([row["baseline"] for row in rows], float)
542     after = np.asarray([row["updated"] for row in rows], float)
543     fig, ax = plt.subplots(figsize=(8.5, 8.0))
544     changes = np.abs(after - before)
545     threshold = np.sort(changes)[-10]
546     for yi, left, right, change in zip(y, before, after, changes):
547         color = pu.PALETTE[1] if right < left else pu.PALETTE[2]
548         ax.plot([left, right], [yi, yi], color=color, lw=0.75 + 1.6 * change / max(changes.max(), 1e-12), alpha=0.65)
549     ax.scatter(before, y, s=14, color=pu.PALETTE[0], alpha=0.70, edgecolor=pu.COLORS["light"], linewidth=0.3, label="更新前基准")
550     ax.scatter(after, y, s=18, color=pu.PALETTE[1], alpha=0.78, edgecolor=pu.COLORS["light"], linewidth=0.3, label="前两轮更新后")
551     ax.set_yticks(y, [row["team"] for row in rows], fontsize=5.6)
552     ax.set_xlabel("晋级概率")
553     ax.set_ylabel("参赛队")
554     ax.set_xlim(-0.02, 1.02)
555     ax.legend(loc="best")
556     ax.text(0.02, 0.02, f"变化最大的10队阈值: {threshold:.3f}", transform=ax.transAxes,
557           fontsize=7, color=pu.COLORS["text"],
558           bbox={"boxstyle": "round,pad=0.25", "facecolor": pu.COLORS["bg_box"], "edgecolor": pu.COLORS["grid"]})
559     pu.save_fig(fig, _output("fig_q3_advancement_shift"))
560
561
562 def _sankey_panel(ax, transitions, label):
563     sources = sorted({int(row["source"]) for row in transitions})
564     targets = sorted({int(row["target"]) for row in transitions})
565     source_totals = {level: sum(row["count"] for row in transitions if int(row["source"]) == level) for level in sources}
566     target_totals = {level: sum(row["count"] for row in transitions if int(row["target"]) == level) for level in targets}
567     total = sum(source_totals.values())
568     gap = 0.035
569     usable = 0.78
570
571     def positions(levels, totals):
572         current = 0.10
573         result = {}

```

```

574     scale = (usable - gap * max(len(levels) - 1, 0)) / max(total, 1)
575     for level in levels:
576         height = totals[level] * scale
577         result[level] = [current, current + height]
578         current += height + gap
579     return result, scale
580
581 left_pos, scale = positions(sources, source_totals)
582 right_pos, _ = positions(targets, target_totals)
583 left_cursor = {k: v[0] for k, v in left_pos.items()}
584 right_cursor = {k: v[0] for k, v in right_pos.items()}
585 for row in sorted(transitions, key=lambda r: (r["source"], r["target"])):
586     source, target, count = int(row["source"]), int(row["target"]), int(row["count"])
587     h = count * scale
588     sy0, sy1 = left_cursor[source], left_cursor[source] + h
589     ty0, ty1 = right_cursor[target], right_cursor[target] + h
590     left_cursor[source] = sy1; right_cursor[target] = ty1
591     vertices = [(0.17, sy0), (0.46, sy0), (0.54, ty0), (0.83, ty0),
592                (0.83, ty1), (0.54, ty1), (0.46, sy1), (0.17, sy1), (0.17, sy0)]
593     codes = [MplPath.MOVETO, MplPath.CURVE4, MplPath.CURVE4, MplPath.CURVE4,
594             MplPath.LINE10, MplPath.CURVE4, MplPath.CURVE4, MplPath.CURVE4, MplPath.CLOSEPOLY]
595     color = pu.PALETTE[(source - 1) % len(pu.PALETTE)]
596     ax.add_patch(PathPatch(MplPath(vertices, codes), facecolor=color, edgecolor="none", alpha=0.28))
597 for level, (y0, y1) in left_pos.items():
598     color = pu.PALETTE[(level - 1) % len(pu.PALETTE)]
599     ax.add_patch(patches.Rectangle((0.12, y0), 0.05, y1 - y0, facecolor=color, edgecolor=color, alpha=0.78))
600     ax.text(0.10, (y0 + y1) / 2, f"静态{level}\n{source_totals[level]}场", ha="right", va="center", fontsize=6.5)
601 for level, (y0, y1) in right_pos.items():
602     color = pu.PALETTE[(level - 1) % len(pu.PALETTE)]
603     ax.add_patch(patches.Rectangle((0.83, y0), 0.05, y1 - y0, facecolor=color, edgecolor=color, alpha=0.78))
604     ax.text(0.90, (y0 + y1) / 2, f"动态{level}\n{target_totals[level]}场", ha="left", va="center", fontsize=6.5)
605 ax.set_xlim(0, 1); ax.set_ylim(0, 1); ax.set_xticks([]); ax.set_yticks([])
606 _panel_label(ax, label)
607
608
609 def fig_q3_resource_transition_sankey(d):
610     frame = pd.DataFrame(d["q3"]["resource_transitions"])
611     fig, axes = plt.subplots(1, 3, figsize=(11, 4.2))
612     for i, resource in enumerate(["转播", "安保", "交通"]):
613         _sankey_panel(axes[i], frame[frame["resource"] == resource].to_dict("records"), f"({chr(97 + i)}) {resource}")
614     pu.save_fig(fig, _output("fig_q3_resource_transition_sankey"))
615
616
617 def fig_q3_dynamic_value_dumbbell(d):
618     rows = sorted(d["q3"]["rows"], key=lambda row: row["dynamic_net_value"] - row["static_net_value"])
619     y = np.arange(len(rows))
620     static = np.asarray([row["static_net_value"] for row in rows], float)
621     dynamic = np.asarray([row["dynamic_net_value"] for row in rows], float)
622     fig, ax = plt.subplots(figsize=(8.4, 7.0))
623     for yi, left, right in zip(y, static, dynamic):
624         color = pu.PALETTE[2] if right >= left else pu.PALETTE[1]
625         ax.plot([left, right], [yi, yi], color=color, lw=1.6, alpha=0.65)
626         ax.text(max(left, right) + (dynamic.max() - static.min()) * 0.025, yi, f"right - left: +.4f",
627                va="center", fontsize=6.2, color=pu.COLORS["text"])
628     ax.scatter(static, y, s=26, color=pu.PALETTE[0], edgecolor=pu.COLORS["light"], linewidth=0.5, label="静态策略重评")
629     ax.scatter(dynamic, y, s=30, color=pu.PALETTE[1], edgecolor=pu.COLORS["light"], linewidth=0.5, label="动态重优化")
630     ax.set_yticks(y, [row["match_id"] for row in rows], fontsize=7)
631     ax.set_xlabel("逐场无量纲净效益")
632     ax.set_ylabel("第三轮比赛")
633     ax.legend(loc="best")
634     span = dynamic.max() - static.min()
635     ax.set_xlim(static.min() - 0.05 * span, dynamic.max() + 0.20 * span)
636     pu.save_fig(fig, _output("fig_q3_dynamic_value_dumbbell"))
637
638
639 def fig_q3_risk_value_bubble_kde(d):
640     frame = pd.DataFrame(d["q3"]["risk_value"])
641     x = frame["risk"].to_numpy(float); y = frame["commercial_mud"].to_numpy(float)
642     importance = frame["importance"].to_numpy(float); cost = frame["resource_cost"].to_numpy(float)
643     bins = np.quantile(cost, [0, 0.34, 0.67, 1.0])
644     group = np.clip(np.digitize(cost, bins[1:-1]), 0, 2)
645     sizes = 55 + 420 * (importance - importance.min()) / max(importance.max() - importance.min(), 1e-12)
646     fig = plt.figure(figsize=(8.7, 7.2))
647     grid = fig.add_gridspec(4, 4, hspace=0.08, wspace=0.08)
648     ax_top = fig.add_subplot(grid[0, :3])
649     ax_main = fig.add_subplot(grid[1:, :3], sharex=ax_top)

```

```

650 ax_right = fig.add_subplot(grid[1:, 3], sharey=ax_main)
651 for gidx in range(3):
652     mask = group == gidx
653     ax_main.scatter(x[mask], y[mask], s=sizes[mask], color=pu.PALETTE[gidx], alpha=0.50,
654                    edgecolor=pu.COLORS["light"], linewidth=0.7, label=f"资源成本层级{gidx + 1}")
655 xgrid = np.linspace(x.min() - 0.02, x.max() + 0.02, 180)
656 ygrid = np.linspace(y.min() - 0.2, y.max() + 0.2, 180)
657 xdensity = gaussian_kde(x)(xgrid); ydensity = gaussian_kde(y)(ygrid)
658 ax_top.fill_between(xgrid, xdensity, color=pu.PALETTE[0], alpha=0.24)
659 ax_top.plot(xgrid, xdensity, color=pu.PALETTE[0], lw=1.2)
660 ax_right.fill_betweenx(ygrid, 0, ydensity, color=pu.PALETTE[1], alpha=0.24)
661 ax_right.plot(ydensity, ygrid, color=pu.PALETTE[1], lw=1.2)
662 score = x * y
663 offsets = [(7, 8), (7, -13), (-32, 8), (-32, -13)]
664 for k, index in enumerate(np.argsort(score)[-4:]):
665     ax_main.annotate(frame.iloc[index]["match_id"], (x[index], y[index]), xytext=offsets[k],
666                    textcoords="offset points", fontsize=7, color=pu.COLORS["text"],
667                    arrowprops={"arrowstyle": "-", "color": pu.COLORS["ref_line"], "lw": 0.6})
668 ax_main.set_xlabel("风险暴露指数")
669 ax_main.set_ylabel("商业价值 (百万美元)")
670 ax_main.legend(loc="best", fontsize=7)
671 ax_top.tick_params(labelbottom=False); ax_top.set_yticks([])
672 ax_right.tick_params(labelleft=False); ax_right.set_xticks([])
673 pu.save_fig(fig, _output("fig_q3_risk_value_bubble_kde"))
674
675
676 def fig_q3_sensitivity_surface(d):
677     frame = pd.DataFrame(d["q3"] ["sensitivity_surface"])
678     xs = sorted(frame["state_coefficient_multiplier"].unique())
679     ys = sorted(frame["injury_coefficient_multiplier"].unique())
680     matrix = frame.pivot(index="injury_coefficient_multiplier", columns="state_coefficient_multiplier", values="Z3").loc[ys, xs].to_numpy(float)
681     xx, yy = np.meshgrid(xs, ys)
682     fig = plt.figure(figsize=(8.2, 6.2))
683     ax = fig.add_subplot(111, projection="3d")
684     surface = ax.plot_surface(xx, yy, matrix, cmap="YlOrRd", alpha=0.88, edgecolor=pu.COLORS["light"], linewidth=0.5)
685     ax.contour(xx, yy, matrix, zdir="z", offset=matrix.min() - 0.015, cmap="YlOrRd", levels=8)
686     index = np.unravel_index(np.argmax(matrix), matrix.shape)
687     ax.scatter([xx[index]], [yy[index]], [matrix[index]], s=70, marker="*", color=pu.PALETTE[3], edgecolor=pu.COLORS["light"])
688     ax.set_xlabel("状态差系数倍率")
689     ax.set_ylabel("伤病系数倍率")
690     ax.set_zlabel("动态目标值 Z3")
691     ax.set_zlim(matrix.min() - 0.015, matrix.max() + 0.015)
692     ax.view_init(elev=28, azim=-52)
693     cbar = fig.colorbar(surface, ax=ax, shrink=0.62, pad=0.08)
694     cbar.set_label("Z3")
695     pu.save_fig(fig, _output("fig_q3_sensitivity_surface"))
696
697
698 def fig_q4_schedule_difference(d):
699     rows = sorted(d["q4"] ["differences"], key=lambda row: row["preferred_change"])
700     labels = [METRIC_ZH[row["indicator_name"]] for row in rows]
701     values = np.asarray([row["preferred_change"] for row in rows], float)
702     y = np.arange(len(rows))
703     fig, ax = plt.subplots(figsize=(8.8, 5.2))
704     limit = max(abs(values.min()), abs(values.max())) * 1.18
705     ax.axvspan(-limit, 0, color=pu.PALETTE[1], alpha=0.07)
706     ax.axvspan(0, limit, color=pu.PALETTE[2], alpha=0.07)
707     colors = [pu.PALETTE[2] if value >= 0 else pu.PALETTE[1] for value in values]
708     bars = ax.barh(y, values, color=colors, alpha=0.32, edgecolor=colors, linewidth=1.2)
709     for bar, value in zip(bars, values):
710         ax.text(value + (0.025 * limit if value >= 0 else -0.025 * limit),
711                bar.get_y() + bar.get_height() / 2, f"{value:+.3f}",
712                ha="left" if value >= 0 else "right", va="center", fontsize=7.5, color=pu.COLORS["text"])
713     ax.axvline(0, color=pu.COLORS["ref_line"], lw=1.0)
714     ax.set_yticks(y, labels)
715     ax.set_xlim(-limit, limit)
716     ax.set_xlabel("按偏好方向统一的相对变化 (正值表示优化方案更优)")
717     ax.set_ylabel("评价指标")
718     pu.save_fig(fig, _output("fig_q4_schedule_difference"))
719
720
721 def fig_q4_metric_radar(d):
722     rows = d["q4"] ["radar"]
723     labels = [METRIC_ZH[row["indicator"]] for row in rows]
724     actual = np.asarray([row["actual"] for row in rows], float)
725     optimized = np.asarray([row["optimized"] for row in rows], float)

```

```

726 angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False)
727 closed_angles = np.r_[angles, angles[0]]
728 fig, ax = plt.subplots(figsize=(8.4, 6.8), subplot_kw={"projection": "polar"})
729 for radius in np.linspace(0.2, 1.0, 5):
730     ax.fill(closed_angles, np.full(len(closed_angles), radius), color=pu.PALETTE[4], alpha=0.018)
731 ax.plot(closed_angles, np.r_[actual, actual[0]], color=pu.PALETTE[0], lw=1.7, marker="o", markersize=4, label="2022实际赛程")
732 ax.fill(closed_angles, np.r_[actual, actual[0]], color=pu.PALETTE[0], alpha=0.12)
733 ax.plot(closed_angles, np.r_[optimized, optimized[0]], color=pu.PALETTE[1], lw=1.9, marker="s", markersize=4, label="2026优化赛程")
734 ax.fill(closed_angles, np.r_[optimized, optimized[0]], color=pu.PALETTE[1], alpha=0.13)
735 ax.set_xticks(angles, labels, fontsize=7.5)
736 ax.set_ylim(0, 1.05)
737 ax.set_yticks([0.2, 0.4, 0.6, 0.8, 1.0], ["0.2", "0.4", "0.6", "0.8", "1.0"], fontsize=7)
738 ax.legend(loc="upper right", bbox_to_anchor=(1.16, 1.12), fontsize=8)
739 fig.subplots_adjust(left=0.14, right=0.86, bottom=0.08, top=0.94)
740 pu.save_fig(fig, _output("fig_q4_metric_radar"))
741
742
743 DISPATCH = {
744     "fig_q1_target_raincloud": fig_q1_target_raincloud,
745     "fig_q1_feature_pairplot": fig_q1_feature_pairplot,
746     "fig_q1_stage_timezone_violin": fig_q1_stage_timezone_violin,
747     "fig_q1_model_accuracy_heatmap": fig_q1_model_accuracy_heatmap,
748     "fig_q1_prediction_fit": fig_q1_prediction_fit,
749     "fig_q1_residual_diagnostics": fig_q1_residual_diagnostics,
750     "fig_q1_shap_summary": fig_q1_shap_summary,
751     "fig_q2_schedule_gantt": fig_q2_schedule_gantt,
752     "fig_q2_venue_travel_map": fig_q2_venue_travel_map,
753     "fig_q2_venue_slot_matrix": fig_q2_venue_slot_matrix,
754     "fig_q2_objective_waterfall": fig_q2_objective_waterfall,
755     "fig_q2_constraint_margins_lollipop": fig_q2_constraint_margins_lollipop,
756     "fig_q2_weight_sensitivity_contour": fig_q2_weight_sensitivity_contour,
757     "fig_q3_advancement_shift": fig_q3_advancement_shift,
758     "fig_q3_resource_transition_sankey": fig_q3_resource_transition_sankey,
759     "fig_q3_dynamic_value_dumbbell": fig_q3_dynamic_value_dumbbell,
760     "fig_q3_risk_value_bubble_kde": fig_q3_risk_value_bubble_kde,
761     "fig_q3_sensitivity_surface": fig_q3_sensitivity_surface,
762     "fig_q4_schedule_difference": fig_q4_schedule_difference,
763     "fig_q4_metric_radar": fig_q4_metric_radar,
764 }
765
766
767 def generate(name: str):
768     if name not in DISPATCH:
769         raise KeyError(f"Unknown figure: {name}")
770     DISPATCH[name](_data())

```

Listing 15: 论文表格生成程序

```

1  """Generate the planned LaTeX tables and all figure/table include snippets."""
2  from __future__ import annotations
3
4  import json
5  from pathlib import Path
6
7  import numpy as np
8  import pandas as pd
9
10 ROOT = Path(__file__).resolve().parents[1]
11 FIGURES = ROOT / "figures"
12
13
14 def read_json(name):
15     return json.loads((FIGURES / name).read_text(encoding="utf-8"))
16
17
18 def esc(value):
19     if value is None:
20         return "--"
21     text = str(value)
22     replacements = {
23         "\\": r"\textbackslash{}", "&": r"&", "%": r"%", "$": r"\$",
24         "#": r"#", "_": r"\_", "{": r"\{", "(": r"\(",
25         "-": r"\textasciitilde{}", "~": r"\textasciicircum{}",
26     }

```

```

27     for old, new in replacements.items():
28         text = text.replace(old, new)
29     return text
30
31
32 def fmt(value, digits=3):
33     if value is None or (isinstance(value, float) and not np.isfinite(value)):
34         return "---"
35     if isinstance(value, (int, np.integer)):
36         return str(int(value))
37     return f"{float(value):.{digits}f}"
38
39
40 def soft_esc(value):
41     """Escape LaTeX and add legal breakpoints to long technical identifiers."""
42     return (esc(value)
43             .replace(r"\_", r"\_\\allowbreak{")
44             .replace(r"/", r"/\\allowbreak{")
45             .replace(r";", r";\\allowbreak{"))
46
47
48 def normal_table(filename, caption, label, headers, rows, align=None, resize=False, note=None):
49     align = align or ("l" + "r" * (len(headers) - 1))
50     body = [r"\begin{table}[H]", r"\centering", f"\caption{{{caption}}}", f"\label{{{label}}}", r"\small"]
51     if resize:
52         body.append(r"\resizebox{\textwidth}{!}{%")
53     body.extend([f"\begin{{{tabular}}}{{{align}}}", r"\toprule", " & ".join(headers) + r" \\", r"\midrule"])
54     body.extend(" & ".join(row) + r" \\ " for row in rows)
55     body.extend([r"\bottomrule", r"\end{tabular}"])
56     if resize:
57         body.append("%")
58     if note:
59         body.append(r"\vspace{2pt}")
60         body.append(r"\noindent\begin{minipage}{0.96\textwidth}\footnotesize 注: " + note + r"\end{minipage}")
61     body.append(r"\end{table}")
62     (FIGURES / filename).write_text("\n".join(body) + "\n", encoding="utf-8")
63
64
65 def long_table(filename, caption, label, headers, rows, align=None, note=None):
66     align = align or ("l" * len(headers))
67     header = " & ".join(headers) + r" \\ "
68     body = [f"{", r"\footnotesize", r"\huzz=0.2pt", r"\setlength{\tabcolsep}{3pt}",
69             f"\begin{{{longtable}}}{{{align}}}",
70             f"\caption{{{caption}}}\label{{{label}}}\\\",
71             r"\toprule", header, r"\midrule", r"\endfirsthead",
72             f"\multicolumn{{{len(headers)}}}{{{c}}}{{{续表}}} " + r"\\",
73             r"\toprule", header, r"\midrule", r"\endhead",
74             f"\midrule \multicolumn{{{len(headers)}}}{{{r}}}{{{续下页}}} " + r"\\", r"\endfoot",
75             r"\bottomrule", r"\endlastfoot",
76     ]
77     body.extend(" & ".join(row) + r" \\ " for row in rows)
78     body.append(r"\end{longtable}")
79     if note:
80         body.append(r"\noindent\begin{minipage}{0.96\textwidth}\footnotesize 注: " + note + r"\end{minipage}")
81     body.append(r"\par")
82     body.append("%")
83     (FIGURES / filename).write_text("\n".join(body) + "\n", encoding="utf-8")
84
85
86 def main():
87     p1 = read_json("problem_1_results.json")
88     p2 = read_json("problem_2_results.json")
89     p3 = read_json("problem_3_results.json")
90     p4 = read_json("problem_4_results.json")
91     plot = read_json("_plot_data.json")
92
93     lineage_rows = [
94         [soft_esc(row["feature_name"]), soft_esc(row["source_table"]), soft_esc(row["source_column"])],
95         [soft_esc(row["source_time_rule"]), soft_esc(row["fitted_on_ids"])]
96     ]
97     for row in p1["feature_lineage"]
98
99     long_table(
100         "TABLE_q1_data_dictionary.tex", "问题一特征字典与时间血缘", "tab:q1-data-dictionary",
101         ["特征", "来源表", "来源列", "时间规则", "拟合样本"], lineage_rows,
102         align=r"p{0.15\textwidth}p{0.16\textwidth}p{0.15\textwidth}p{0.18\textwidth}p{0.19\textwidth}",

```

```

103
104 fold_rows = []
105 for row in p1["fold_metrics"]:
106     fold_rows.append([
107         esc(row["model"]), f"折{int(row['fold'])}", fmt(row["mse"], 3), fmt(row["rmse"], 3), fmt(row["mae"], 3),
108         esc(str(row["train_end"])[10]), esc(str(row["valid_start"])[10]),
109     ])
110 for row in p1["model_summary"]:
111     fold_rows.append([esc(row["model"]), "总体均值", fmt(row["mean_mse"], 3), fmt(row["mean_rmse"], 3),
112         fmt(row["mean_mae"], 3), "--", "--"])
113 normal_table(
114     "TABLE_q1_model_metrics.tex", "问题一时序交叉验证误差", "tab:q1-model-metrics",
115     ["模型", "窗口", "MSE", "RMSE", "MAE", "训练截至", "验证起始"], fold_rows,
116     align="llrrrrr", resize=True,
117     note=f"误差单位为百万人口径; 最终选择模型为 {esc(p1['selected_model'])}。",
118 )
119
120 pair = pd.DataFrame(plot["q1"]["pairplot"])
121 desc_rows = []
122 for column in pair.columns:
123     series = pair[column].dropna().astype(float)
124     desc_rows.append([
125         esc(column), str(len(series)), fmt(series.mean(), 2), fmt(series.std(ddof=1), 2), fmt(series.min(), 2),
126         fmt(series.quantile(0.25), 2), fmt(series.median(), 2), fmt(series.quantile(0.75), 2), fmt(series.max(), 2),
127     ])
128 normal_table(
129     "TABLE_q1_descriptive_statistics.tex", "问题一核心变量描述性统计", "tab:q1-descriptive",
130     ["变量", "样本数", "均值", "标准差", "最小值", "P25", "中位数", "P75", "最大值"],
131     desc_rows, align="llrrrrrrr", resize=True,
132     note="观着人数单位为百万人; 其余变量采用源数据或赛前构造口径。",
133 )
134
135 constraint_rows = []
136 for row in plot["q2"]["constraint_margins"]:
137     constraint_rows.append([
138         esc(row["name"]), fmt(row["actual"], 3), fmt(row["limit"], 3), esc(row["unit"]),
139         fmt(row["margin_pct"], 1) + r"%", "通过" if row["margin_pct"] >= -1e-9 else "未通过",
140     ])
141 normal_table(
142     "TABLE_q2_constraint_audit.tex", "问题二关键约束复算与裕度", "tab:q2-constraint-audit",
143     ["约束族", "实际值", "界值", "单位", "最小裕度", "状态"], constraint_rows,
144     align="llrrrc", resize=False,
145     note="裕度为按各约束自身上限归一化后的最小剩余比例; 零表示约束在最紧样本上取等。",
146 )
147
148 schedule = pd.DataFrame(p2["schedule"])
149 schedule_rows = []
150 for venue, group in schedule.groupby("venue_id", sort=True):
151     schedule_rows.append([
152         esc(venue), esc(group["city"].iloc[0]), str(len(group)), str(int(group["is_prime"].sum())),
153         fmt(group["travel_cost_index"].mean(), 3), fmt(group["expected_attendance"].sum() / 10_000, 1),
154     ])
155 normal_table(
156     "TABLE_q2_schedule_summary.tex", "问题二场馆负载与赛程汇总", "tab:q2-schedule-summary",
157     ["场馆", "城市", "场次", "黄金时段场次", "平均旅行指数", "预计观众合计 (万人)"], schedule_rows,
158     align="llrrrr", resize=True,
159     note=f"主方案 Z2={p2['metrics']['Z2']:.6f}, 最小轮间休息 {p2['constraint_audit']['min_rest_hours']:.0f} 小时。",
160 )
161
162 cond = {row["match_id"]: row for row in p3["conditional_probabilities"]}
163 probability_rows = []
164 for row in p3["rows"]:
165     c = cond[row["match_id"]]
166     probability_rows.append([
167         esc(row["match_id"]), esc(row["team_a"]), fmt(row["updated_p_team_a_advance"], 3),
168         esc(row["team_b"]), fmt(row["updated_p_team_b_advance"], 3),
169         fmt(c["p_a_cond_win"], 3), fmt(c["p_b_cond_win"], 3),
170         fmt(row["stakeless_risk"], 3), fmt(row["collusion_risk"], 3),
171     ])
172 normal_table(
173     "TABLE_q3_probability_risk.tex", "问题三晋级概率、条件概率与风险", "tab:q3-probability-risk",
174     ["比赛", "A队", "A队晋级", "B队", "B队晋级", "A胜条件晋级", "B胜条件晋级", "无悬念风险", "默契风险"],
175     probability_rows, align="llrlrrrrr", resize=True,
176     note="条件晋级概率来自同一 20000 次联合蒙特卡洛样本, 晋级名额质量守恒为每次 32 队。",
177 )
178

```

```

179 budget_rows = []
180 for row in p3["daily_dynamic_audit"]:
181     caps = row["caps"]
182     budget_rows.append([
183         esc(row["date"]), str(int(row["broadcast3"])), str(int(caps["high_broadcast_capacity"])),
184         str(int(row["security3plus"])), str(int(caps["high_security_capacity"])),
185         str(int(row["transport3"])), str(int(caps["enhanced_transport_capacity"])),
186         fmt(row["budget"], 1), fmt(caps["daily_resource_budget_index"], 1), "通过" if row["pass"] else "未通过",
187     ])
188 normal_table(
189     "TABLE_q3_resource_budget.tex", "问题三逐日动态资源使用与容量", "tab:q3-resource-budget",
190     ["日期", "高转播", "转播上限", "高安保", "安保上限", "强化交通", "交通上限", "预算使用", "预算上限", "状态"],
191     budget_rows, align="lrrrrrrrrc", resize=True,
192 )
193
194 action_lookup = {
195     (row["match_id"], int(row["b"]), int(row["q"]), int(row["l"])): row
196     for row in p3["action_table"]
197 }
198 static_actions = []
199 dynamic_actions = []
200 for row in p3["rows"]:
201     s = row["static_decision"]; q = row["dynamic_decision"]
202     static_actions.append(action_lookup[(row["match_id"], int(s["b"]), int(s["q"]), int(s["l"]))])
203     dynamic_actions.append(action_lookup[(row["match_id"], int(q["b"]), int(q["q"]), int(q["l"]))])
204 static_dynamic_rows = []
205 definitions = [
206     ("票务价值 TV", "TV", "sum", "百万美元", 1_000_000),
207     ("转播价值 BV", "BV", "sum", "百万美元", 1_000_000),
208     ("吸引力 A", "A", "mean", "无量纲", 1),
209     ("资源成本 C", "C", "sum", "成本指数", 1),
210     ("风险 R", "R", "mean", "无量纲", 1),
211 ]
212 for label, key, agg, unit, divisor in definitions:
213     svals = np.asarray([row[key] for row in static_actions], float)
214     dvals = np.asarray([row[key] for row in dynamic_actions], float)
215     sval = (svals.sum() if agg == "sum" else svals.mean()) / divisor
216     dval = (dvals.sum() if agg == "sum" else dvals.mean()) / divisor
217     direction = dval - sval if key in {"TV", "BV", "A"} else sval - dval
218     static_dynamic_rows.append([esc(label), fmt(sval, 3), fmt(dval, 3), fmt(direction, 3), esc(unit)])
219 static_dynamic_rows.append(["综合目标 Z3", fmt(p3["Z3_static_reeval"], 6), fmt(p3["Z3_dynamic"], 6),
220     fmt(p3["Z3_dynamic"] - p3["Z3_static_reeval"], 6), "无量纲"])
221 normal_table(
222     "TABLE_q3_static_dynamic.tex", "问题三静态策略与动态重优化同界比较", "tab:q3-static-dynamic",
223     ["指标", "静态策略重评", "动态重优化", "偏好方向改善", "单位"], static_dynamic_rows,
224     align="lrrrr", resize=False,
225     note="正的偏好方向改善表示动态方案更优; 成本与风险按越低越优转换。",
226 )
227
228 structural_labels = {
229     "rest_min_hours": "最小休息", "rest_mean_hours": "平均休息",
230     "timezone_crossings": "跨时区次数", "timezone_crossing_rate": "跨时区率",
231     "travel_mean_km": "场馆旅行", "venue_change_rate": "换馆率",
232     "venue_utilization_rate": "场馆日利用率", "venue_daily_peak": "场馆日峰值",
233     "capacity_occupancy_rate": "容量匹配", "prime_coverage_rate": "黄金覆盖",
234     "expected_attendance_per_match": "场均预计观众",
235 }
236 percent_metrics = {"timezone_crossing_rate", "venue_change_rate", "venue_utilization_rate",
237     "capacity_occupancy_rate", "prime_coverage_rate"}
238 integer_metrics = {"timezone_crossings", "venue_daily_peak"}
239 structural_rows = []
240 for row in p4["structural_comparison"]:
241     name = row["indicator_name"]
242     spec = p4["structural_definitions"][name]
243     actual = row["actual_schedule_value"]
244     optimized = row["optimized_schedule_value"]
245     if name in percent_metrics:
246         actual_text, optimized_text = fmt(100 * actual, 4), fmt(100 * optimized, 4)
247     elif name in integer_metrics:
248         actual_text, optimized_text = fmt(int(round(actual))), fmt(int(round(optimized)))
249     elif name == "expected_attendance_per_match":
250         actual_text, optimized_text = fmt(actual, 2), fmt(optimized, 2)
251     else:
252         actual_text, optimized_text = fmt(actual, 4), fmt(optimized, 4)
253     structural_rows.append([
254         structural_labels[name], esc(spec["definition"]), esc(spec["unit"]),

```

```

255     actual_text, optimized_text,
256     "越高越优" if spec["preferred_direction"] == "higher" else "越低越优",
257 )
258 normal_table(
259     "TABLE_q4_structural_comparison.tex", "两套赛程的可观测结构与规模强度比较",
260     "tab:q4-structural-comparison",
261     ["指标", "定义与分母", "单位", "2022实际", "2026优化", "偏好方向"], structural_rows,
262     align="lllrrl", resize=True,
263     note="休息、时区、场馆路径与场馆日由逐场赛程直接复算；黄金覆盖来自唯一时段映射，容量匹配和预计观众的需求端为题内赛前代理。实际赛程有64次球队转场，优化赛程有96次。",
264 )
265
266 formula = {
267     "T": r"$T=\mathrm{norm}(\sum \mathrm{ticket})$", "B": r"$B=\mathrm{norm}(\sum \mathrm{broadcast})$",
268     "U": r"$U=\overline{\mathrm{uncertainty}}$", "H": r"$H=\overline{\mathrm{attractiveness}}/100$",
269     "C": r"$C=\mathrm{norm}(\mathrm{setup+operation+security})$", "D": r"$D=\overline{\mathrm{travel}}$",
270     "F": r"$F=(\Delta \text{prime}+\Delta \text{large})/6$", "R": r"$R=\overline{\mathrm{risk}}$",
271     "Z4": r"$Z4=0.25T+0.25B+0.15U+0.10H-0.08C-0.07D-0.06F-0.04R$",
272 }
273
274 definition_rows = []
275 for row in p4["comparison"]:
276     name = row["indicator_name"]
277     direction = "越高越优" if row["preferred_direction"] == "higher" else "越低越优"
278     if name in p4["structural_definitions"]:
279         spec = p4["structural_definitions"][name]
280         evidence = {"direct": "现实赛程直接复算", "proxy_demand": "需求端为题内代理",
281                    "time_mapping": "赛事进度与UTC时刻映射"}[spec["evidence"]]
282         definition_rows.append([soft_esc(name), esc(spec["definition"]), esc(spec["unit"]), direction,
283                                "统一结构评价函数", evidence])
284     else:
285         note = "共同包络Min--Max" if name in {"T", "B", "C"} else ("条件性加权总分" if name == "Z4" else "题内代理评价")
286         definition_rows.append([soft_esc(name), formula[name], "无量纲", direction, "统一代理评价函数", note])
287
288 long_table(
289     "TABLE_q4_metric_definition.tex", "问题四评价指标定义与可比口径", "tab:q4-metric-definition",
290     ["指标", "定义", "单位", "偏好方向", "数据源", "口径说明"], definition_rows,
291     align="t@{p{0.08\textwidth}p{0.26\textwidth}p{0.08\textwidth}p{0.11\textwidth}p{0.18\textwidth}p{0.209\textwidth}@{}}",
292     note=esc(p4["proxy_note"]),
293 )
294
295 captions = {
296     "fig_q1_target_raincloud": ("训练集观看人数分布与长尾特征", 0.82),
297     "fig_q1_feature_pairplot": ("问题一核心赛前特征成对关系", 0.88),
298     "fig_q1_stage_timezone_violin": ("赛事阶段与时区组合下观看人数异质性", 0.96),
299     "fig_q1_model_accuracy_heatmap": ("候选模型时序验证误差比较", 0.90),
300     "fig_q1_prediction_fit": ("最终模型OOF预测拟合与残差分布", 0.94),
301     "fig_q1_residual_diagnostics": ("最终模型残差诊断四联图", 0.92),
302     "fig_q1_shap_summary": ("ElasticNet置换重要性与单样本预测影响", 0.96),
303     "fig_q2_schedule_gantt": ("72场小组赛UTC排程甘特图", 0.98),
304     "fig_q2_venue_travel_map": ("场馆负载与球队跨轮旅行网络", 0.92),
305     "fig_q2_venue_slot_matrix": ("场馆与参考日期占用矩阵", 0.98),
306     "fig_q2_objective_waterfall": ("问题二目标函数八项贡献分解", 0.90),
307     "fig_q2_constraint_margins_lollipop": ("问题二最紧约束归一化裕度", 0.86),
308     "fig_q2_weight_sensitivity_contour": ("商业与惩罚权重联合灵敏度", 0.78),
309     "fig_q3_advancement_shift": ("更新前后48队晋级概率变化", 0.82),
310     "fig_q3_resource_transition_sankey": ("静态策略到动态策略的资源等级流转", 0.96),
311     "fig_q3_dynamic_value_dumbbell": ("24场静态与动态净效益配对比较", 0.86),
312     "fig_q3_risk_value_bubble_kde": ("风险暴露与商业价值联合分布", 0.82),
313     "fig_q3_sensitivity_surface": ("状态差与伤病系数下的动态目标曲面", 0.76),
314     "fig_q4_schedule_difference": ("按偏好方向统一的两方案指标差异", 0.88),
315     "fig_q4_metric_radar": ("实际赛程与优化赛程多维偏好得分", 0.72),
316 }
317
318 includes = []
319 for name, (caption, width) in captions.items():
320     label = "fig:" + name.removeprefix("fig_").replace("_", "-")
321     includes.extend([
322         r"\begin{figure}[H]", r"\centering",
323         f"\includegraphics[width={width:.2f}\textwidth]{{figures/{name}.pdf}}",
324         f"\caption{{{caption}}}", f"\label{{{label}}}", r"\end{figure}", "",
325     ])
326
327 for filename in [
328     "TABLE_q1_data_dictionary.tex", "TABLE_q1_model_metrics.tex", "TABLE_q1_descriptive_statistics.tex",
329     "TABLE_q2_constraint_audit.tex", "TABLE_q2_schedule_summary.tex", "TABLE_q3_probability_risk.tex",
330     "TABLE_q3_resource_budget.tex", "TABLE_q3_static_dynamic.tex", "TABLE_q4_structural_comparison.tex",
331     "TABLE_q4_metric_definition.tex",
332 ]:
333     includes.append(f"\input{{figures/{filename}}}")

```

```
330     (FIGURES / "latex_includes.tex").write_text("\n".join(includes) + "\n", encoding="utf-8")
331     print("Generated 10 tables and figures/latex_includes.tex")
332
333
334 if __name__ == "__main__":
335     main()
```